OxideMerge User Guide

Version 0.9.8

Table of Contents

1. Introduction . . . 2
   1.1. What OxideMerge does . . . 2
   1.2. How to use this guide . . . 2
   1.3. Key terms . . . 2
2. Installation . . . 4
   2.1. Windows . . . 4
   2.2. macOS . . . 4
   2.3. Linux . . . 4
   2.4. Building from source . . . 4
3. Getting started . . . 6
   3.1. The start page . . . 6
   3.2. Your first comparison . . . 7
   3.3. The main window . . . 7
   3.4. Opening a comparison . . . 8
   3.5. Comparing over SFTP or FTP . . . 9
   3.6. Comparison types . . . 14
   3.7. Choosing a workflow . . . 16
   3.8. Getting help . . . 16
4. Comparing text files . . . 17
   4.1. The text comparison window . . . 17
   4.2. Moving between differences . . . 18
   4.3. Copying differences . . . 19
   4.4. Saving files . . . 19
   4.5. Editing text . . . 20
   4.6. Find and replace . . . 23
   4.7. Bookmarks . . . 23
   4.8. Synchronization points . . . 23
   4.9. View options . . . 24
   4.10. CSV and TSV files . . . 25
   4.11. Comparing with a revision . . . 28
5. Merging files and resolving conflicts . . . 29
   5.1. Three-way merge . . . 29
   5.2. Resolving conflict files . . . 30
6. Comparing folders . . . 32
   6.1. The folder list . . . 32
   6.2. Display options . . . 33
   6.3. The toolbar . . . 34
   6.4. Comparison methods . . . 35

6.5. Selecting and opening items . . . 35
6.6. Copying and deleting items . . . 36
6.7. Reviewing all differences . . . 36
6.8. Merging folders automatically . . . 38
6.9. Synchronizing two folders . . . 39
6.10. Documents, spreadsheets and programs . . . 42
6.11. Archives . . . 44
7. Comparing binary files . . . 46
7.1. Moved blocks . . . 46
7.2. Editing bytes . . . 47
8. Comparing images . . . 48
9. Comparing web pages . . . 50
9.1. Comparison modes . . . 50
9.2. The Formatted mode . . . 50
9.3. How pages are loaded . . . 52
9.4. Resource tree limits . . . 52
9.5. Comparing web pages in the console . . . 52
10. Controlling which differences are shown . . . 54
10.1. Comparison options . . . 54
10.2. Compare rules . . . 55
10.3. Creating a rule from a text comparison . . . 60
10.4. Filters . . . 60
10.5. Unpacker plugins . . . 60
11. Saving, sharing and reporting . . . 62
11.1. Projects . . . 62
11.2. Workspaces . . . 63
11.3. Saved comparisons . . . 63
11.4. Folder reviews . . . 63
11.5. HTML reports . . . 64
11.6. Other reports . . . 65
11.7. Slideshows . . . 66
11.8. PDF reports and printing . . . 66
11.9. Patches . . . 68
12. Analysis tools . . . 69
12.1. Merge verification . . . 69
12.2. Semantic comparison . . . 70
12.3. Review workspace . . . 77
12.4. Release summary / Compare baselines . . . 78
12.5. Potential impact . . . 80
12.6. Limits . . . 81
12.7. Running analyses from the command line . . . 81

13. Command line . . . 83
13.1. Opening comparisons . . . 83
13.2. Options . . . 83
13.3. Scripting and exit codes . . . 87
13.4. Jobs . . . 88
14. Integrating with other tools . . . 92
14.1. Git . . . 92
14.2. Subversion, Mercurial and Perforce (experimental) . . . 92
14.3. File manager . . . 101
15. Options and configuration . . . 102
15.1. Application options . . . 102
15.2. Configuration files . . . 103
15.3. Portable and separate configurations . . . 103
15.4. Adding a language . . . 103
16. Keyboard shortcuts . . . 104
17. Troubleshooting . . . 106
17.1. Installation and startup . . . 106
17.2. Editing and saving . . . 106
17.3. Comparison results . . . 107
17.4. Web pages . . . 107
17.5. Performance . . . 108
18. Known limitations . . . 109

OxideMerge is a visual tool for comparing and merging files and folders. It runs on Windows, macOS and Linux. Use it to review changes, combine edits and resolve merge conflicts. It also compares binary files, images and web pages. OxideMerge is written in Rust.

This guide is also available in 繁體中文.

Chapter 1. Introduction

1.1. What OxideMerge does

- Compare two or three folders, including archives. Copy, delete or merge the items.
- Compare and edit text files. Copy changes between versions and resolve version-control conflicts.
- Compare the bytes of binary files, the pixels of images and the content of web pages.
- Hide differences that do not matter with comparison options and reusable compare rules.
- Export reports and patches. Save projects and comparisons for a later review.
- Run comparisons from scripts, and use OxideMerge as the diff and merge tool for Git.

The interface is available in English, Traditional Chinese, Simplified Chinese, Japanese, Korean, German, French, Spanish, Russian, Brazilian Portuguese and Italian.

1.2. How to use this guide

Start with Your first comparison for a short walkthrough. Then read the chapter for the kind of comparison you need. The chapters at the end are reference material: the command line, options, keyboard shortcuts and troubleshooting.

This guide uses these conventions:

- **Bold text** is a name that you see in the program, such as a button or a menu command.
- A menu path such as **File** → **Open...** means: open the **File** menu, then choose **Open...**.
- Command-line examples assume that `oxidemerge` is in your `PATH`. If it is not, use the full path of the program.
- Replace example paths, such as `old-folder`, with your own paths.



Keyboard shortcuts on macOS. This guide uses the key names of Windows and Linux. On macOS, press Command (`⌘`) instead of `Ctrl`, and Option (`⌥`) instead of `Alt`. The one exception is `Ctrl` + `Tab`, which uses the Control key on macOS too. Menus and tooltips always show the shortcuts for your system.

1.3. Key terms

Side

One version in a comparison: the left, middle or right version.

Pane

The area of the window that shows one side.

Difference

A place where the versions are not the same. In a text comparison, one difference can include

several lines.

Base (common ancestor)

The original version that two edited versions came from. In a three-way comparison, the base is in the middle.

Conflict

A place where both edited versions changed the same text in different ways. You must choose or edit the result yourself.

Merge

To copy or combine changes into a destination version.

Filter

A rule that selects which files and folders a folder comparison includes.

Compare rule

A rule that changes which text differences count. It does not change the files.



A merge changes its destination. Before you copy or save, check the source, the destination and the output path. Make a side read-only if you only need to look at it.

Chapter 2. Installation

Download the package for your system and processor from GitHub Releases. Each release also includes this guide as a PDF file, in English and in Traditional Chinese. The release page shows the SHA-256 digest of each download, so you can check the file.

System	Package
Windows 10 or 11, x64 or Arm64	`oxidemerge-<version>-windows-x64.zip` or `oxidemerge-<version>-windows-arm64.zip`
macOS 11 or later	`oxidemerge-<version>-macos-universal.dmg`, for Apple silicon and Intel
Linux x64 or Arm64	An AppImage (for Ubuntu 20.04, RHEL 8, Debian 10 and later) or a TAR.GZ archive. Choose `linux-x64` for Intel and AMD processors, or `linux-arm64` for 64-bit Arm.

2.1. Windows

Extract the ZIP file and run `oxidemerge.exe`. There is nothing to install.

2.2. macOS

1. Open the DMG file and drag **OxideMerge** to **Applications**.
2. Start OxideMerge. The app is signed ad hoc and is not notarized by Apple, so macOS can block the first start. If this happens, choose **Open Anyway** in **System Settings** → **Privacy & Security**. On macOS 14 and earlier, you can also Control-click the app and choose **Open**.

To start OxideMerge from Terminal, use the full path of the program. To use the short command `oxidemerge`, create a link in a folder that is in your `PATH`. For example:

```
sudo ln -s /Applications/OxideMerge.app/Contents/MacOS/oxidemerge
/usr/local/bin/oxidemerge
```

2.3. Linux

- Use a desktop session with X11 or Wayland.
- Make the AppImage executable before you start it: `chmod +x oxidemerge-*.AppImage`.
- If your system does not have FUSE, start the AppImage with `--appimage-extract-and-run`.
- File dialogs need `xdg-desktop-portal`. Most desktop environments include it.
- To show Chinese, Japanese and Korean text, install a CJK font, such as Noto Sans CJK.

2.4. Building from source

1. Install a Rust toolchain from https://rustup.rs/.

2. In the repository folder, run `cargo build --release --locked`.

The program is `target/release/oxidemerge` (`target/release/oxidemerge.exe` on Windows). For the requirements and checks, see the development guide.

Chapter 3. Getting started

3.1. The start page

When you start OxideMerge without a comparison, it shows the start page.

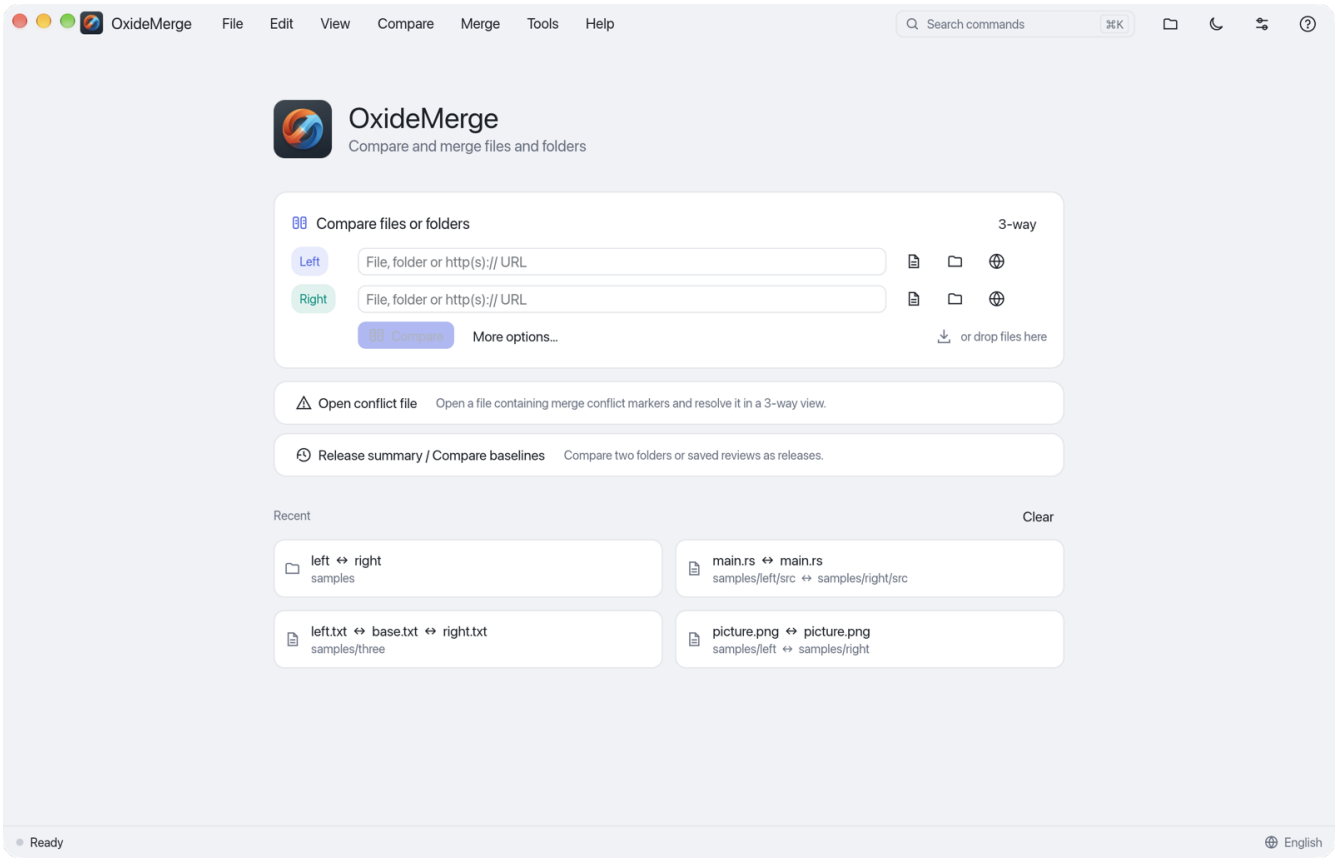


Figure 1. The start page

The start page has these parts:

Compare files or folders

Enter a **Left** path and a **Right** path. You can type a path, paste it, or click the file and folder buttons next to the field. Click **3-way** to add a **Middle** path. Click **More options…** to open the **Select files or folders** dialog with these paths.

Open conflict file

Resolve a file that contains conflict markers. See Resolving conflict files.

Release summary / Compare baselines

Compare two folders or saved reviews as two releases. See Release summary / Compare baselines.

Recent

Your last comparisons, shown as cards.

- Click a card to run that comparison again.
- Right-click a card to change its paths (**Change paths…**) or remove it (**Remove from Recent**).

- Click **Clear**, at the right of the **Recent** heading, to empty the list. **File** → **Open recent** → **Clear recent comparisons** does the same. A message then offers **Undo** for a few seconds.

When the recent list is empty, for example on the first start, the start page shows **Open a sample**. This button opens two small sample folders, so you can see how comparisons work. The sample folders are temporary: OxideMerge removes them when you close the tab, and does not add the sample to the recent list.

3.2. Your first comparison

1. On the start page, enter an older file as **Left** and a newer file as **Right**.
2. Click **Compare**. To protect a side from changes, click **More options…** instead and turn on **Read-only** for that side.
3. Press `Alt` + `↓` to go to the next difference.
4. To copy a difference, check which pane is the destination. Then click the arrow that points to that pane.
5. Check the result. To save it as a new file, choose **File** → **Save as**. To update the file in the pane, choose **Save**.

To review files without changing them, make both sides read-only. To share your review without sending the files, export an HTML report (see Saving, sharing and reporting).

3.3. The main window

The window has these parts:

Menu bar

File, **Edit**, **View**, **Compare**, **Merge**, **Tools** and **Help**. Commands that do not apply to the current tab are unavailable.

Title bar

At the right end: **Search commands**, which opens the command palette, **Open**, a switch for the light and dark themes, **Options** and **Help & Shortcuts**. In a narrow window, the search field becomes an icon, and buttons that do not fit are hidden. The menus have the same commands.

Tabs

Each comparison opens in its own tab. If both sides have the same file name, the tab shows the name once and then the folders that are different. A dot in place of the close button means that the tab has unsaved changes. Move the pointer over the dot to close the tab.

Status bar

Shows whether the sides are different and the current line or offset. The interface language is at the right end. The numbers of differences and items are shown in the toolbar and in the folder chips.

To work with tabs:

- `Ctrl` + `Tab` and `Ctrl` + `Shift` + `Tab` go to the next and the previous tab.
- `Ctrl` + `W` or `Esc` closes the current tab. If a dialog, search bar, preview or editor is open, `Esc` closes it first. If the tab has unsaved changes, OxideMerge asks before it closes the tab.
- Right-click a tab to close it, the other tabs, the tabs to its right, or all tabs.

To change the interface language, use **Options** → **General**, or start OxideMerge with `--lang`.

To see the version, click the app logo or choose **Help** → **About OxideMerge**. The dialog also shows a build identifier:

- A development build shows a short Git hash. `-dirty` is added when tracked source files had uncommitted changes.
- A clean optimized build of a release tag shows its build date (UTC).

`oxidemerge --version` prints the same identifier.

3.4. Opening a comparison

You can start a comparison in several ways:

- On the start page, enter the paths under **Compare files or folders** and click **Compare**.
- Choose **File** → **Open...** (`Ctrl` + `O`), or click **More options...** on the start page. This opens the **Select files or folders** dialog.
- Drop two or three files or folders on the window. If you drop one item, the dialog opens with that item filled in.
- Choose a recent comparison on the start page, in **File** → **Open recent** or in the command palette.
- Start OxideMerge from the command line (see Command line) or from your file manager (see File manager).
- Choose **File** → **Compare over SFTP/FTP...**, or click the globe button beside a path on the start page or in the **Select files or folders** dialog, to fetch a file or folder from a server over SFTP, FTPS or FTP first. See Comparing over SFTP or FTP.

To use the **Select files or folders** dialog:

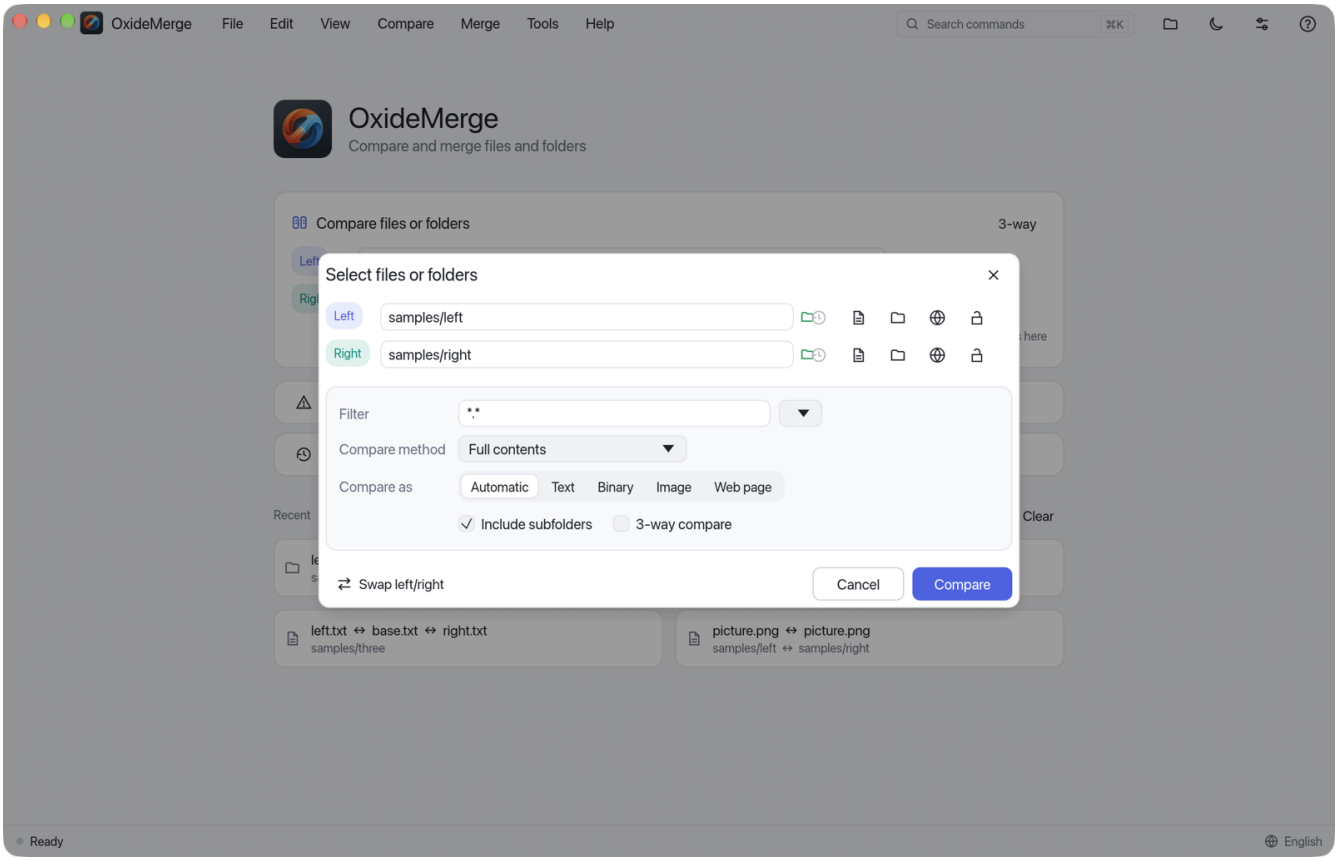


Figure 2. The Select files or folders dialog

1. Enter or browse for the **Left** and **Right** paths. A path can be a file, a folder, or an address that starts with `http://` or `https://`. To compare three versions, turn on **3-way compare** and enter a **Middle** path.
2. Turn on **Read-only** next to a side to protect it from changes. **Swap left/right** exchanges the two sides.
3. For folders, choose **Include subfolders**, a **Filter** and a **Compare method**.
4. In **Compare as**, choose the type of comparison: text, binary, image or web page. **Automatic** chooses the type for you (see Comparison types).
5. Click **Compare**.



To compare text that is not in a file, choose **File → New text compare** (`Ctrl` + `N`). This opens two empty sides. Paste text with `Ctrl` + `V`, or double-click a line to type.



To compare the clipboard's text against something you are about to paste, choose **File → Compare clipboard**. This opens a text compare with the clipboard's current text on the left and a blank, writable right side; paste the second text with `Ctrl` + `V`. **Save** works like any other unsaved comparison, since there is no file to write back to.

3.5. Comparing over SFTP or FTP

File → Compare over SFTP/FTP…, and the globe button beside a path on the start page or in the **Select files or folders** dialog, connect to a server to fetch a file or folder to compare, instead of

browsing the local disk.

1. Choose the **Protocol**: **SFTP (over SSH)**; **FTPS (explicit TLS)**, which connects to the FTP port (21) and switches to TLS before signing in; **FTPS (implicit TLS, port 990)**, for servers that expect TLS from the start; or **FTP (not encrypted)**. Changing the protocol changes the **Port** to that protocol's usual one, unless you typed another.
2. Enter the server's **Host**, **Port** and **User**. For SFTP, the host can be a `Host` alias of your `~/.ssh/config`, and the user can be left empty, as with `ssh` (see SSH configuration, jump hosts and sign-in questions).
3. For SFTP, choose how to sign in: **Password**, **Private key** (which can have a **Passphrase**), or **SSH agent**, which uses the keys `ssh-agent` holds (Pageant on Windows) and those `~/.ssh/config` names, so nothing is typed here. FTP and FTPS sign in with a password.
4. If needed, open **Advanced** for a proxy, FTP's active mode or an FTPS client certificate (see Proxies, active mode and client certificates).
5. Enter the **Remote path** of the file or folder to fetch, or click the folder button beside it to choose it from a list of the server's folders and files.
6. Click **Connect**.

The folder button connects and lists the folder of the **Remote path**, or your home folder on the server if it is empty. Folders come first, then files with their sizes; hidden entries, whose names start with a dot, come last in each group. Click a folder to open it, or a file to choose it: the **Remote path** follows what you click, so **Connect** fetches the folder you are in or the file you chose. Double-click a file to fetch it straight away. The up-arrow button opens the folder above, and the refresh button lists the **Remote path** again, including one you typed. A path that does not exist lists the nearest folder above it. The list stays connected while it is shown, so going through folders does not sign in again each time. It closes when the dialog closes, or when you change the host, port, user, password or key. A folder shows its first 10,000 entries.

The fetched path then fills in the **Left**, **Middle** or **Right** path, the same as a local one. Connect again, for the other side, to compare two remote locations with each other, or leave the other side local to compare a remote path against a local one. The dialog starts from the host, port, user and key path of the last connection that succeeded, for fetching a second path from the same server; the password or a key's passphrase is never filled in this way.

A fetched **file** is writable: edit it and **Save** uploads it back to the server over a fresh connection in the background. The upload updates the file in place to preserve its owner, group and access controls. If the server's path is a link, the file it points to is updated. **Cancel** can stop an upload before it starts writing. A connection failure during the write can leave partial contents on the server; the message says so, and **Save** again finishes writing the file. If the server's file changed or disappeared after you fetched it, the upload is refused before writing. Your edits stay in the tab, and you can compare them with the server's version. Until an upload succeeds the tab counts as unsaved, so closing it or quitting asks first, and **Save** tries again. Closing a tab with **Save** waits for its upload to finish before it closes.

A fetched **folder** is writable too. Copy or delete files in the folder comparison, or open a file from it and edit it, and **Save** writes what changed back to the server: files that are new or whose contents changed are uploaded, folders that are new are created, and files and folders that were removed are deleted on the server. Files you did not change are not sent again. Before each upload or deletion

OxideMerge checks that the server's file is still the one it fetched: a file changed on the server meanwhile is neither overwritten nor deleted, and a new file whose name was created on the server meanwhile is not overwritten. A file changed within a few seconds before it was fetched is also checked by its contents, since a second change in the same second would leave its size and time unchanged. The message after the save lists those files; your copies stay in the tab, so you can compare them with the server's and save again. The rest is written as usual. Until every change is written, the tab counts as unsaved. If the folder's local copy is gone, or part of it can't be read, nothing is written, so a file that couldn't be read is never deleted on the server.

Files are uploaded before the removed ones are deleted, so a file you moved to another folder is deleted from its old place only once its new copy is on the server: if that upload fails, the old copy stays. A symbolic link or special file in the fetched copy, or a name that isn't valid Unicode, is never uploaded and never counts as removed; the server's file stays as it is, and the message after the save names it.

A fetch is limited to 4 GiB, 50,000 files and folders, and folders nested 64 deep, so a very large folder cannot be fetched by mistake. Names that can't be file names on this computer are left out, and so is the second of two names this computer can't keep apart, such as `A.txt` and `a.txt` where letter case doesn't count. No `.bak` backups are made inside a fetched copy, since saving would upload them.

File → Reload from server fetches the server's copy of the current tab's remote file or folder again, over a fresh connection, and shows it in place of the copy fetched before. It asks first when that would replace changes not saved to the server yet; edits of a remote file are then replaced (**Undo** brings them back), and edits of a local side are kept.

On FTP and FTPS, OxideMerge tells whether a file changed on the server by its size and the modification time the server reports; for a server that reports no modification time, it reads the file and compares its contents instead. Servers that can't list folders in the standard machine-readable way (`MLSD`) are read from their plain listing instead, and links are followed to the file or folder they point to. Transfers use passive mode unless **Active mode** is set (see Proxies, active mode and client certificates), and always connect to the server the sign-in went to.

Over FTPS, a download whose connection ended without TLS's closing message, which anyone in between could cause, is checked against the size the server reports, and refused if it came short. Many FTPS servers, such as ProFTPD and vsftpd, require each transfer to resume the TLS session of the sign-in; each of the session tickets a server gives for that is good for one transfer, so when they run out OxideMerge signs in again before the next transfer.



Plain FTP sends your password and your files unencrypted: anyone on the network between you and the server can read them, and the dialog says so. Use FTPS or SFTP whenever the server offers them.

The connection, including the password or the key's passphrase, is kept only in memory for as long as the tab stays open, and the password is overwritten in memory once it is no longer needed. It is never written to the settings file. Tabs showing a fetched file or folder are left out of saved workspaces (see Workspaces), the session OxideMerge reopens on its own at startup, and the recent paths, since their temporary copies are gone after the app closes: connect again to compare the same remote path.

Before the password or key is sent, OxideMerge checks the server's host key. It first reads your own SSH `~/.ssh/known_hosts` the way `ssh` does: host patterns such as `.example.com` or `!host.example.com`, in any letter case, and names hashed by `ssh-keygen -H` all match, and a server on another port than 22 is found under `[host]:port`. A server listed there with the same key is trusted if it also matches any key saved by OxideMerge. A server listed there with a different key of the same type, or whose key is marked `@revoked` there, is refused, as ssh refuses it: if the server was reinstalled, remove its old key with `ssh-keygen -R host` (or `ssh-keygen -R "[host]:port"`) and connect again. OxideMerge also checks its own known_hosts file next to its settings file. The first time you connect to a server found in neither, OxideMerge shows the key's type and fingerprint (SHA256:…, as `ssh` shows it) and asks whether to trust it: compare it with the fingerprint the server's administrator gives you before you click *Trust and connect*. A warning is shown when your `~/.ssh/known_hosts` lists only keys of other types for that server.

A server can also prove who it is with a host certificate, signed by an authority your `~/.ssh/known_hosts` names in a `@cert-authority` line, such as `@cert-authority *.example.com ssh-ed25519 AAAA…`. OxideMerge asks for a certificate only from a server such a line names; it trusts the certificate when that authority signed it, it names the server (or its `HostKeyAlias`) among its principals, it is valid now, and it has no critical options, without asking you first. A certificate that fails any of these is refused, and so is one whose authority or key is marked `@revoked`. Certificates signed with Ed25519, ECDSA (P-256 or P-384) or RSA with SHA-2 are checked; an authority using another algorithm, such as RSA with SHA-1 or ECDSA P-521, is refused. A certificate from a server no authority is named for is checked like its plain key.

On every later connection, the key is checked against the one remembered: if it no longer matches, OxideMerge refuses the connection rather than trusting a server that may not be the one you expect. This usually means the server was rebuilt or reinstalled. If you are sure that is what happened, remove that server's old entries from OxideMerge's `known_hosts` file (`host:port` followed by its key) and connect again. If either `known_hosts` file exists but can't be read, or holds a line OxideMerge can't read (any line of its own file, or a line of `~/.ssh/known_hosts` for a key type it checks), OxideMerge connects to no server until it can.

FTPS servers are checked the way a web browser checks a website: their certificate must be valid for the host name and vouched for by a well-known certificate authority. Many FTP servers have a self-signed certificate instead. The first time you connect to one, OxideMerge shows the certificate's fingerprint and asks whether to trust it, as for an unknown SSH host key; once trusted, the certificate is remembered in the `known_certs` file next to the settings file (`oxidemerge-known_certs` in portable use), and a later connection that shows another certificate is refused. Remove the server's line (`host:port` followed by the certificate) from that file if the server's certificate was renewed. In both cases the certificate is checked before the password is sent, and a server that does not offer TLS is refused for FTPS rather than signed in to without it.

3.5.1. SSH configuration, jump hosts and sign-in questions

An SFTP server is reached the way `ssh` reaches it, as your `~/.ssh/config` (and the system's `/etc/ssh/ssh_config`) says. A **Host** such as `prod` can be an alias there, and its block can set the `HostName`, `User`, `Port`, `IdentityFile` (with `IdentitiesOnly`), `HostKeyAlias` (the name its key is looked up and certified under) and `UserKnownHostsFile`. A `#` starting a word begins a comment. `Include` is read in place, skipping files whose names start with a dot unless the pattern's file name does too, and `Match` blocks on `host` (the `HostName` set so far, as in `ssh`), `originalhost`, `user`, `localuser` or `all` apply; `Match exec` is never run. As in `ssh`, the first value found wins, and a user, or a port other than 22, typed in

the dialog wins over the file; in an address on the command line, any port written wins, `:22` included. Without a user, the file's `User` or your own user name is used.

A server reached through jump hosts (`ProxyJump bastion` or `ProxyJump user@hop1,hop2:2222`) is connected to through each in turn, up to 8, as `ssh -J` does; a jump host signs in with the SSH agent's keys and the keys its own block names, and its key is checked like the server's, before anything is sent to it. A `ProxyCommand` is run as `ssh` runs it, by the shell, with `%h`, `%p`, `%r` and `%n` filled in, and the connection goes through it. As in OpenSSH, a host or user name with characters the shell would act on (quotes, `$`, `;`, `|`, spaces and the like), or starting with `-`, is refused before any command runs.

A server can ask questions while signing in (**keyboard-interactive** sign-in): a one-time code, a second factor, a new password. OxideMerge shows them in a **Sign-in question** window naming the server, the answers hidden unless the server says they may be shown; **Cancel** declines. The password you gave answers a server's single password question by itself. A private key that is encrypted and was given without a passphrase has its passphrase asked for the same way; when its `.pub` file is beside it, only after the server has said it would take the key. A password the server refused is not sent again. Ways of signing in are tried in turn, starting with the one chosen, for servers that need more than one.

3.5.2. Proxies, active mode and client certificates

Advanced in the connection dialog holds settings most servers don't need:

- **Proxy**: connect through a SOCKS5 proxy (`socks5://host:1080`, or `socks5://user@host:1080` with a **Proxy password**) or an HTTP proxy that allows `CONNECT` (`http://host:8080`). A password typed into the address moves to **Proxy password**, where it is hidden, once you leave the field. Both SFTP and FTP use it, every FTP data connection included. A jump host or a `ProxyCommand` from `~/.ssh/config` is used instead of it for the servers it applies to; the first jump host is reached through it.
- **Active mode** (FTP): the server connects back to this computer for each transfer (`EPRT`, or `PORT` for a server without it), for servers that refuse passive mode. A data connection is accepted only from the server's address. Active mode can't go through a proxy, and usually not from behind a router that translates addresses (NAT) or a firewall that blocks incoming connections.
- **Client certificate** and **Certificate key** (FTPS): a certificate to present to a server that asks for one, in PEM, with its private key in the same file or in the key file. The key must not be encrypted.

The dialog remembers the proxy's address, active mode and the certificate's files with the last connection that succeeded, never the proxy password.

3.5.3. Server paths on the command line and in jobs

On the command line and in job files (see Jobs), a path can also be a file or folder on a server, written as an address:

```
oxidemerge sftp://ann@build.example.com/var/www/site ./site     # SFTP, port 22
oxidemerge ftpes://ann@ftp.example.com/public/index.html index.html  # explicit FTPS,
port 21
oxidemerge ftps://ann@ftp.example.com:990/public ./public -r    # implicit FTPS
```

```
oxidemerge --cli ftp://ftp.example.com/pub/README README        # anonymous plain FTP
oxidemerge sftp://ann@host/~/notes.txt notes.txt               # in ann's home folder
```

The address is `scheme://[user@]host[:port]/path`. An IPv6 address goes in brackets, as in `sftp://ann@[2001:db8::1]/srv`. A path starting with `/~/` is in the user's home folder on the server, and other characters can be written as `%XX`. For SFTP, the host can be an alias of `~/.ssh/config` (see SSH configuration, jump hosts and sign-in questions), and without a user name, its `User` or your own user name is used; FTP without one signs in as `anonymous`, with `anonymous@` as the password, never the one in `OXIDEMERGE_REMOTE_PASSWORD`.

A password never goes in the address, where anyone who can see the command line or the job file could read it; one there is refused. Put it in the `OXIDEMERGE_REMOTE_PASSWORD` environment variable instead. SFTP signs in with `--ssh-key FILE` when given (the variable then holds the key's passphrase, if it has one), else with the password in the variable, else with `ssh-agent` and the keys `~/.ssh/config` names. FTP and FTPS need the password, unless they sign in as `anonymous`. OxideMerge removes the variable when it starts, so the programs it runs, such as an external tool, don't inherit it. Questions a server asks while signing in, and the passphrase of an encrypted key given without one, are asked on the terminal, without echoing the answer (**Ctrl+C** there turns the echo back on as it stops the program); without a terminal they are declined.

`--proxy URL` connects through a proxy, as **Proxy** does in the dialog; a proxy password is read from the `OXIDEMERGE_PROXY_PASSWORD` environment variable, also removed at startup, and one in the address is refused. `--ftp-active` uses FTP's active mode. `--client-cert FILE` presents an FTPS client certificate, with its key from `--client-key FILE` when not in the same file.

Servers are checked as in the program, against the same `known_hosts` and `known_certs` files. A server not trusted yet is refused, and the error shows its fingerprint. Compare it with the one its administrator gives you, then run again with `--trust-fingerprint SHA256:…` to trust it; it is then remembered, as when you click **Trust and connect**. A server whose key or certificate changed is refused.

Each server path is fetched to a temporary copy before the comparison and opened like a local path. In the window, **Save** writes changes back to the server as described above. Without a window, what the run changed in a fetched copy, as a `sync` job does in its destination, is written back to the server after the run, with the same checks; a file the server changed meanwhile is reported as an error (exit code 2). A synchronization that stopped part way without undoing all it changed (exit code 5) is undone in the fetched copy instead, and nothing is written back. A sync plan made with `--sync-plan` names a server folder by its address, so `--sync-apply` applies it to the copy that run fetches; as for local folders, it is refused if the files it plans to change are no longer as planned.

3.6. Comparison types

OxideMerge has five types of comparison:

Type	What it shows
Text	Lines side by side, with the changed words highlighted. You can edit and merge the text. Comparison options and compare rules decide which changes count as differences. Use it for source code, configuration files, CSV files and any other readable text.
Binary	The bytes in a hexadecimal view. You can copy and save differences, but the lines are not aligned. For example, one byte inserted near the start moves all the bytes after it.
Image	The pixels: side by side, as an overlay, with a swipe divider, blinking, or as a difference image. You can copy regions that are different from one image to the other.
Folder	The items of two or three folders. You can copy, delete and merge items, and open each file in its own comparison.
Web page	Two or three web pages: as visible text, as formatted Markdown, as HTML source or as downloaded resources. Web pages are read-only. OxideMerge never saves anything to a web server.

Documents, spreadsheets and programs are compared as the text OxideMerge reads from them, which is a text or table comparison of what was read rather than a type of its own. Word documents, workbooks and tagged PDF files can also be shown **Formatted**, with their headings, lists and tables; see Documents, spreadsheets and programs.

When **Compare as** is **Automatic**, OxideMerge checks these conditions in this order and uses the first one that matches:

1. Any side is an `http://` or `https://` address: web page comparison.
2. Every side is an HTML file (`.html` or `.htm`): web page comparison. It uses the Formatted mode, unless you chose another mode before. Exception: a merge with an output file (`-o`) or a console patch (`--patch`) compares the files as text.
3. Every side is a folder: folder comparison.
4. Every file is one OxideMerge reads as text — a document, a spreadsheet or an ELF program (see Documents, spreadsheets and programs) — or an unpacker plugin converts it (see Unpacker plugins): text comparison of what was read, read-only. A spreadsheet of one sheet that OxideMerge reads itself opens as a table comparison. This comes before the archive and binary rules, so `.docx`, `.xlsx` and `.odt` open as their text rather than as the zip archives they are. Turning off **Read documents, spreadsheets and ELF files as text** in **Options** → **Plugins** leaves them to the rules below.
5. Every side is an archive: the archives are extracted and compared as folders.
6. Every file has an image extension, such as `.png`, `.jpg`, `.gif`, `.bmp`, `.tif` or `.webp`: image comparison.
7. A file has a zero byte in its first 8 KB: binary comparison. Exception: UTF-16 text with a byte order mark is still text.
8. Anything else: text comparison.

To choose the type yourself, use **Compare as** in the dialog, **Compare** → **Compare as** in an open tab,

or `-t` on the command line.

3.7. Choosing a workflow

Task	What to do
Review two versions	Open two files or folders. Make a side read-only if you do not need to change it.
Combine two edited versions	Open three versions in this order: left, base, right. See Three-way merge.
Resolve a conflict file	Open the file that contains conflict markers. See Resolving conflict files.
Review many changed files	Compare the folders, then open **All differences**. See Reviewing all differences.
Repeat a comparison later	Save a project (see Projects). To keep the text and your review notes as they are now, save the comparison instead (see Saved comparisons).

3.8. Getting help

- **Help** → **Help & Shortcuts** (`F1` or `?`) lists every keyboard shortcut and mouse action, sorted by view. It also shows where the settings file is.
- **Search commands** in the title bar, or **Help** → **Command palette** (`Ctrl` + `K` or `Ctrl` + `Shift` + `P`), finds commands, recent comparisons and settings. Type part of a name and press `Enter`. For settings that you can turn on and off, the palette shows the current state.

Chapter 4. Comparing text files

This chapter covers plain text. For PDFs, documents, spreadsheets, Jupyter notebooks and ELF programs, which OxideMerge also reads as text, see Documents, spreadsheets and programs.

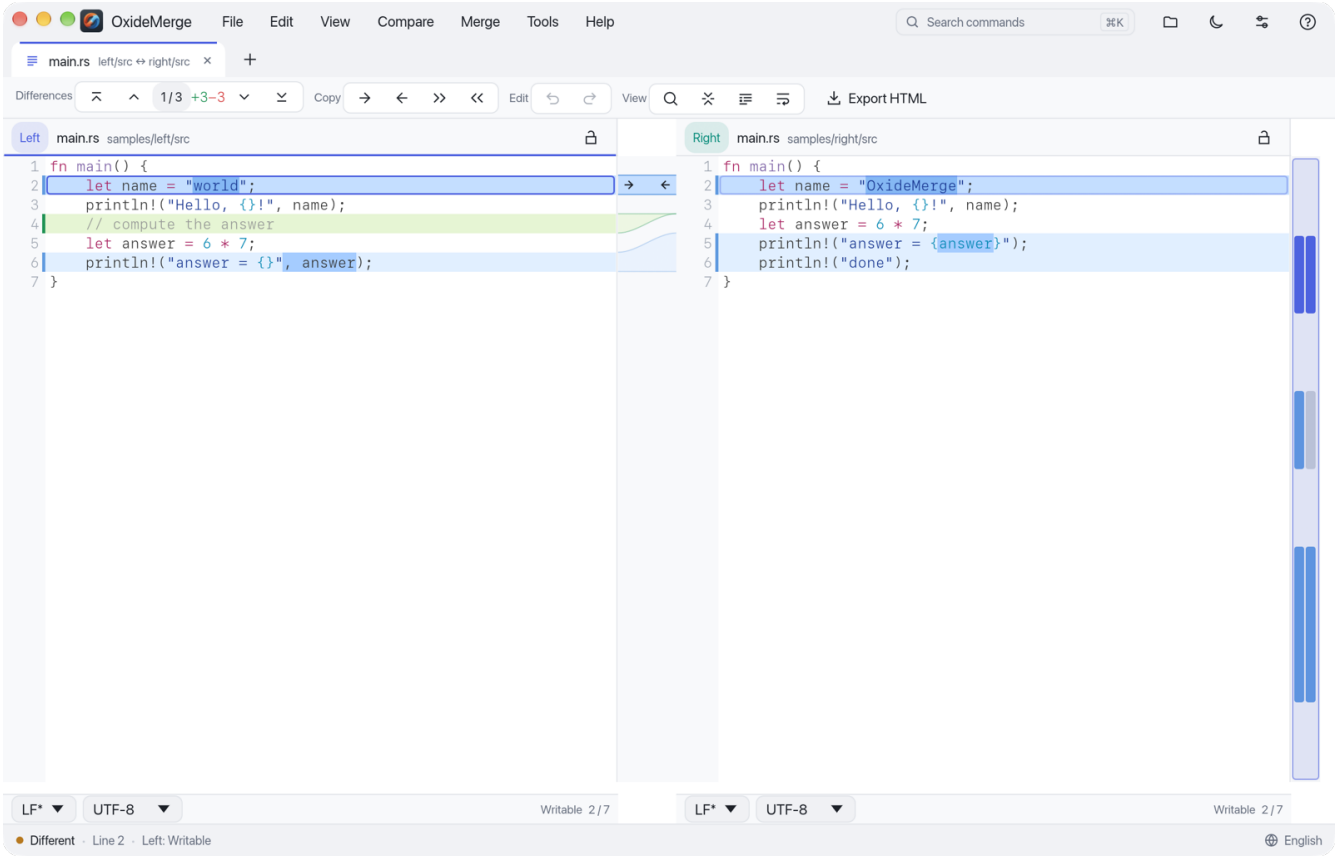


Figure 3. A two-way text comparison

4.1. The text comparison window

Each side has a header and a footer:

- The header shows the file name and its folder. The lock button makes the side read-only.
- The footer shows the line-ending style and the encoding that OxideMerge uses when it saves the file. It also shows the current line and the number of lines.

If you choose another encoding in the footer, OxideMerge reads the file again with that encoding. Check the text after you change the encoding, especially before you edit or save.

Colors show the differences:

- Changed lines are blue.
- In a two-way comparison, lines that are on one side only are green.
- In a three-way comparison, conflicts that are not resolved are red.
- Inside a changed line, the changed words or characters are highlighted.

If you turn on **Color-blind friendly diff colors** in **Options** → **General**, changed lines are purple and

lines on one side only are blue.

Between the panes, a connector joins the two sides of each difference. The current difference is highlighted. Its copy arrows appear when the difference is current or when the pointer is over it.

The toolbar has four groups of buttons:

- **Differences**: go to the next, previous, first or last difference.
- **Copy** (**Merge** in a three-way comparison): copy differences between the sides.
- **Edit**: undo and redo.
- **View**: find, collapse unchanged lines and word wrap.

In a wide window, the toolbar shows the group names. The +/− numbers next to the difference number are the numbers of added and removed lines.

The location pane, at the right, shows an overview of the whole file. Click in it to go to that place. Binary comparisons, the Formatted mode of web page comparisons and the Formatted view of documents have a location pane too.

4.2. Moving between differences

Keys	Action
`Alt` + `↓`, `Alt` + `↑`	Go to the next or previous difference
`Alt` + `Home`, `Alt` + `End`	Go to the first or last difference
`Alt` + `PgDn`, `Alt` + `PgUp`	Go to the next or previous conflict (three-way comparison)
`Ctrl` + `G`	Go to a line number

If you opened the file from a folder comparison, you can continue to other files. After the last difference, `Alt` + `↓` opens the next file that has differences. Before the first difference, `Alt` + `↑` opens the previous one, at its last difference. In a three-way folder comparison, `Alt` + `PgDn` and `Alt` + `PgUp` go on in the same way to the next or previous file that still has an open conflict.

Hold the pointer over a navigation button to see what it goes through, its keys, and that it stops at the end instead of starting again at the top.

4.2.1. Moving through the differences of a folder

In a folder comparison, the same keys work on the file shown below the folder list, and on the other files of the folder:

- `Alt` + `↓` goes to the next difference of the file. After its last difference, it selects the next file that has differences and goes to its first difference.
- `Alt` + `↑` goes the other way: before the first difference, it goes to the last difference of the previous file.
- `Alt` + `Home` and `Alt` + `End` go to the first difference of the first file and the last difference of the last file.

- The status bar shows where you are, for example *File 2 / 5 · Difference 3 / 7*.

The files are the ones that the folder list shows as different, in the order of the list. Binary files are included. Files that could not be read are not. At the last difference of the last file, OxideMerge says so; it does not start again at the top.

If you press a key while a file is still loading, OxideMerge remembers it and moves when the file is shown, so no difference is skipped. If a file cannot be read, the pane says so and offers **Retry** and **Skip**. While the folders are still being compared, more files may follow: the message says so instead of ending. If the comparison was stopped, the message at the end says that it covers only the files compared.

A file that changed after the comparison is shown as it is now. If it no longer has differences, the next step goes on to the next file. A file shown below the list that changes on disk is loaded again at the same difference.

In a three-way folder comparison, `Alt` + `PgDn` and `Alt` + `PgUp` go through the open conflicts of the files that both sides changed. A file whose changes merge without conflicts is passed over once it is loaded, and a file changed on one side only is not visited. `Alt` + `↓` still stops at every changed file.

4.3. Copying differences

When you copy a difference, OxideMerge changes the text in the destination pane. The file on disk changes only when you save it. Before you copy, check which pane is active and which way the arrow points.

- Click an arrow on the divider between the panes to copy the current difference, or the difference under the pointer.
- `Alt` + `→` and `Alt` + `←` copy the current difference, or the selected lines, to the right or to the left.
- `Ctrl` + `Alt` + `→` and `Ctrl` + `Alt` + `←` copy all differences to the right or to the left.
- In a three-way comparison, the toolbar's **Merge** group copies the current difference, or all differences, from the left or from the right into the middle. Right-click a difference to copy it from the left or the right to the middle, or from the middle to the left or the right.
- `Ctrl` + `Z` undoes a change. `Ctrl` + `Y` or `Ctrl` + `Shift` + `Z` redoes it.

4.4. Saving files

- `Ctrl` + `S` saves all changed panes in the current tab.
- `Ctrl` + `Shift` + `S` saves all changed panes in all tabs.
- **File** → **Save as** saves one side with another name.

OxideMerge saves a file safely:

1. It writes the new content to a temporary file in the same folder.
2. It replaces the original file with the temporary file.

If one of these steps fails, OxideMerge shows an error. It never writes directly over the original. For this reason, the folder must allow OxideMerge to create and replace files, even when the file itself is writable.

To keep the previous content of a file, turn on **Keep a .bak copy when saving over a file** in **Options → General**. OxideMerge then keeps the old content as `<name>.bak`. The next save replaces this backup, so it is not a version history.

4.5. Editing text

You can edit each pane that is not read-only.

Click in the text to place the cursor there, and type, as in any text editor.

- `Enter` starts a new line, with the indentation of the line above. `Backspace` and `Delete` remove a character, or join a line to the one next to it.
- `Ctrl` + `V` pastes at the cursor, any number of lines at once.
- To select text, drag over it, `Shift`+click, or move the cursor with `Shift` held. Double-click selects a word, triple-click selects the line with its line break, and `Ctrl` + `A` selects the whole file. Dragging past the top or bottom of the pane scrolls it and goes on selecting. Typing or pasting replaces the selection.
- `Ctrl` + `C` copies the selection and `Ctrl` + `X` cuts it. With nothing selected, they copy or cut the whole line.
- The arrow keys, `Home`, `End`, `PgUp` and `PgDn` move the cursor. `Ctrl` + `Home` and `Ctrl` + `End` go to the start and end of the file. On macOS, `Cmd` + `←` and `Cmd` + `→` go to the line start and end. With word wrap on, the up and down arrows, `PgUp` and `PgDn` move by display line, so a long line takes several presses, and `Home` and `End` (`Cmd` + `←` and `Cmd` + `→`) go to the start or end of the display line first and of the whole line on a second press. With `Shift` held, each of these extends the same selection.
- `Tab` types a tab, or indents the lines when the selection goes over more than one line or is a whole line; `Shift` + `Tab` outdents the selected lines, or the cursor's line. `Ctrl` + `]` and `Ctrl` + `[` do the same with the text cursor in a pane. A file that indents more lines with tabs than with spaces is indented with a tab, and any other with the **Tab size** in spaces. Blank lines are not indented, and outdenting removes one tab or up to the tab size in spaces. A selection that ends at the start of a line leaves that line alone, and the selection moves with its text, so `Shift` + `Tab` right after `Tab` gives back the same text and selection. Each is one undo step.
- `Ctrl` + `←` and `Ctrl` + `→` (`Option` + `←` and `Option` + `→` on macOS) move by word, and with `Shift` held they select by word. A word move goes to the end of the next word, or to the start of the one before; from a line end or start it goes to the next or previous line. Letters, digits and `_` make one word (`snake_case` and `camelCase` are both one word), a run of punctuation is a word of its own (`x.len` is three), and each Chinese, Japanese or Korean character is a word.
- `Ctrl` + `Backspace` and `Ctrl` + `Delete` (`Option` + `Backspace` and `Option` + `Delete` on macOS) delete the word before or after the cursor, or the selection when there is one. One `Ctrl` + `Z` brings the word back.
- On macOS, while the cursor is in the text, `Option` + `←` and `Option` + `→` move by word rather than copy the difference; press `Esc` to leave the text, or use the copy buttons. `Option` with a letter, a digit or a

sign types its character (`Option` + `S` types ß), so `Alt` + `C`, `Alt` + `Z`, `Alt` + `P` and `Alt` + `S` wait until the cursor leaves the text; their commands stay in the menus.

- Each `Esc` does one thing, the first of these that applies:
  1. Closes an open menu.
  2. Closes a dialog, or a field you are typing in.
  3. Closes the search bar.
  4. Goes back from several cursors to one.
  5. Ends the selection (the cursor stays where it was).
  6. Removes the cursor, so the arrow keys move from row to row again.
  7. Closes the tab.
- Input methods work at the cursor, so you can type Chinese, Japanese or Korean.
- In very large files, typing within a line stays fast: OxideMerge compares the files again once you pause for a moment, or before any other key, click or command, so the colors of the line you are typing in may lag briefly. A new line, a paste or an undo compares at once.
- A selection over several lines also selects their rows, so the copy buttons and `Alt` + `→` and `Alt` + `←` copy those lines to the other side.
- `Ctrl` + `]` indents the selected lines, and `Ctrl` + `[` unindents them (with the text cursor, also `Tab` and `Shift` + `Tab`; see above).
- `Ctrl` + `D` puts a copy of the selected lines, or of the cursor's line, under them and selects the copy. `Alt` + `Shift` + `↑` and `Alt` + `Shift` + `↓` move them up or down past the next line, with their selection. Like `Tab`, they take the lines of the text selection (one that ends at the start of a line leaves that line out), or without a text cursor, the active pane's lines in the selected rows. Each is one undo step, and both are also in the **Edit** menu. Without `Shift`, `Alt` + `↓` and `Alt` + `↑` still go to the next and previous difference.
- Without a cursor, `F2` or `Enter` puts it at the end of the current line.

4.5.1. Several cursors

To type the same thing in several places of one pane, give it several text cursors:

- `Ctrl`+click (`Cmd`+click on macOS) adds a cursor where you click, or removes the one there. `Ctrl`+double-click and `Ctrl`+triple-click add a word or a line selection.
- `Ctrl` + `Alt` + `↑` and `Ctrl` + `Alt` + `↓` (`Cmd` + `Option` + `↑` and `Cmd` + `Option` + `↓` on macOS) add a cursor on the line above the first cursor or below the last, at the same column. Some Linux desktops use these keys to switch workspaces; `Ctrl`+click works everywhere.

Typing, deleting, `Enter`, the arrow keys, `Home` and `End` then act at every cursor; cursors that meet become one. Each edit is one undo step for all of them, and undo puts the cursors back where they were. Pasting as many lines as there are cursors puts one line at each; any other text goes in at every cursor. Copying gives the selections one per line, in the order of the text. `Tab`, `Shift` + `Tab`, **Duplicate lines** and **Move lines** act on the lines of all the cursors, each line once. A read-only pane refuses the edit at every cursor.

The last cursor added is the one the view follows and where an input method shows what you are composing. `Esc` first goes back to that one cursor, and a click without `Ctrl` does too. The column selection (`Alt`+drag) is separate: making one removes the extra cursors.

The pane with the cursor has a colored line under its header. The cursor is a solid bar where typing goes into the text; it is thin and faint in a read-only pane, while a field such as the search bar has the keyboard, and while the window is in the background. Typing into a read-only pane changes nothing, and the status bar says which side it is and how to unlock it.

The cursor works in the side-by-side view. In the inline view, double-click a line, or press `Enter` or `F2`, to edit it side by side (see Inline view); in the table view, double-click a cell to edit it.

The bar under each pane shows whether you can edit it:

- **Writable**: you can edit the file and save it.
- **Read-only**: the pane is locked, for example with the lock button in its header, and does not take edits.
- **Read-only on disk**: you can edit, but the file is marked read-only, so **Save** refuses to replace it and keeps your edits: save it under another name, or clear the read-only mark and save again.
- **Compare only**: the pane shows text that OxideMerge read from the file, not the file itself: a document, spreadsheet, PDF or program, a web page, or a formatted copy of JSON or XML. It cannot be unlocked, edited or saved back. To edit the text itself, save it with **File** → **Save as** to a plain file, such as a `.txt` file, which the pane then edits; it is never saved over a file that is itself read this way, whatever the options say.
- **Modified** appears while the file has changes that are not saved.

The status bar at the bottom of the window shows the same for the active pane. It also shows **Compare only** for the Tree and Formatted views and for web pages shown as Markdown.

To select a column of text, hold `Alt` (`Option` on macOS) and drag. The selection covers the same columns on every line it crosses. It does not work with word wrap or in the table view.

- Typing replaces the selected columns on every line. A selection with no width is a cursor on every line, so typing inserts the same text on each.
- `Backspace` and `Delete` remove the selected columns, or one character on each line.
- `Ctrl` + `C` copies the columns, one line per row, and `Ctrl` + `X` cuts them.
- `Ctrl` + `V` pastes. A single line goes into every selected line. Several lines go in one per line, and lines past the selection go into the lines below it.
- `Esc` or a click ends the column selection.

Columns are counted as the lines are shown: a tab reaches the next tab stop. Text put past the end of a short line is padded with spaces so that it lines up. Characters that are drawn wider than others, such as Chinese characters, still count as one column, so a column selection across them can look uneven.

Each edit, paste, cut or deletion is one undo step; typing within a line is one step until the cursor moves. Saving keeps each file's line endings, encoding and byte order mark.

Right-click a line to edit it, insert a line below it, delete it, copy its text, set a bookmark, or create a compare rule from it (see Creating a rule from a text comparison). A right-click in the selected text keeps the selection, and **Copy text** then copies just that text; a right-click elsewhere moves the cursor there first.

Edit markers in the gutter show the lines that changed since you opened the files.

When another program changes a file, OxideMerge notices it when its window becomes active again:

- A file without unsaved changes reloads automatically.
- A file with unsaved changes shows a message with a reload option. Reloading replaces your edits with the file on disk.

4.6. Find and replace

- `Ctrl` + `F` opens the find bar. `F3` and `Shift` + `F3` find the next and the previous match. `Esc` closes the bar.
- `Ctrl` + `H` (`Cmd` + `Alt` + `F` on macOS) opens find and replace.
- The search options are **Match case**, **Whole word** and **Regular expression**. With a regular expression, `$1` in the replacement text means the first captured group.
- You can undo **Replace all** in one step.

4.7. Bookmarks

- `Ctrl` + `F2` sets or removes a bookmark on the current line.
- `F4` and `Shift` + `F4` go to the next and the previous bookmark.
- **Edit → Bookmarks → Bookmark comment...** adds a comment to a bookmark. The comment appears when you move the pointer over the gutter, and in HTML reports.
- **Edit → Bookmarks → Clear bookmarks** removes all bookmarks.

Bookmarks are marked in the gutter and in the location pane.

4.8. Synchronization points

Sometimes the comparison pairs the wrong lines. For example, this can happen when a function was rewritten and moved. Use a synchronization point to tell OxideMerge which lines belong together:

1. Put the cursor on a line on one side. Then choose **Merge → Set synchronization point** (`Alt` + `S`), or right-click the line.
2. Do the same on the matching line on the other side. In a three-way comparison, do it on the third side too.

OxideMerge shows these lines on the same row. It compares the text before them and the text after them separately.

- You can add as many points as you need. OxideMerge refuses a point that would cross another point.
- Synchronization points are marked in the gutter. They stay with their lines when you edit, undo or reload, and they are kept in saved comparisons.
- To cancel a line that you picked, run the command again on that line.
- To remove a point, run the command again on one of its lines.
- To remove all points, choose **Merge** → **Clear synchronization points**.

4.9. View options

View → **Text compare** has these options:

- **Location pane** and **Line detail pane**. The line detail pane shows the current line of each side, one above the other, with word wrap. Use it to find small differences in long lines.
- **Line numbers**, **Edit markers**, **View whitespace** and **Syntax highlighting**.
- **Collapse unchanged lines** (`Alt` + `C`) hides unchanged lines behind a bar. Three lines stay visible around each difference. Click the bar to show the hidden lines.
- **Word wrap** (`Alt` + `Z`).

View → **Zoom** (`Ctrl` + `+`, `Ctrl` + `-` and `Ctrl` + `0`) changes the text size.

Text layout is in **View** → **Text compare** and in **Options** → **General**:

Compact rows
: The default. Each side shows its own lines, without empty space.

Aligned rows
: Empty space is added opposite inserted and deleted lines, so the lines after them stay side by side.

The layout applies to all open two-way and three-way text comparisons, and is saved for the next session. It does not change the files or the compare rules.

4.9.1. Inline view

In a two-way comparison, the **Inline view** button next to **Word wrap** shows the differences in one column. The command palette has the same command.

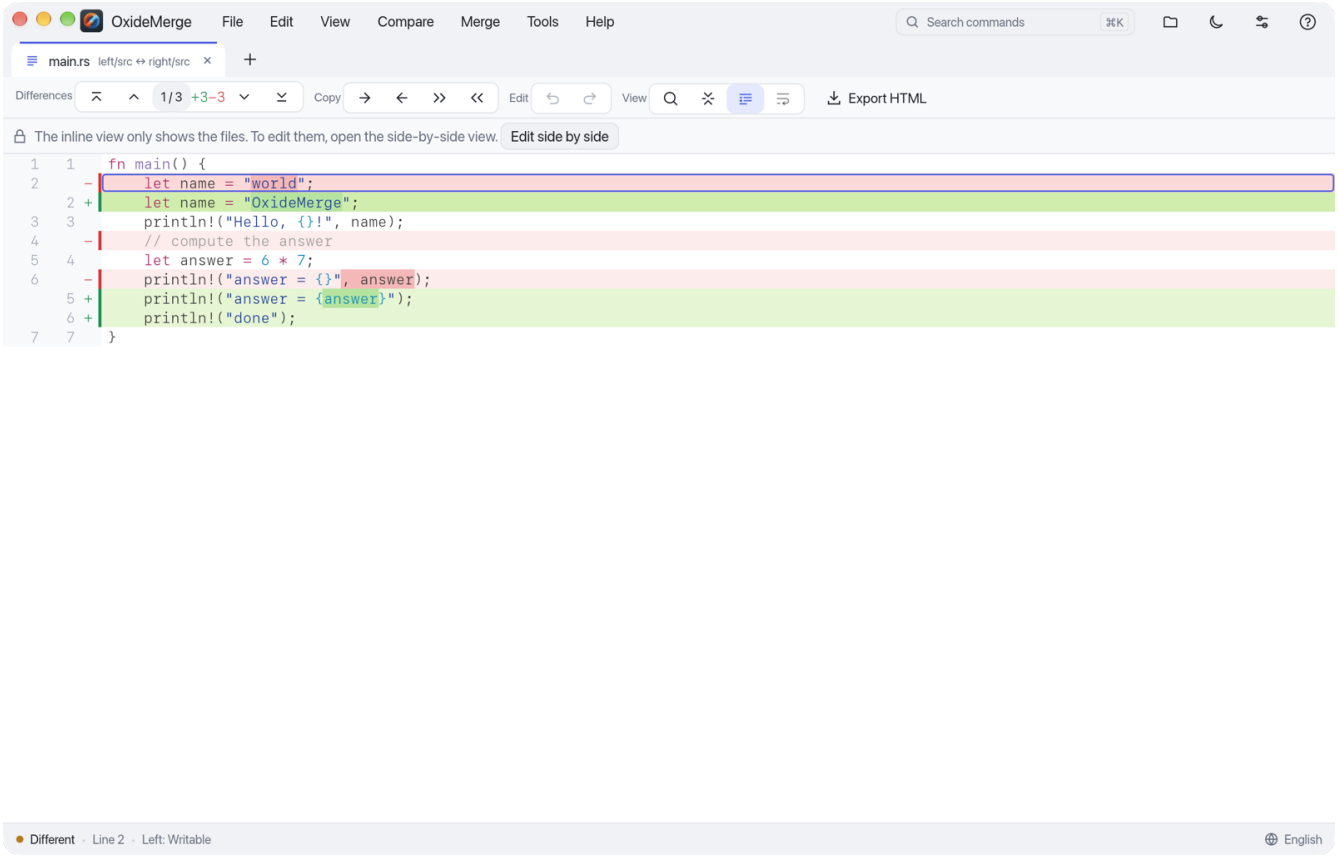


Figure 4. The inline view

- Removed lines come first, then the lines that replace them. Each line shows its old and its new line number.
- When **Collapse unchanged lines** is on, unchanged lines are hidden. Click a fold to show them.
- You can move between differences and copy them as usual, and Undo and Redo work. The arrow keys move from line to line, and `Ctrl` + `C` copies the current line.
- The inline view only shows the files: a bar at its top says so, and typing, pasting, indenting and **Replace** do nothing there. To edit, click **Edit side by side** on that bar, double-click a line, or press `Enter` or `F2`: the side-by-side view opens with the text cursor at the start of the current line.
- Switching between the views keeps every edit, the selected difference, and the text cursor and its selection from the side-by-side view, which come back when you switch back.

4.10. CSV and TSV files

For CSV and TSV files, the toolbar has a **Table view**. It compares records, not lines, so a quoted field can contain line breaks.

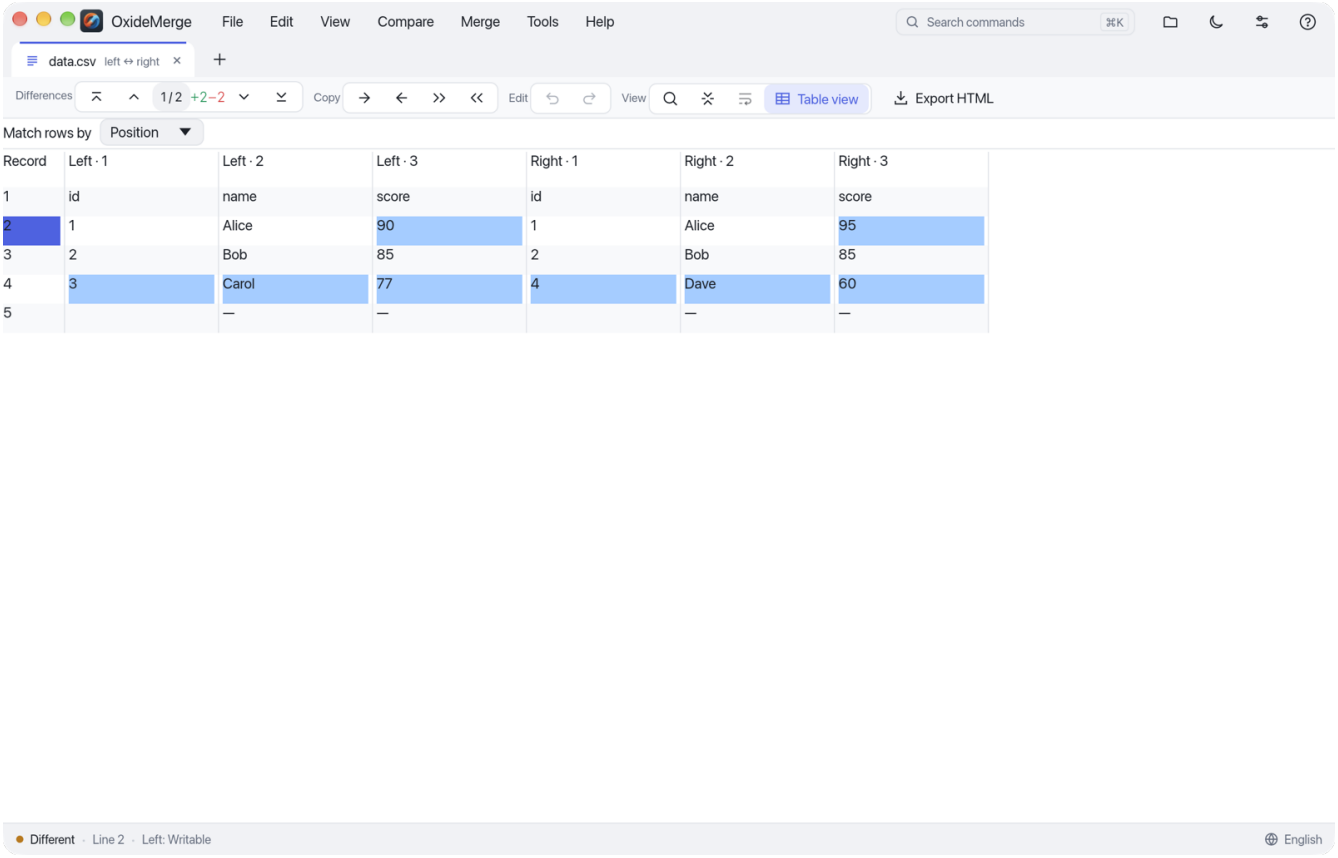


Figure 5. The table view of two CSV files

- Drag a column border to change its width.
- To edit a cell, double-click it, or right-click it and choose **Edit cell**. You can also select it and press `F2` or `Enter`. A cell can contain several lines.
- To copy one field to the matching record on another side, right-click the cell and choose **Copy cell**. Both records must have that column.
- The arrow keys move between records and columns. `PgUp` and `PgDn` move through records, and `Ctrl` + `Home` and `Ctrl` + `End` select the first and the last record.
- **Go to line** uses the line numbers of the file.

When you edit a cell, OxideMerge keeps the quotes and line endings outside that field, and you can undo and redo the change. You cannot edit converted or read-only documents.

When you copy a difference in the table view, only its selected lines change. If this would break the CSV quoting, OxideMerge refuses the copy and does not change the document. In this case, copy cells or use the text view. If a quoted record is not valid, the table shows an error; switch to the text view to correct it.

4.10.1. Matching rows by key

By default, the table view pairs records by their position, like lines. When records are sorted differently, or some are added or removed, pair them by a key instead. In a two-way comparison, the bar above the table has these settings:

Match rows by

Position (the default) or **Key columns**.

Keys

The columns that identify a record, such as `id`. Choose one or more.

First row is header

Use the first row for column names. It is not compared.

Columns by

Pair columns by **Position** or by **Header name**, so that reordered columns still match.

Ignore row order

A record that only moved is not a difference: next and previous difference skip it, but it keeps its ↕ mark.

Numeric tolerance

Numbers this close count as equal. `0.01` is a difference, `1%` a fraction of the larger number. Leave it empty to compare exactly.

Rows in

Left order (the default) lists the rows as the left table has them, with records only on the right after the record before them. **Right order** follows the right table. **Key order** sorts by key: numbers by value, before text, and rows without a key last.

The summary shows how many records were added, removed, modified, reordered and ambiguous. The row markers are `+` added, `−` removed, `~` modified, ↕ moved and `?` ambiguous.

- Next and previous difference go through the records that differ.
- **Copy cell** copies into the paired column of the matching record. Copying a whole record replaces the cells of the matching record, or adds or removes it.
- A key that is empty, or used by more than one record, is *ambiguous*. Ambiguous records are never copied. **Next ambiguous row** finds them.
- If a key column is missing on one side, OxideMerge matches by position and says why.

Tables with more than 100,000 records on both sides together are matched in the background. The bar shows how many records are done, with **Cancel**. After a cancel, the rows are matched by position, and **Match by key** starts again.

HTML and XML reports of a table matched by key say how the rows were matched: the keys, the header row, how columns were paired, the row order, the tolerance, the counts and the warnings. XML reports hold them in a `<table-match>` element.

The settings are saved in projects. Older projects open with matching by position.

4.11. Comparing with a revision

File → Compare with revision... compares a file or folder with an earlier version from Git, Subversion or Mercurial. The nearest working copy above the path decides which one. The dialog lists the recent revisions that changed the path.



Comparing with a Subversion or Mercurial revision is experimental and not fully tested: it was tried on macOS only, with local repositories.

In Git, the version can be `HEAD`, a branch, a tag or a commit; left empty, it is `HEAD`. In Subversion, it can be `BASE` (the revision the working copy was updated to), `PREV`, `HEAD`, a number such as `1234` or `r1234`, or a date such as `{2026-11-01}`; left empty, it is `BASE`. In Mercurial, it can be `.` (the working copy's parent), `.^`, `tip`, a number, a hash, a tag, a bookmark or another revset; left empty, it is `.`.

The older version is shown read-only on the left. The current file or folder is on the right. Git, Subversion or Mercurial must be installed and in your `PATH`. Git and Mercurial read their revisions from the local repository. Subversion reads `BASE` from the working copy; other revisions, and the list of recent ones, come from the repository server, which may ask for a sign-in that OxideMerge cannot give. Sign in with `svn` itself first. The revision is read in the background; **Cancel** stops a server that is slow or does not answer, and the dialog comes back. Clicking **Compare** lets Mercurial read the repository's own `.hg/hgrc`, as Git's filters are honored; the list of recent revisions, read while you type, skips it (Mercurial 5.4 or later). When you compare the top folder of a working copy, the `.git`, `.svn` or `.hg` folder is listed on the right only; the **Exclude Source Control** filter hides it. Exporting a revision's folder is refused if it would come to more than 8 GB or 500,000 files, the same limit archive extraction already has, so a mistaken or huge revision cannot fill the disk.

On the command line, use `--rev` (or `--git-rev`, the same option):

```
oxidemerge --rev HEAD~1 src/main.rs
oxidemerge --rev PREV src/main.c
oxidemerge --rev .^ docs
```

See also Integrating with other tools: OxideMerge can also be set up as Git, Subversion, Mercurial or Perforce's own diff and merge tool, invoked by them rather than opened on a revision from within OxideMerge.

Chapter 5. Merging files and resolving conflicts

5.1. Three-way merge

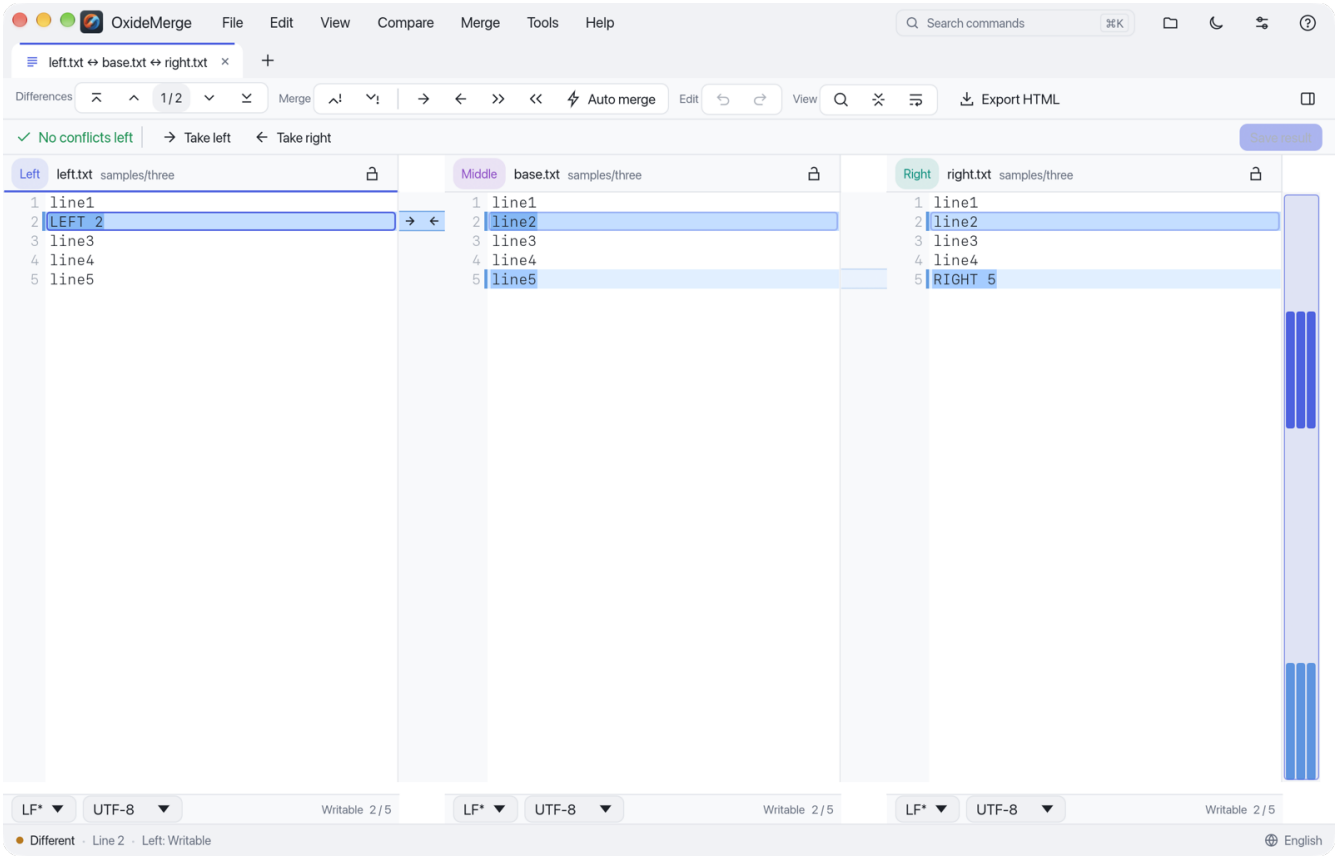


Figure 6. A three-way comparison

A three-way merge combines two edited versions of a file. Open the files in this order: your version, the base, the other version. OxideMerge compares the left and right versions with the base, and combines the changes in the middle pane.

5.1.1. Step 1: Choose the output file

By default, saving the middle pane overwrites the base file. To keep the base file, give a separate output file when you start:

```
oxidemerge mine.txt base.txt theirs.txt -o merged.txt
```

The middle pane starts with the content of the base file. When you save, OxideMerge writes the result to `merged.txt`. You can also choose **File** → **Save as** to save the result in another file.

Make the left and right panes read-only if you only use them as sources.

5.1.2. Step 2: Merge and resolve conflicts

The merge bar above the panes shows how many conflicts are left. It has these buttons:

- **Next conflict**.
- **Take left**, **Take both** and **Take right**, for the current conflict.
- **Mark resolved**, for a conflict that you edited by hand. It keeps the middle text as it is and counts the conflict as resolved.
- **Save result**, at the right end, saves the middle pane. It becomes the main button when no conflicts are left.

To merge:

1. Choose **Merge** → **Auto merge**. This copies two kinds of changes to the middle pane: changes made on one side only, and the same change made on both sides.
2. Go to each remaining conflict with `Alt` + `PgDn` and `Alt` + `PgUp`.
3. Copy the change that you need from the left or the right, or edit the middle pane.
4. To keep both changes, right-click the difference and choose **Take left, then right** or **Take right, then left**. Then check the result for repeated or conflicting text.
5. Save the result. Check that the destination path is correct.



Auto merge works only on the text. It does not check that the result is correct. Review the result, and run the checks that your project needs, before you use it.

5.2. Resolving conflict files

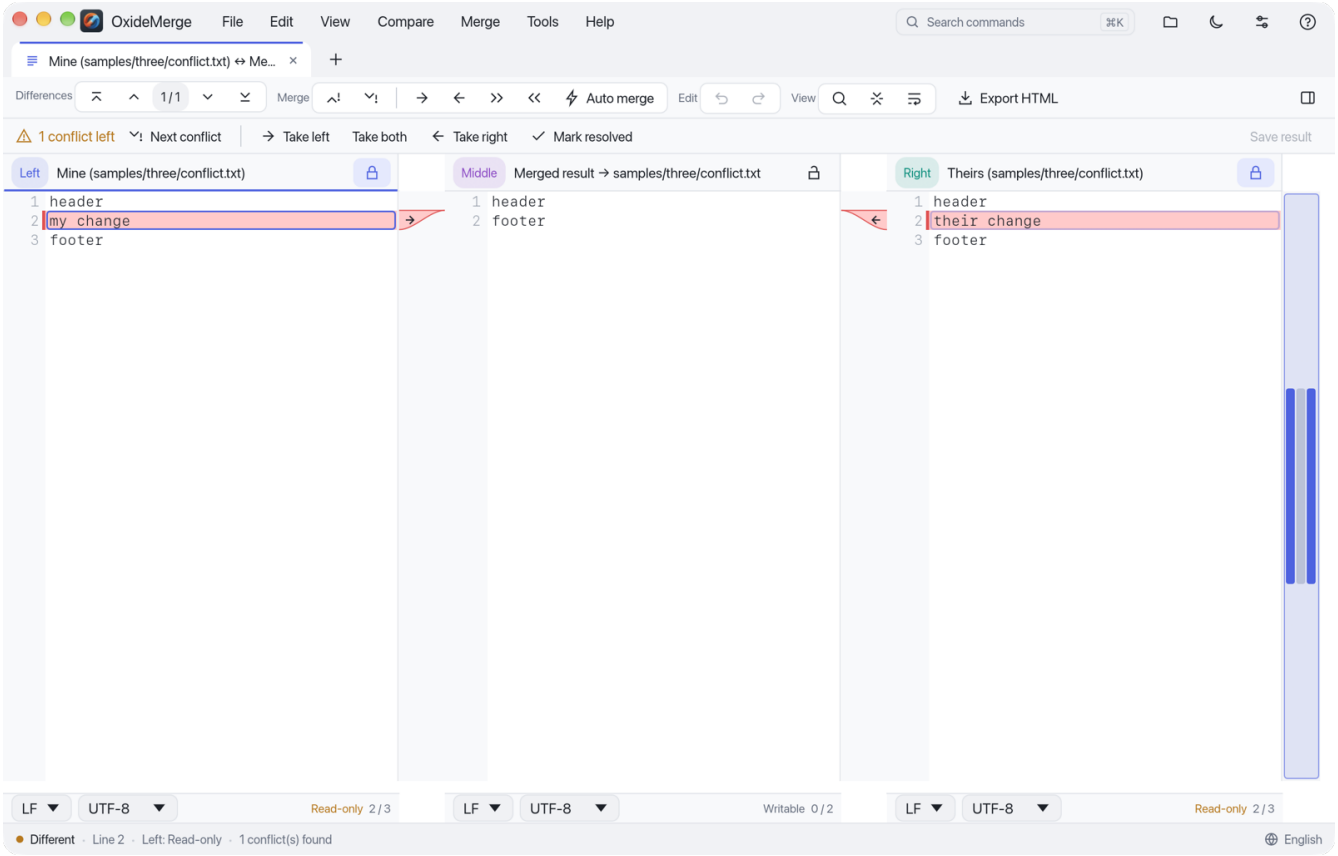


Figure 7. A conflict file in the three-way view

A conflict file is a file with version-control conflict markers: `<<<<<<<`, `|||||||`, `=======` and `>>>>>>>`. OxideMerge opens it in the three-way view.

To open a conflict file, use one of these:

- **File → Open conflict file...**
- **Open conflict file** on the start page.
- The command line: `oxidemerge conflict.txt`.

The left side is **Mine**, the right side is **Theirs**, and the middle is the **Merged result**. Resolve each conflict, then save the middle pane. OxideMerge writes the result back to the conflict file.



This is different from opening three separate files. For a conflict file, the original file is the output.

Chapter 6. Comparing folders

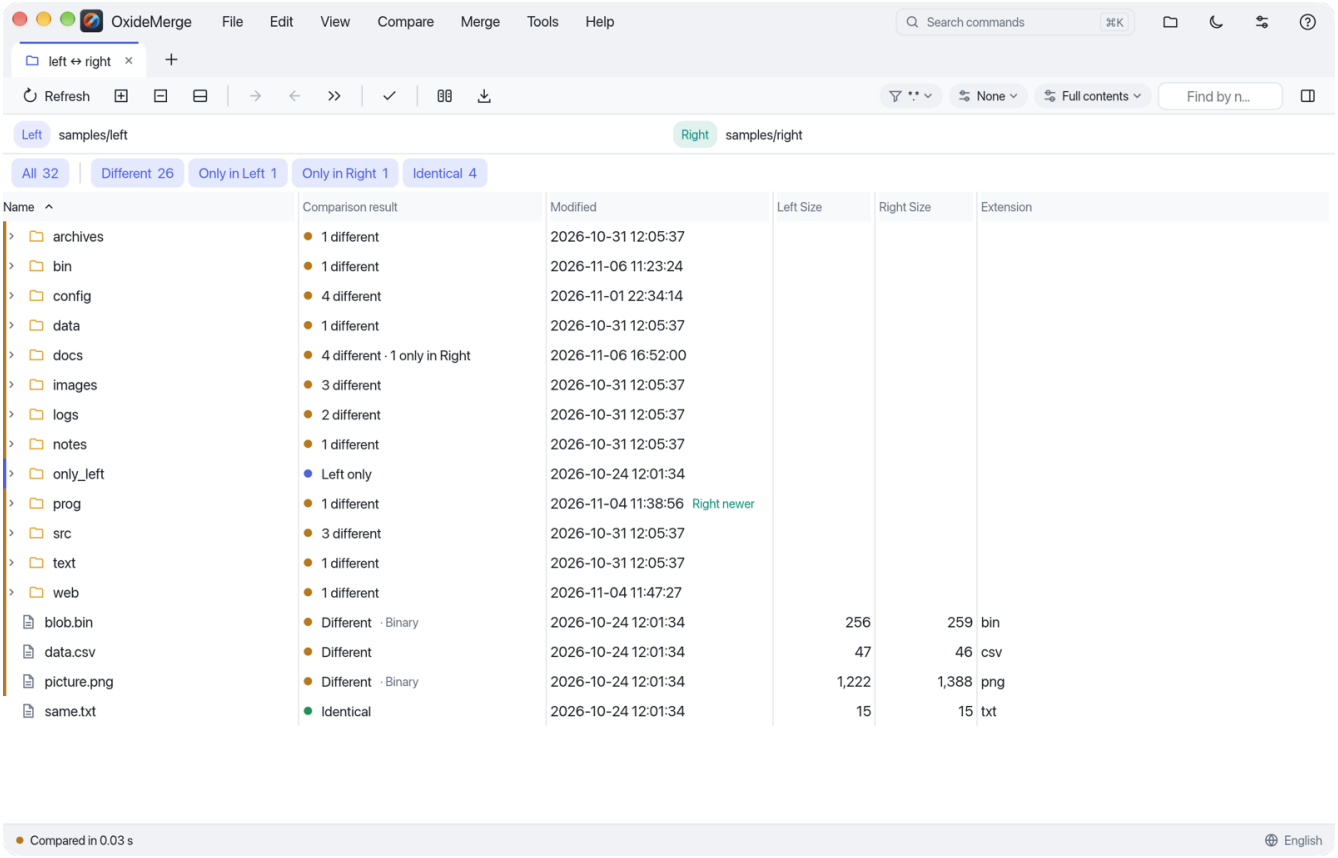


Figure 8. A folder comparison

A folder comparison lists the items of two or three folders and the result for each item. Results appear while the comparison runs: OxideMerge compares the files during the folder scan and lists each difference as soon as it finds it. If a long comparison finishes while the window is in the background, OxideMerge asks for your attention.

6.1. The folder list

Each row shows these columns:

- **Name**.
- **Folder**: the folder that contains the item. This column is shown in a flat list only.
- **Comparison result**.
- **Modified**: the newest date of all sides. If the dates are different, the newer side is named after the date. Move the pointer over the date to see the date of each side.
- The size on each side.
- **Extension**.

Click a column header to sort by that column. Click it again to reverse the order. A colored bar at the left edge of a row marks a difference or an item that is on one side only.

The comparison result is one of these:

Result	Meaning
Identical, **Different**	The files are the same or not the same. **Binary** marks binary files.
For example, **3 different · 1 only in Left**	A folder that is different. The text tells what is different inside it.
Left only, **Middle only**, **Right only**	The item is on one side only.
Left is different, **Middle is different**, **Right is different**	Three-way comparison: one side is different from the other two.
Missing in Left, **Missing in Middle**, **Missing in Right**	Three-way comparison: the item is on the other two sides only.
Error	OxideMerge could not read the item.

Move the pointer over a result to see the full description, for example **Text files are different**.

The chips under the paths count the items: **All**, **Different**, **Only in Left**, **Only in Right**, **Only in Middle** (three-way only) and **Identical**. Click a chip to hide or show that kind of item. Some details about the counts:

- In a three-way comparison, an item that is missing from one side counts as different.
- Empty folders, and folders that were not scanned, count by the result that they show.
- If **Include subfolders** is off, the subfolders are not compared. They count only in **All**, and no chip hides them.

The status bar shows how long the comparison took.

6.2. Display options

View → **Folder list** has these options:

Tree mode

Shows subfolders as a tree. Turn it off for a flat list. `→` and `←` expand and collapse a folder. The toolbar has **Expand all** and **Collapse all**.

Show

Hides or shows **Identical**, **Different**, **Left only**, **Middle only** and **Right only** items.

File compare below the folder list

Shows the differences of the selected file under the list, without opening a tab. The pane button on the toolbar, or `Alt` + `P`, shows or hides this pane.

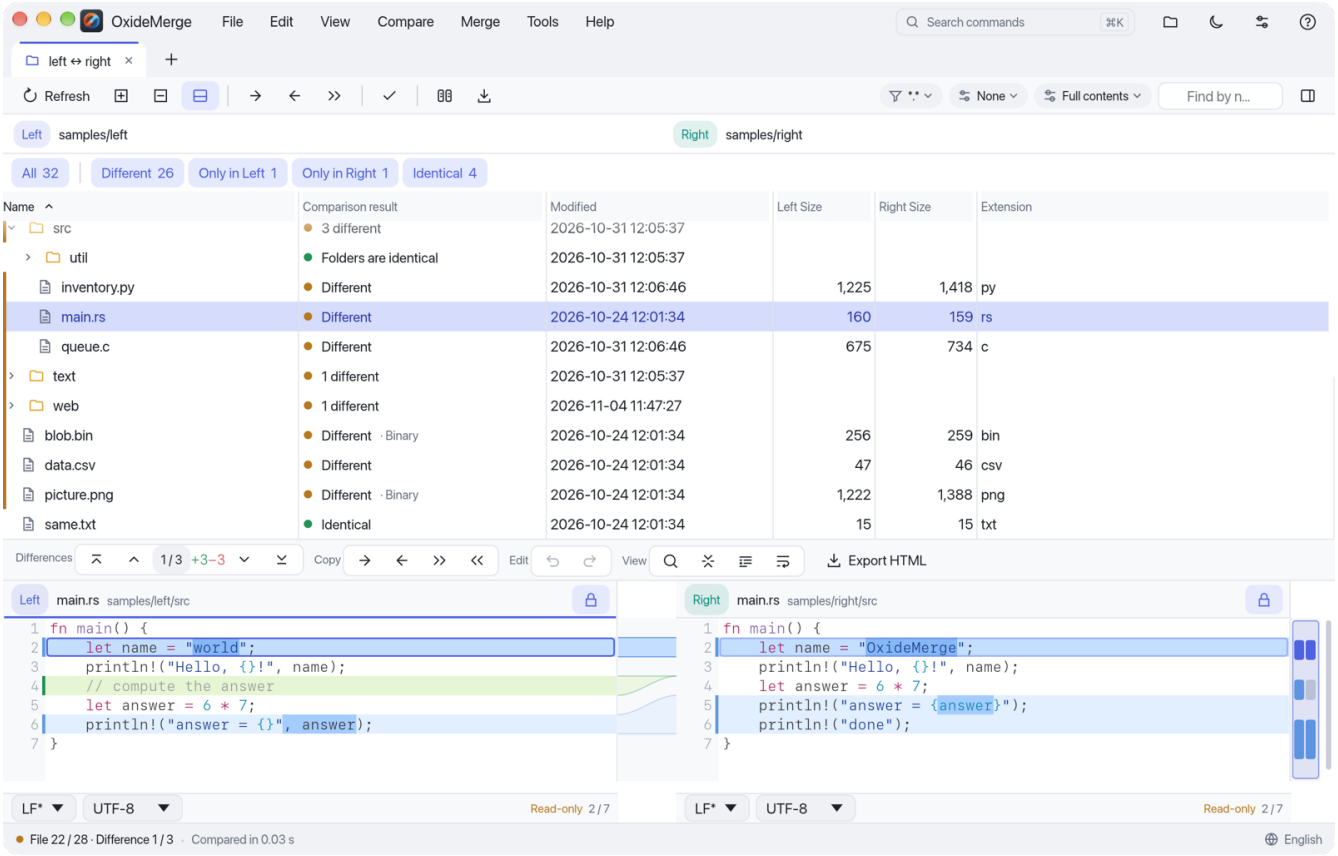


Figure 9. The file compare below the folder list

The navigation keys work differently when this pane is shown:

Keys	With the file compare pane	Without the pane
`Alt` + `↓`, `Alt` + `↑`	Go to the next or previous difference in the file. After the last difference, go on to the next file that has differences.	Select the next or previous file that has differences.
`Alt` + `Home`, `Alt` + `End`	Go to the first or last difference in the file.	Select the first or last file that has differences.

The arrow buttons of the pane do the same as `Alt` + `↓` and `Alt` + `↑`. If you press a key while a file is still loading, OxideMerge waits for the file, so no difference is skipped. If a review filter is active, these keys go only through the files that the filter lists.

6.3. The toolbar

From left to right:

- **Refresh** (`F5`) compares the folders again.
- **Expand all** and **Collapse all**, in tree mode.
- **File compare below the folder list** (`Alt` + `P`) shows or hides the file compare pane.
- **Copy to right** and **Copy to left** copy the selected items. In a three-way comparison, these buttons are **Copy from left to middle**, **Copy from right to middle** and **Auto merge folders...**.
- **Synchronize folders...**, in a two-way comparison, previews and applies a synchronization (see

Synchronizing two folders).

- **Select by name...** selects the items whose names match wildcards, such as `*.c; *.h`.
- **All differences** opens all changed text files in one tab (see Reviewing all differences).
- **Export HTML** saves an HTML report (see Saving, sharing and reporting).
- **Filter** chooses which files and folders are compared (see Filters).
- **Rules** chooses the compare rules file for this comparison (see Compare rules).
- **Compare method** chooses how files are compared (see Comparison methods).
- **Find by name** (`Ctrl` + `F`) shows only the items whose names contain the text you type.
- **Side panel**, at the right end, opens a drawer with the review, verification and baseline panels (see Analysis tools).

In a narrow window, **All differences** and **Export HTML** become icons, and the chips show only their numbers.

6.4. Comparison methods

Method	How files are compared
Full contents	Text files are compared line by line, with the comparison options and compare rules. Files that differ only in ignored ways count as identical.
Quick contents	The bytes are compared. Files of different sizes are found without reading them.
Binary contents	The bytes are compared, as in **Quick contents**. The result is marked as binary.
Modified date	The modification dates are compared. The **Date tolerance (seconds)** in **Options** → **Folder** sets how far apart two dates can be.
Modified date and size	The dates and the sizes must both match.
Size	Only the sizes are compared.

Which method to use:

- Use **Full contents** when compare rules and ignored differences matter.
- Use **Quick contents** or **Binary contents** to check that files are exactly the same.
- Use a date or size method for a fast first check. Note that the same date and size do not prove that the contents are the same.

The byte methods read files in small parts, so large files do not have to fit in memory. A full text comparison can still load changed files into memory.

6.5. Selecting and opening items

Select items with the mouse or the keyboard:

- The arrow keys, `PgUp`, `PgDn`, `Home` and `End` move the selection. Hold `Shift` to extend it.
- `Ctrl`+click adds an item to the selection, or removes it.
- `Ctrl` + `A` selects all items.

To open items:

- Double-click an item, or press `Enter`, to compare it in its own tab. In tree mode, this expands or collapses a folder instead. To open a folder in its own folder comparison, use a flat list, or right-click the folder and choose **Compare**.
- Press `Space` for a **Quick preview**: a colored view of the differences of the selected file, without a new tab. In the preview, `↑` and `↓` go to the next item, `Enter` opens the item, and `Space` or `Esc` closes the preview.

6.6. Copying and deleting items

- `Alt` + `→` and `Alt` + `←` copy the selected items to the right or to the left.
- Right-click an item for more commands:
  - **Copy** to another side.
  - **Copy to folder...**: copies the selected items of one side to another folder, and keeps their relative paths.
  - **Delete** from a side.
  - **Open with associated application**.
  - **Show in file manager**.
  - **Copy full path**.
  - **Select by name...**.

These settings in **Options** → **General** control file operations:

- **Confirm copy operations**: OxideMerge asks before it copies items. It always asks before it deletes items.
- **Use the recycle bin when deleting**: deleted items go to the recycle bin or trash. If an item cannot go there, OxideMerge does not delete it.
- **Keep a .bak copy when saving over a file**: OxideMerge keeps the old content of a replaced file.



Copy and delete change the files on disk at once. They do not wait for **Save**, and the **Undo** command of the text editor does not reverse them. Check the selected items and the destination first.

6.7. Reviewing all differences

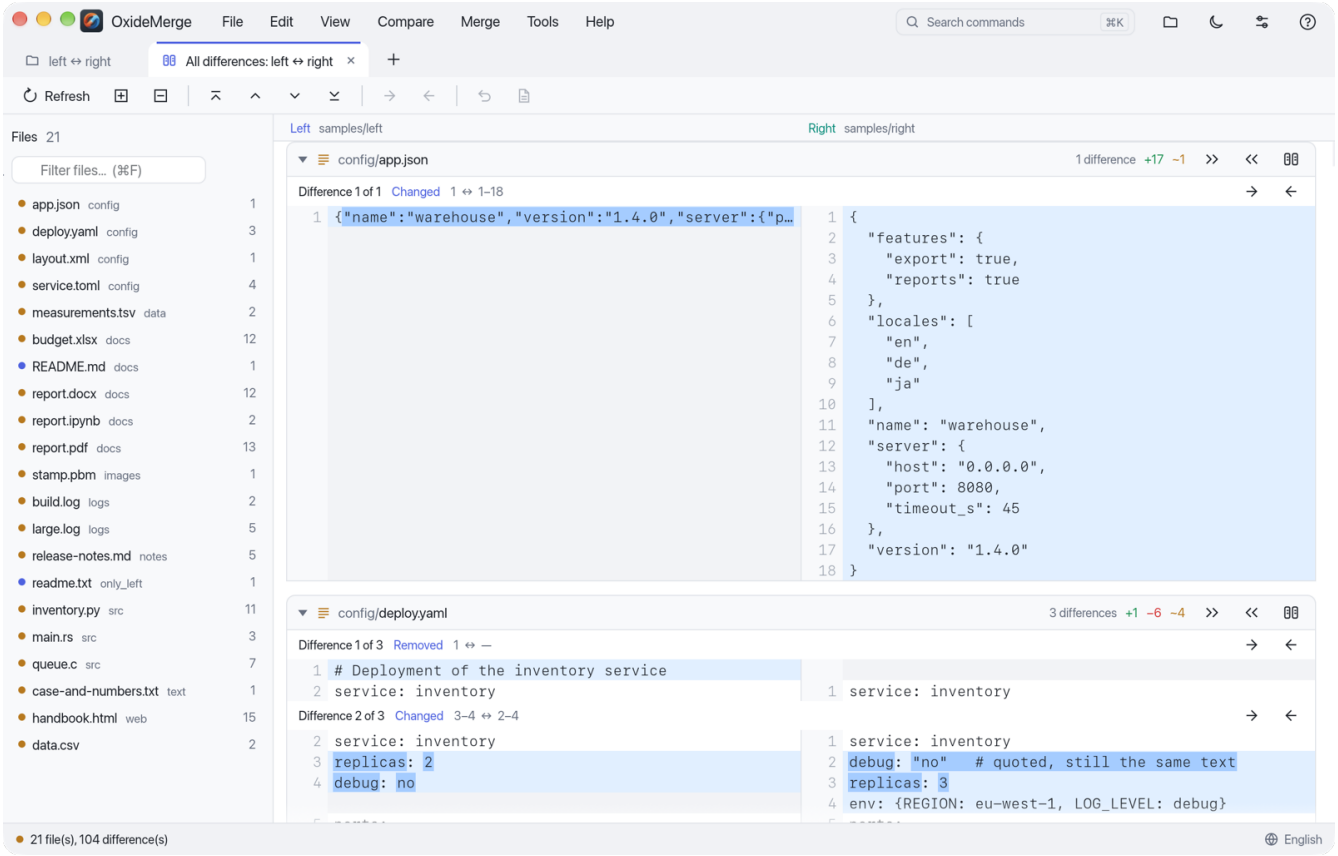


Figure 10. All differences

All differences shows all changed text files of a folder comparison in one tab. Click it on the folder comparison toolbar, or choose **Compare** → **All differences**. The filter and the **Show** settings of the folder comparison apply.

- The list on the left shows each file and its number of differences. **Filter files...** (`Ctrl` + `F`) shortens the list. Click a file to scroll to it.
- Each file is a card that shows its differences side by side. Click the arrow in a card header to collapse or expand the card. The toolbar collapses or expands all cards.
- `Alt` + `↓` and `Alt` + `↑` go through the differences of all files. When you open **All differences** from a folder comparison, it starts at the difference that was selected there. A card that is still loading is waited for, not skipped. The status bar shows the position, as in Moving through the differences of a folder.
- The arrows next to a difference copy it to another side. In a two-way comparison, the buttons in a card header copy the whole file.
- **Undo** reverses the last change.
- **Save** writes the changed files. The folder comparison is then refreshed.
- The button at the right of a card header opens the file in its own tab.

Some details:

- If you copy from a side where a file is missing, the file is deleted when you save. Folders that become empty are deleted too. If backups are on, the deleted file is kept as `.bak`, so its folder stays.

- If a file changes on disk after it was loaded, you cannot save its card. The card offers to discard your edits and load the file again. Copy any edits that you need to keep before you choose this.
- A file marked read-only on disk is not saved, replaced or deleted; its card keeps your edits.
- Cards without edits reload when their files change on disk.
- **Refresh** (`F5`) also reloads the compare rules and the options.
- To include files that changed after you opened **All differences**, save or discard your edits, then open **All differences** again.

6.8. Merging folders automatically

In a three-way folder comparison, the middle folder is the base. **Merge** → **Auto merge folders...** brings the changes from both sides into the middle folder:

- An item that was changed, added or removed on one side only is copied or removed.
- A change that is the same on both sides is applied once.
- A text file that was changed on both sides is merged line by line, if the changes do not overlap.

6.8.1. Before you start

1. In **Options** → **Folder**, turn on **Include subfolders** and **Include contents of unique subfolders**.
2. Let the comparison finish.



The merge writes the result into the middle folder. To keep the original base folder, use a copy of it as the middle folder.

6.8.2. Review the plan

OxideMerge first shows a plan: the items that it will copy, delete and merge, and the conflicts that it cannot resolve. Review the plan before you confirm it.

OxideMerge does not change these files: files with edits that conflict, files with content that it cannot merge, symbolic links, and files that it cannot read. If a file changed after the comparison, refresh the comparison and try again.

6.8.3. How the merge protects your files

The merge is all or nothing: either every change is made, or the middle folder stays as it was.

- *Before the merge*, OxideMerge checks every item. Nothing is merged if an item changed after the plan was made, if an item would be written outside the middle folder, or if a backup file would replace a compared file. OxideMerge lists these items.
- *During the merge*, OxideMerge checks each file again just before it writes it. If another program changes a file, the merge stops. You can also click **Stop** to stop the merge.
- *When the merge stops or fails*, for example because a folder is read-only or the disk is full, OxideMerge undoes all changes made so far. The middle folder is then as it was, and removed

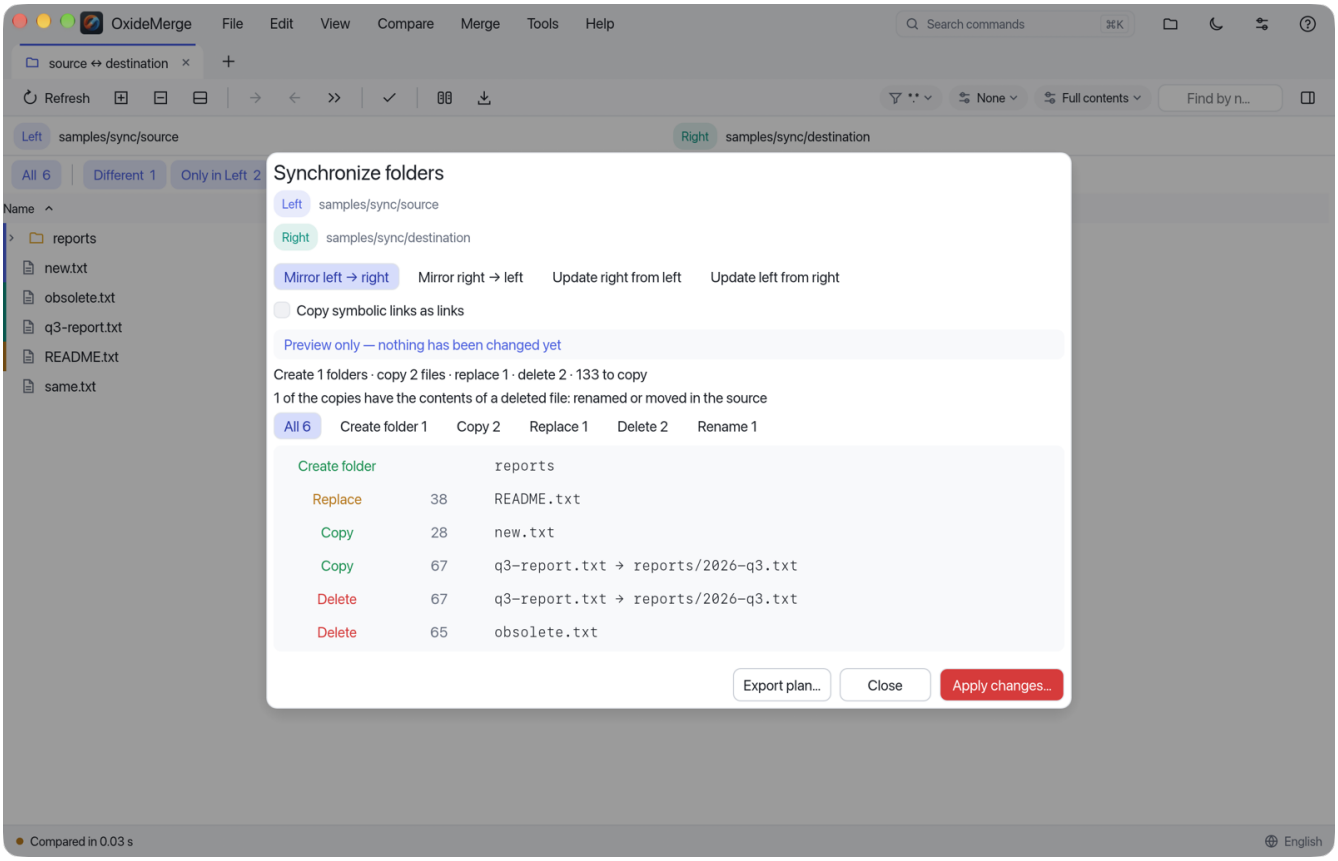
folders get back their permissions. A file that the merge moved to the trash is restored from a copy, so the trash keeps a copy too.

To make the undo possible, OxideMerge keeps copies of the original files in a `.oxidemerge-undo-⋯` folder at the top of the middle folder. If it cannot create this folder there, it uses the system's temporary folder. Folder comparisons do not list this folder. A deleted file is only moved to this folder, and is moved back if the merge is undone.

If OxideMerge cannot put a file back, it keeps the undo folder and shows where it is. The `index.txt` file in that folder lists the original path of each copy, so you can restore the files by hand.

6.9. Synchronizing two folders

In a two-way folder comparison, **Synchronize folders…** (on the toolbar and in the **Merge** menu) makes one folder, the *destination*, match the other, the *source*. You see the plan first. Nothing changes until you apply it.



6.9.1. Modes

Mirror left → right, Mirror right → left

The destination ends up with the files of the source. New and changed files are copied, and files and folders that only the destination has are deleted.

Update right from left, Update left from right

New and changed files are copied. Nothing is deleted.

A read-only side, such as a Git revision, cannot be the destination. OxideMerge decides by the

contents (SHA-256), never by a newer modification time alone. Files that compared identical but differ in size or time are read to be sure.

6.9.2. Before you start

1. Turn on **Include subfolders** and **Include contents of unique subfolders** in **Options** → **Folder**.
2. Let the comparison finish. A stopped comparison cannot be synchronized.

The two folders must be different folders, and neither may be inside the other.

6.9.3. The preview

The preview is read-only. It lists each item with its action and the reason:

Action	Meaning
Create folder, Copy	The item is only in the source.
Replace	The contents differ. The destination file is overwritten.
Delete, Remove folder	The item is only in the destination (mirror only). A folder is removed only when it is empty afterwards.
Skip	The item is only in the destination, and the mode is update.
Conflict	OxideMerge leaves the item alone because it is not safe to decide.

Conflicts are items that cannot be read, symbolic links (never followed or copied), devices, a file on one side and a folder on the other, names that differ from another only in letter case or in how accents on Latin letters are written (composed or not), and items that changed after the comparison. A file whose name the two sides spell differently only in case or in that way, with the same contents on both, is not a conflict when mirroring: it is renamed (see below).

On macOS and Linux, select **Copy symbolic links as links** to synchronize links rather than leave them alone. A link is then treated as the link itself, never as what it points to:

- A link only in the source is made in the destination, with the same target text.
- A link whose target text differs is replaced.
- When mirroring, a link only in the destination is deleted; an update keeps it.
- A link facing a file or folder of the same name is still a conflict.

What a link points to is never read, written or deleted, even when it is outside both folders, and the links changed are kept in the recovery journal like files, so undoing puts them back.

Click an item for its details: the size and the start of the SHA-256 on each side, and whether the action adds to, or overwrites and deletes in, the destination.

A copy whose contents match a file that the plan deletes is a file renamed or moved in the source. **Rename** lists such pairs as *old name* → *new name*, and the copy and the deletion show the other name too. A rename is applied by moving the file in the destination, so nothing is copied or kept in the journal: the file is first renamed to a temporary name, which the journal records, and then to its

new name. A rename that only changes the case of a name, or how its accents are written, is made the same way, also on file systems that take both names for one, such as those of Windows and macOS. When the old and the new place are on different drives, the file is copied and the old one deleted. A folder renamed only in case is still left alone, and so is a file renamed only in case whose contents changed too.

Warnings appear above the list:

- When a filter or the hidden-file setting left items out, a mirror does not delete them, so the destination may not be an exact mirror. A folder that still holds such items is kept.
- Large deletions are flagged and ask for a second confirmation.

Export plan… saves the plan as JSON, with a version number and paths relative to the folders. It does not contain the folders' absolute paths. It records the filter, and a SHA-256 of the two folder paths, so a plan is applied only to the folders it was made for.

6.9.4. Applying the plan

Apply changes… asks for confirmation and names the destination. When many items would be deleted, you must also tick a box that states their number.

OxideMerge then works in this order: it creates folders, copies and replaces files, deletes files and finally removes empty folders. The dialog shows the count and the item being changed. For each item it first checks that the source and the destination are still as in the preview. Every file is copied to a temporary file next to its destination, checked against the planned SHA-256 and then renamed into place; a renamed file is moved instead. At the end, all copied and moved files are checked again.

If a file changed after the preview, even with the same size and modification time, nothing is applied, and the file is named.

A file that cannot be read now is reported as such, not as one that changed: on Windows, another program holding a file open without sharing it stops the synchronization before anything is changed, and the message names that file. Close the other program and preview again.

On Windows, paths longer than the old 260-character limit are copied and deleted like any other.

Nothing is applied either when the new and replacing files, or the journal copies of the files that are replaced and deleted, do not fit in the free space of their drive. The message names the drive, the space needed and the space free. Files that are only moved within the destination do not count. This check is made on macOS and Linux; on Windows the free space is not measured. If a drive still fills up part way, the write fails and the changes are undone.

6.9.5. If something goes wrong

Before it replaces or deletes a file, OxideMerge copies it into a recovery journal in the `sync-recovery` folder of the settings folder, never inside the synchronized folders.

- If a change fails, or you click **Stop**, OxideMerge undoes the changes made so far, last first.
- If a file was changed again meanwhile, OxideMerge does not overwrite it. It reports *Partially*

applied — recovery required and keeps the journal with the copies.

- If OxideMerge ends before the synchronization finishes, the next start offers to undo it or to keep the folders as they are. Until then, these folders cannot be synchronized again.



Applying is not one atomic step for several files. Each file is replaced atomically, but another program can see some files already changed. The trash is not used.

6.10. Documents, spreadsheets and programs

Some files are not text but hold text. OxideMerge reads these itself, with nothing to install:

Files	What is compared
`.pdf`	The text of each page, in the order it is drawn, with a blank line between pages, including the text of forms a page draws. The characters come from the `ToUnicode` table the file carries for its fonts, or from the encoding the font names. An encrypted file is reported rather than read. When part of a file cannot be read — a stream in a compression OxideMerge does not read, or a font with no table of the characters its glyphs stand for — a line at the end says the text is incomplete and why; a file none of whose text can be read is reported.
`.docx`, `.odt`, `.rtf`	The words of the document, a line to a paragraph or heading. The cells of a table are separated by tabs, a row to a line. Words shown as deleted and the codes of fields are left out.
`.xlsx`, `.xlsm`, `.ods`	Every sheet, a line to a row and its cells separated by commas. Empty rows between rows are kept, so rows keep their places; empty rows after the last one are not. A workbook of one sheet opens as a table compare: columns, sorting and **Match rows by** → **Key columns** all apply. A workbook of more than one sheet names each one before its rows and is compared as text, since its sheets are not one table.
ELF programs, shared objects and object files	The header: class, byte order, kind, machine, ABI, entry point and the counts of the headers. Then the program headers, a line to a segment with its type, offset, virtual and physical address, sizes in the file and in memory, alignment and `R`, `W`, `E` flags (and the interpreter a dynamically linked program asks for). Then the section table, a line to a section with its name, type, address, offset, size, alignment and flags. Last, the symbols of each symbol table (`.symtab`, `.dynsym`), sorted by name: a line to a symbol with its address, size, type, binding, visibility and section (`UND` when it is undefined, `ABS` for an absolute value), so two builds line up function by function and a function that grew or moved shows as a changed line. The mapping symbols ARM and RISC-V assemblers add (`$x`, `$d`, …) are left out and only counted, and at most 100,000 symbols of a table are listed.

Files	What is compared
`.ipynb`	A Jupyter notebook, cell by cell: each cell under a line naming it a code cell, a Markdown cell or a raw cell, with that cell's own source and nothing else. Cells are not numbered, so adding one doesn't change the label of every cell after it. Outputs, execution counts and other metadata a notebook carries from being run are left out, so re-running a notebook without changing it compares equal. The older nbformat 3 notebooks are read too, their heading cells as Markdown headings. At most 20,000 cells are read; the end of the text says how many were left out. A notebook that can't be read this way, such as a broken one, is compared as the JSON text it is.

A file read this way is **read-only**: the pane says what was read (for example `report.docx [text of the document]`), and nothing can be edited or saved over the original. The text is a reading of the file, not a copy of it, so the layout, fonts, formulas and anything else the format holds are not compared.

Two or three `.docx` files can also be compared **Formatted**: choose it above the panes. The documents are shown as Markdown, block by block side by side: headings, paragraphs with their bold, slanted and struck-through words, list items, links and tables, each block that differs highlighted with its changed words. Tables are drawn as tables, with merged cells drawn across the rows and columns they cover; a row added inside a merged cell is highlighted where it is, and the rows a header cell reaches down into are part of the header. An image is shown as its description with a short fingerprint of its picture, so a changed picture shows as a change even when its description is the same. **Next difference** and **Previous difference** step through the changed blocks. Like the text, the Formatted view leaves out headers, footers, notes and comments, and shows tracked changes accepted.

Workbooks (`.xlsx`, `.xlsm`) can be compared **Formatted** too, whether they have one sheet or several. Each sheet is a heading and a table, and sheets are lined up by name, so a sheet added or removed shows as one. Merged cells are drawn across the cells they cover. Numbers are shown in their number formats: dates and times in ISO form (`2026-09-30 18:00`) whatever order the format puts them in, and amounts with their decimals, thousands separators, currency and percent signs. A formula shows the result it was saved with. A run of three or more empty rows is shown as one row saying how many there are, so a value that moved down a row still shows as a change. Each picture placed on a sheet is listed after its table with its description, the cell it is placed at and a fingerprint of the picture, so a changed or moved picture shows as a change. Fills, fonts, borders, the formulas themselves and charts drawn from data are not compared.

A tagged PDF file can be compared **Formatted** as well. Most word processors tag the PDFs they export, saying which text is a heading, a paragraph, a list item, a table cell or a figure, and which cells span several rows or columns; the view is read from those tags, so a PDF made from a document reads much as the document itself does. Text in a bold or slanted font keeps that format, unless a whole paragraph or heading is in it, as its style shows it. A figure is shown with its alternate text and a fingerprint of its picture. Running headers, footers and page numbers are left out, as the file marks them as artifacts. A PDF that is not tagged, such as a scan or one from a program that does not tag, says so instead; compare it as text.

The **Text** / **Formatted** switch appears above the panes when every side is a `.docx` file, every side a workbook (`.xlsx`, `.xlsm`) or every side a `.pdf` file, read by OxideMerge itself. It is not offered for `.odt`,

`.rtf` or `.ods` files. The Formatted view reads the files itself even when a converter under **Options** → **Plugins** is set up for them. Whether a block differs depends on its text alone; the comparison options decide only which words of a changed block are highlighted. The view is made again when a file or an option changes. While it is shown, the difference counter on the toolbar still counts the differences of the text comparison; **Next difference** and **Previous difference** step through the changed blocks of the Formatted view.

Some limits and rules:

- A PDF or RTF file can be at most 256 MiB, since it is read whole. A larger one is reported rather than read.
- Each part of a `.docx`, `.odt`, `.xlsx` or `.ods` file can be at most 64 MiB. A file with a larger part, or a part that cannot be read, is reported rather than compared without it.
- An ELF file can be any size: only its headers, section table and symbol tables are read, each at most 64 MiB. Since two different builds can still read the same, a folder comparison by **Full contents** compares programs by their bytes.
- A PDF file can ask for at most 1 GiB of decoded content in all; a file that needs more is reported rather than read.
- The text read from one file can be at most 16 MiB, in the **Text** and the **Formatted** modes alike. The rest is left out, and a line at the end says so.
- In the **Formatted** mode a table shows at most 1,048,576 cells and 1,024 columns; the rows past that are left out, with the same line at the end. A part of more than 3 million elements is too large to show formatted and is reported; the **Text** mode still reads it.
- A workbook none of whose listed sheets can be found in the file is reported rather than compared as empty.

A converter under **Options** → **Plugins** is used in preference to this, so a better reader can always be put in front — `pdftotext` for PDF, or Pandoc or Tika for more formats. What a converter writes is compared as text, even for a spreadsheet. Turning **Read documents, spreadsheets and ELF files as text** off there compares these files as the archive or the bytes they are instead.



The older Word `.doc` and the older Excel `.xls` are not read here yet; use a converter under **Options** → **Plugins** for those. A PDF holds the places of glyphs rather than words, so columns, tables and ligatures may not come out as they look; a converter such as `pdftotext -layout` reads a complicated page better.

6.11. Archives

OxideMerge extracts archives and compares them as folders. It supports these formats:

- ZIP-based formats: `.zip`, `.jar`, `.apk`, `.docx`, `.xlsx`, `.odt`, `.epub`, `.nupkg` and more, with entries stored or compressed with Deflate, Deflate64, bzip2, LZMA, xz, Zstandard or PPMd.
- `.tar`, `.tar.gz` / `.tgz`, `.tar.bz2` / `.tbz2`, `.tar.xz` / `.txz` and `.tar.zst` / `.tzst`.
- `.7z`.

A password-protected ZIP (with either the original ZIP encryption, "ZipCrypto", or WinZip AES) or 7z (including one whose file names are encrypted too) asks for its password in an **Archive password** window when it is opened; a wrong password asks again, and **Cancel** gives up opening it. The password is kept only in memory, only while the compare is open, and is overwritten there once it is no longer needed.

Edits to the extracted files are saved back into the archive: copying, deleting, merging or editing a file and then saving writes a new archive in place of the original, keeping its permissions. Only an archive whose files changed is written, and the entries you didn't change are kept exactly as they were, with their dates, compression and order; changed entries get the current time. A changed ZIP entry keeps its compression method; one compressed with Deflate64 or LZMA, which can only be read, is compressed with Deflate instead, and so is an unchanged LZMA entry, as other tools refuse a copied one. A 7z packs entries together into compressed blocks: a block holding only entries you didn't change is copied as it was, and any other block is compressed again with LZMA2, its entries keeping their dates and attributes.

An encrypted archive stays encrypted: changed and new entries are encrypted with the password it was opened with, the way the archive already was (AES with the same key strength, or ZipCrypto; a 7z's file names stay encrypted if they were). Unchanged AES-encrypted ZIP entries and 7z blocks are copied as they were; unchanged ZipCrypto entries are encrypted again, as a copy would lose the mark that says they are encrypted.

A `.tar.xz` is compressed again as `xz` does by default, and a `.tar.zst` as `zstd` does. Compressing a `.tar.xz` or a 7z can take a while for a large archive, so it is written in the background while a window shows which archive is being written; uploads, closing a tab and quitting wait for it. If the archive itself was changed on disk since it was extracted, or is read-only, saving into it is refused instead of losing those changes. No `.bak` backups are made inside an extracted archive. An archive fetched over SFTP or FTP is uploaded again after it is saved.

An archive inside an extracted archive can be edited too: saving it also writes the archive it came from, and so on out to the outermost one, and an open compare of that archive is compared again.

These stay read-only, as there is nowhere for their changes to go back to, or writing them again would break them:

- an archive opened as a snapshot in a review, or a page tree compared from the web;
- signed packages (`.apk`, `.ipa`, `.appx`, `.msix`, `.crx`, `.xpi`, and any zip carrying a `META-INF` signature or an `_xmlsignatures` folder), as rewriting them invalidates the signature;
- an archive inside one of these. It refuses archives that would extract to more than 8 GB or more than 500,000 files. RAR is not supported.

Chapter 7. Comparing binary files

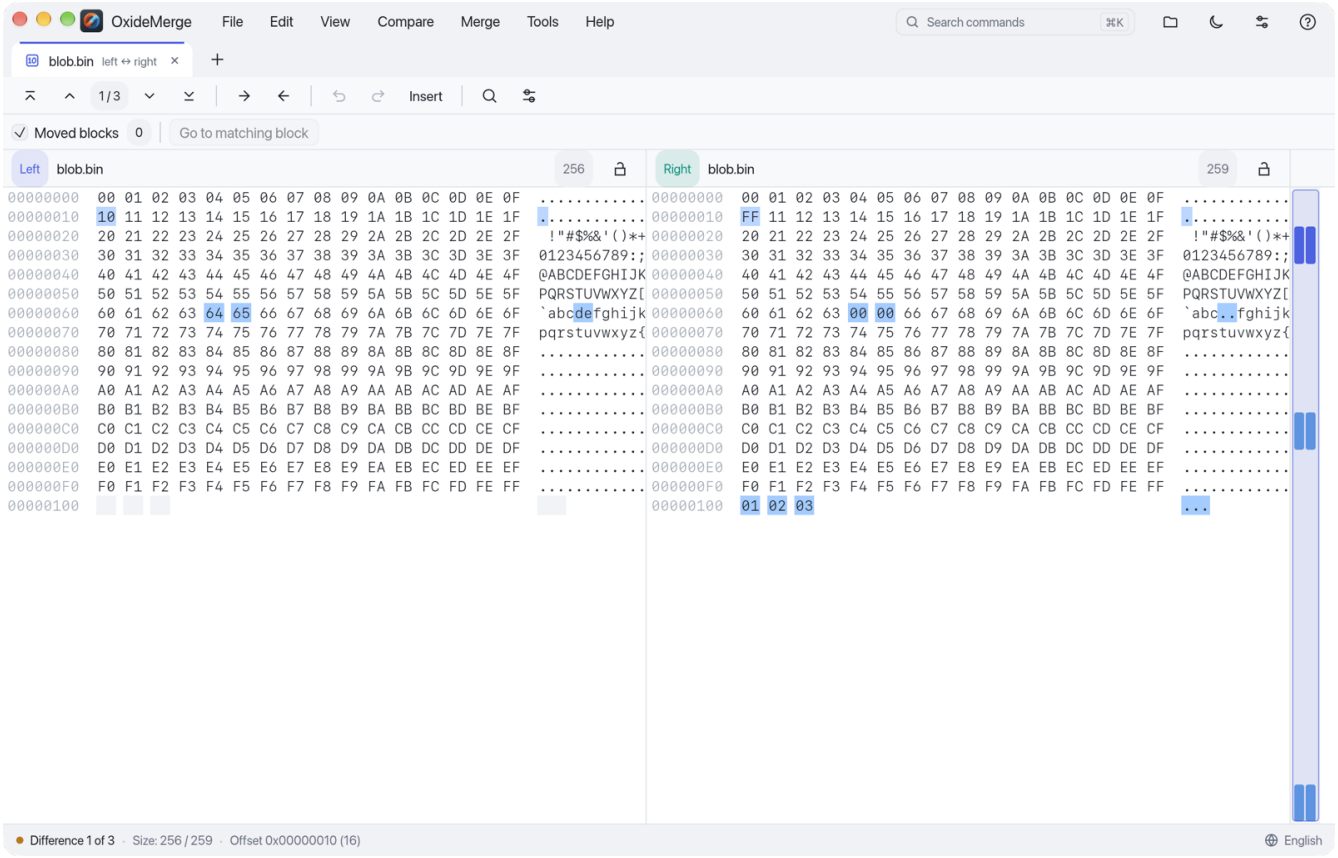


Figure 11. A binary comparison

Binary files open in a hexadecimal view, side by side. Bytes that are different are highlighted, and the sides scroll together.

- `Alt` + `↓` and `Alt` + `↑` go to the next and the previous difference.
- `Ctrl` + `F` finds hexadecimal bytes, such as `DE AD BE EF`, or text.
- `Ctrl` + `G` goes to an offset. Write it as `0x1F00`, `1F00h` or a decimal number.
- You can copy differences to the other side and save them, as in a text comparison.

To change the number of bytes on each line, use **Bytes per line (binary)** in **Options → Image / Binary**.

7.1. Moved blocks

Moved blocks on the binary toolbar highlights identical blocks found at different offsets in the hex and text columns and in the location pane. Select a highlighted byte and use **Go to matching block** to visit its counterpart, keeping your position within the block. You can tab to the button and press Enter. Focus returns to the byte grid so you can continue with the arrow keys. Two-way and three-way comparisons are supported; in three-way comparisons the button uses the first matching side shown beside it. Turn the checkbox off to see only ordinary differences. OxideMerge remembers the checkbox for the next binary comparison.

Detection finds two kinds of identical bytes:

- *Bytes shifted by an insertion or a deletion*: data near an edit that moved up to 128 bytes forward or back, including short runs between two edits.
- *Blocks moved farther or swapped*: data of at least 16 bytes that occurs only once in the file. Repeated data, such as a run of padding bytes, is not matched this way.

The highlights only add information: the differences, copying a difference, the file contents and the saved bytes stay the same.

Detection runs in the background: once when the comparison opens, and after edits, undo, redo or copying a difference, when you stop typing for a moment. An edit, or closing the comparison, stops a scan at once, so detection does not slow down editing or use extra memory. Large files are sampled, so some short matches can be missed; up to 1,024 blocks are shown for each pair of files.

7.2. Editing bytes

1. Click a byte on the side that you want to change. Click in the hexadecimal column to type hexadecimal digits, or in the text column on the right to type characters.
2. Type the new value. In the hexadecimal column, two digits make one byte: the first digit changes the first half of the byte, and the second digit changes the second half. In the text column, each character replaces one byte. Only ASCII characters can be typed.
3. Save with `Ctrl` + `S`.

By default, typing replaces the bytes at the cursor. To insert bytes instead, press `Insert` or click **Insert** on the toolbar. Typing after the last byte always adds bytes.

- The arrow keys, `PgUp`, `PgDn`, `Home` and `End` move the cursor. `Ctrl` + `Home` and `Ctrl` + `End` go to the start and the end of the file.
- `Delete` deletes the byte at the cursor. `Backspace` deletes the byte before it.
- Paste (`Ctrl` + `V`) in the hexadecimal column writes hexadecimal bytes such as `DE AD BE EF`. Other text is refused there. In the text column, the text is written as it is.
- `Ctrl` + `Z` undoes the typing since you last moved the cursor, or a paste.

Sides that are read-only cannot be changed. Click the lock icon above a side to change this.

Chapter 8. Comparing images

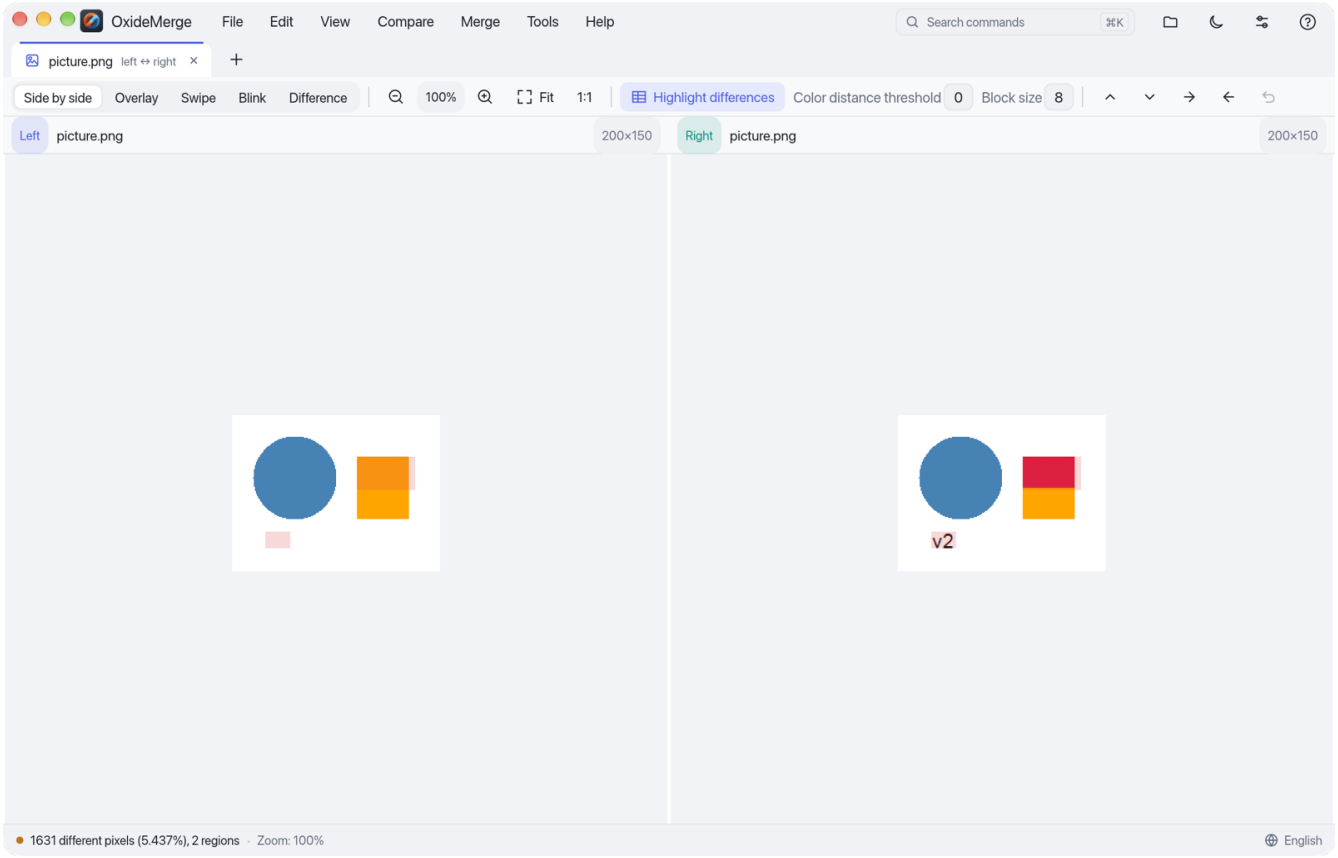


Figure 12. An image comparison

Choose a view mode:

Side by side
: The images are next to each other.

Overlay
: One image is on top of the other. **Opacity** sets how transparent the top image is.

Swipe
: One image is on the left of a divider and the other is on the right. Drag the divider to move it.

Blink
: The images take turns, which makes small changes easier to see.

Difference
: Only the pixel differences are shown.

To find and copy differences:

- **Highlight differences** marks the regions that are different.
  - **Color distance threshold** sets how different two pixels must be to count as different.
  - **Block size** sets the size of the marked blocks.

- Click a region to select it. `Alt` + `↓` and `Alt` + `↑` go to the next and the previous region.
- `Alt` + `→` and `Alt` + `←` copy the current region to the other image. **Undo** reverses the copy. `Ctrl` + `S` saves the changed image.

To zoom and move:

- Zoom with the buttons, the mouse wheel or a pinch gesture. The zoom centers on the pointer.
- **Fit** fits the images to the window. **1:1** shows them at their real size.
- Drag, or scroll with two fingers, to move all images together.

The status bar shows how many pixels are different, and in how many regions.

Chapter 9. Comparing web pages

You can compare two or three web addresses (`http://` or `https://`) or local HTML files. Local `.html` and `.htm` files open as web pages by default. To compare another kind of file as a web page, choose **Compare as** → **Web page**, or use `-t web` on the command line.

9.1. Comparison modes

Choose a mode in the selector at the top of the tab. OxideMerge remembers the last mode you used.

Mode	What is compared
Text	The text of each page, in the text comparison. Each heading, paragraph and list item is one line. The cells of a table row are on one line, separated by tabs. Hidden elements are not included. Changed words are highlighted, and the comparison options, filters and reports all apply.
Formatted	The default mode. Each page is converted to Markdown and shown formatted, side by side. See The Formatted mode.
HTML source	The HTML of each page, in the text comparison. Long minified lines are split between tags.
Resource tree	Each page is downloaded with its style sheets, scripts, images, frames and media. The downloaded files are compared in the folder comparison.

Switching to another mode loads the pages again. After a page changes, click **Reload the pages** (F5).

To compare the HTML source of local files line by line, use the HTML source mode, or choose **Compare as** → **Text** (`-t text`).

9.2. The Formatted mode

The Formatted mode keeps the look of headings, paragraphs, emphasis, links, lists, quotes, code blocks and tables. Images are shown as links.

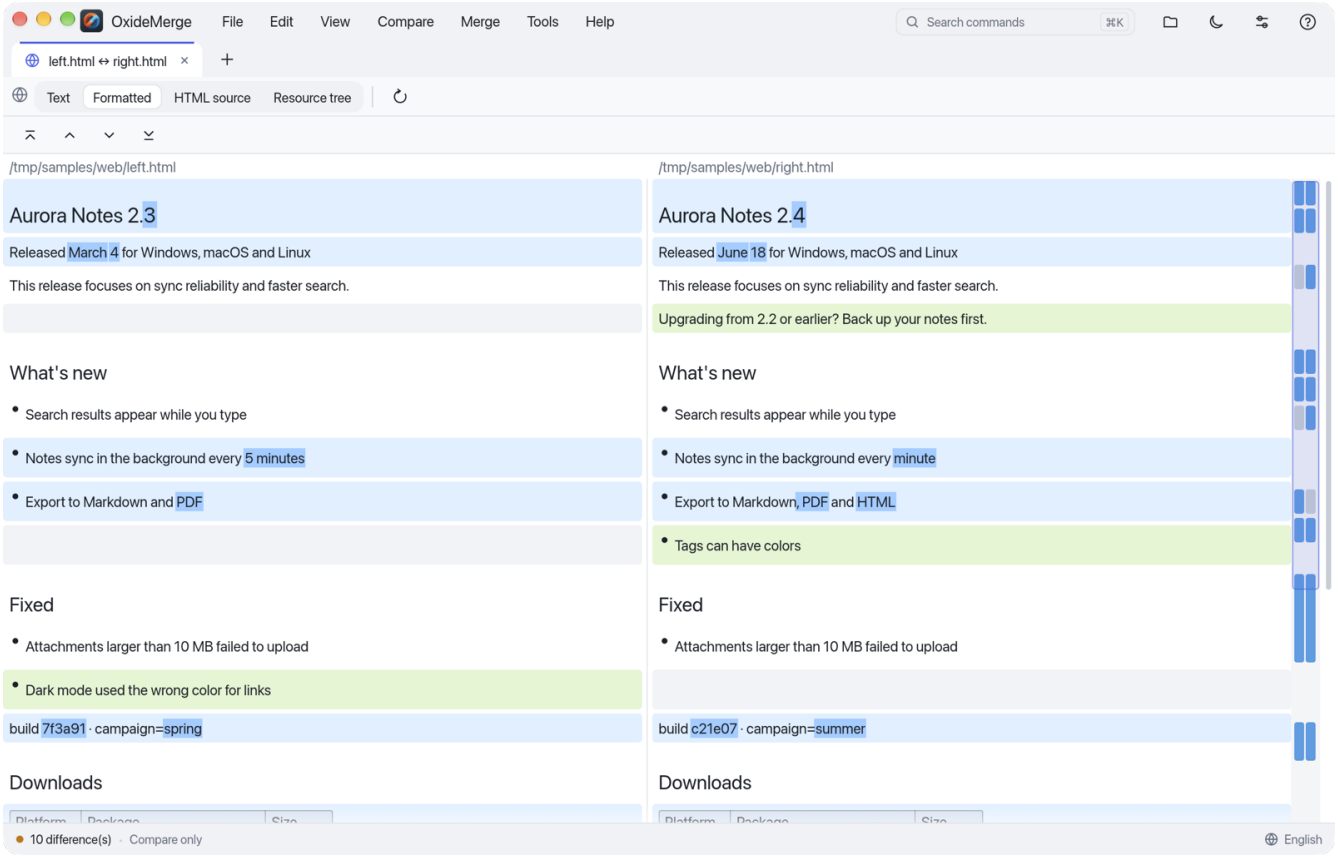


Figure 13. Two web pages in the Formatted mode

OxideMerge compares the pages block by block. A block is a heading, a paragraph, a list item, a table, a quote or a code block.

- Blocks that are different are highlighted. Each one is placed opposite the changed block of the same kind on the other pages.
- A block that is on one page only is placed opposite an empty space.
- Inside a changed block, the changed words or characters have a stronger highlight. The **Inline differences** option controls this, as in the text comparison.
- Tables have lines around their cells, and their rows are aligned. In a row that is different, the changed words are highlighted. If a row or a column is missing on the other page, its cells are highlighted completely.
- A cell that spans rows or columns (`rowspan`, `colspan`) is drawn across the cells it covers, so the other cells of its rows stay in their columns.
- Each changed block is a separate difference, even when it follows another changed block.

To move between differences, use the buttons above the pages, or the same keys as in the text comparison.

- The current difference has a stronger highlight. The blocks that are compared have an outline on every page. The outline is strongest on the page that you clicked last.
- Click a block to make it the current block. The next and previous differences are then found from that block.
- The pages scroll only when the difference is not visible. The difference is then shown about one

third from the top.

The location pane, at the right of the pages, shows all differences. The current difference is in the accent color, and the visible part of the page has an outline. Click or drag in the location pane to go to that place. To show or hide it, choose **View** → **Text compare** → **Location pane**.

9.3. How pages are loaded

Every mode uses the HTML as the server sends it, or as the file is saved. OxideMerge does not open the pages in a browser, so they open quickly. The scripts of the page do not run, so content that scripts add is missing.

The Text and Formatted modes leave out content that the page hides: elements with `hidden`, `aria-hidden="true"`, or an inline `display: none` or `visibility: hidden` style. Content that only a style sheet hides is still compared.

OxideMerge decodes pages the same way as browsers. It uses the first of these that it finds: a byte order mark, the charset in the HTTP `Content-Type` header, a `<meta>` charset, or a guess from the content.

- If the server returns a status code that is not 2xx, the pane header shows it, and the page that came with it is compared.
- OxideMerge reports a page, with its address, if the page cannot be reached, takes more than 30 seconds, sends a response that is not valid, or is larger than 50 MB.

9.4. Resource tree limits

The Resource tree mode downloads only the resources that each page names itself. It does not follow links inside style sheets or scripts.

- Each page can have at most 400 resources, and each resource can be at most 20 MB.
- Resources that cannot be downloaded are counted in the status bar. The other resources are still compared.
- For your safety, a web page cannot load local files. Only a page on this computer or on a private network (such as `localhost` or `192.168.x.x`) can load resources from such an address, or redirect to one. Other resources of this kind are skipped and counted in the status bar.
- The addresses that you enter yourself are always loaded.

9.5. Comparing web pages in the console

The console comparison (`--cli` with HTML files, or with `-t web`) uses the last mode that you chose in the window:

- Formatted (the default) compares the Markdown of the pages line by line.
- HTML source compares the HTML.
- The other modes compare the visible text.



No mode runs the scripts of the page. Logins are not supported, so a page that needs a login can show its login page. No mode saves changes to a web server.

Chapter 10. Controlling which differences are shown

Three kinds of settings decide what OxideMerge shows as a difference:

- Filters select which files and folders a folder comparison includes.
- Comparison options decide which text changes count, for example whitespace or case.
- Compare rules ignore or replace text that matches patterns.

These settings change the comparison result only. They never change the files.



If a change that you expect is not shown, check these settings before you decide that the files are identical.

10.1. Comparison options

Compare → **Compare options** has the most common options. **Options** → **Compare** has all of them:

Whitespace

Compare: whitespace counts. **Ignore changes**: a different amount of whitespace does not count. **Ignore all**: whitespace does not count at all.

Ignore blank lines

Extra blank lines do not count.

Ignore case

Uppercase and lowercase letters are the same.

Ignore carriage return differences (Windows/Unix/Mac)

The line endings `CRLF`, `LF` and `CR` are the same.

Ignore numbers

Digits are ignored.

Ignore code page differences

Files in different encodings that have the same text are the same.

Ignore comment changes

Changes inside comments do not count. This works for languages whose comment syntax is known: C-like languages, scripts, SQL, HTML, XML, CSS, Lua, Haskell and more.

Format JSON and XML before comparing

JSON and XML are indented again, and JSON keys are sorted, so that layout changes do not count. The formatted view is read-only. Turn this option off to edit or merge the files.

Detect moved lines

Blocks of lines that moved are shown as moved.

Diff algorithm

Myers, **Patience** or **LCS**.

Inline differences

How changes inside a line are highlighted: **Off**, **Word level** or **Character level**.

Treat numbers as equal within

Numbers that are at most this far apart, for example `0.001`, are the same. This is useful for measurements and generated data.

In folder comparisons, these options apply to text files when the method is **Full contents**. The other methods do not use them.

10.2. Compare rules

A compare rules file (`.oxrules`) contains rules of three kinds:

Ignore lines matching

One regular expression on each line. A difference that contains only matching lines is ignored.

Substitutions

One rule on each line, written as `regex => replacement`. OxideMerge applies the substitutions to both sides before it compares them, in order, each rule to the text the rules before it left; the changed words within a line are highlighted the same way. Use them for text that is different in an expected way.

Ignore blocks between start and end lines

One block on each line, written as `start regex => end regex`. Differences inside the block are ignored. Use this for a generated section or a license header.

Patterns use the regular expression syntax of the Rust `regex` crate.

10.2.1. Managing rules files

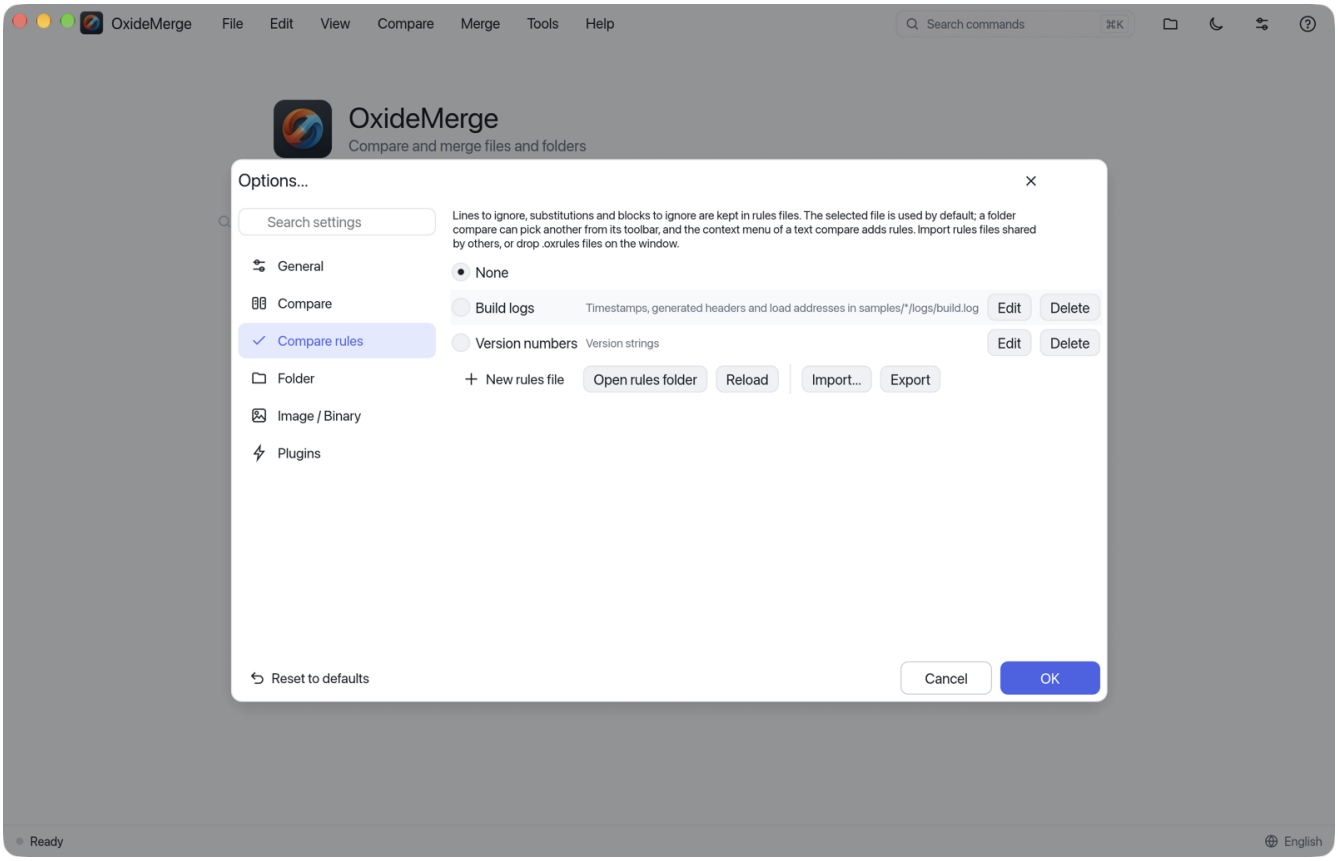


Figure 14. Options → Compare rules

Manage the files in **Options → Compare rules**:

- Choose **None** or a rules file. This file is the default for comparisons that do not choose another rules file.
- **New rules file** creates a file. **Edit** opens a file in the rules editor, and **Delete** removes it.
- **Open rules folder** shows the folder in your file manager. After you change files there, click **Reload**.
- **Import...** adds rules files from other people. **Export** saves one file, or all files to a folder (**All rules files to a folder...**).

A folder comparison can use another rules file. Choose it from **Rules** on the toolbar, or from **Compare → Compare rules**. **Refresh** (`F5`) reads the rules file again.

You can also import rules files by dropping them:

- Drop one `.oxrules` file on the window to import it. If you drop two, OxideMerge compares them instead.
- Drop any number of `.oxrules` files on the options dialog to import them.

When OxideMerge imports a rules file:

- If a file with the same name and the same rules exists, the new file is skipped.
- If a file with the same name but other rules exists, the new file gets a number in its name.
- If a file with the same rules but another name exists, the new file is added with its own name.

10.2.2. The rules file format

Rules files are in the `Rules` folder next to the settings file (see Configuration files). OxideMerge also reads a `Rules` folder next to the program.

A rules file is a JSON file. The name of the rules is the file name without `.oxrules`. This is `samples/rules/Build logs.oxrules`:

Build logs.oxrules

```
{
  "description": "Timestamps, generated headers and load addresses in
samples/*/logs/build.log",
  "ignore_lines": ["^// Generated on .*$"],
  "substitutions": [
    { "find": "^\\[\\d{2}:\\d{2}:\\d{2}\\] ", "replace": "[time] " },
    { "find": "\\b0x[0-9a-fA-F]{8,}\\b", "replace": "0xADDR" }
  ],
  "ignore_blocks": [{ "start": "^/\\* BEGIN LICENSE", "end": "END LICENSE \\*/$" }]
}
```

Every field can be left out:

`description`
: Shown next to the name in **Options** → **Compare rules**.

`ignore_lines`
: A list of regular expressions. A difference whose lines all match one of them is ignored.

`substitutions`
: A list of `find` and `replace` pairs. Each match of `find` is replaced by `replace` in both files before they are compared, in the order of the list. The replacement can use the groups of the match: `$1` is the first group. The panes still show the text as it is in the files.

`ignore_blocks`
: A list of `start` and `end` pairs. A block starts at a line that matches `start` and ends at the next line that matches `end`; both of those lines belong to the block, and differences inside it are ignored. The end can also be later on the start line. A start without an end line is not a block, so nothing is ignored.

Each pattern is matched against one line at a time, so `^` and `$` are the start and the end of the line. In JSON, each backslash of a pattern is written twice: `\\d` in the file is `\d` in the pattern. The rules editor in **Options** → **Compare rules** shows the patterns without that doubling.

10.2.3. Example: build logs

`samples/left/logs/build.log` and `samples/right/logs/build.log` are two runs of the same build. Without rules, nearly every line differs: the build host, the date and every timestamp, and the load address are different each time.

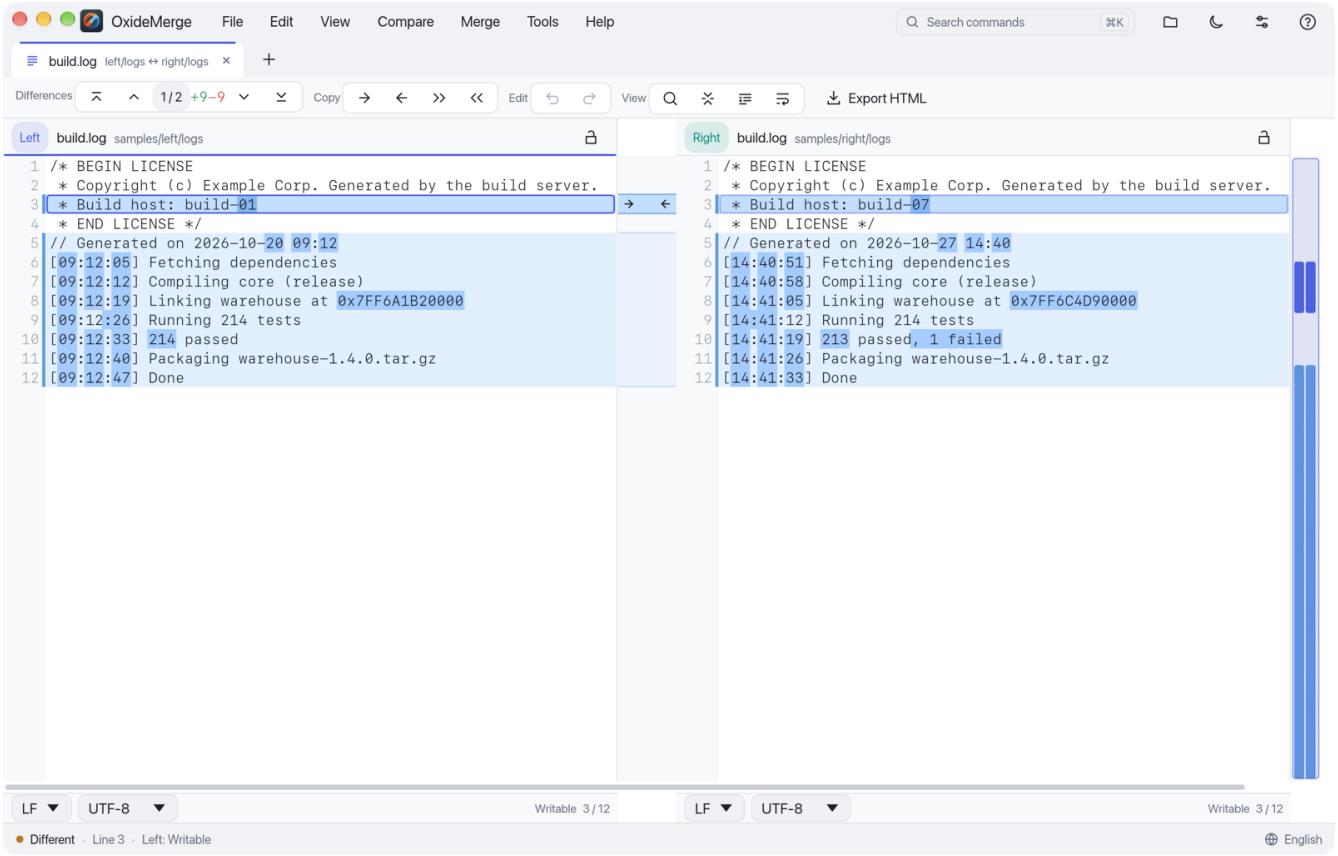


Figure 15. The two build logs without rules

To compare them with the **Build logs** rules, import `samples/rules/Build logs.oxrules` (**Options** → **Compare rules** → **Import…**), select it, and open the two files. Or choose the rules for one comparison on the command line:

```
oxidemerge --rules "Build logs" samples/left/logs/build.log
samples/right/logs/build.log
```

`--rules` also takes the path of the file, so the rules work without importing them: `--rules "samples/rules/Build logs.oxrules"`.

Each rule handles one kind of noise in the logs:

- The ignore block from `^/\* BEGIN LICENSE` to `END LICENSE \*/$` covers lines 1 to 4, the license header, so `Build host: build-01` and `build-07` no longer count.
- The ignored line `^// Generated on .*$` covers line 5, which holds the date of the run.
- The first substitution replaces the timestamp at the start of each line, such as `[09:12:05]`, with `[time]`. The pattern ends with the space after the timestamp, so the replacement does too.
- The second substitution replaces load addresses of 8 or more hexadecimal digits, such as `0x7FF6A1B20000`, with `0xADDR`, so line 8 is the same on both sides.

What is left is the one real difference: line 10, where one test failed. The ignored lines are shown in grey, and the difference counter shows **1/1**. The timestamps on line 10 are not highlighted, because the substitution makes them the same.

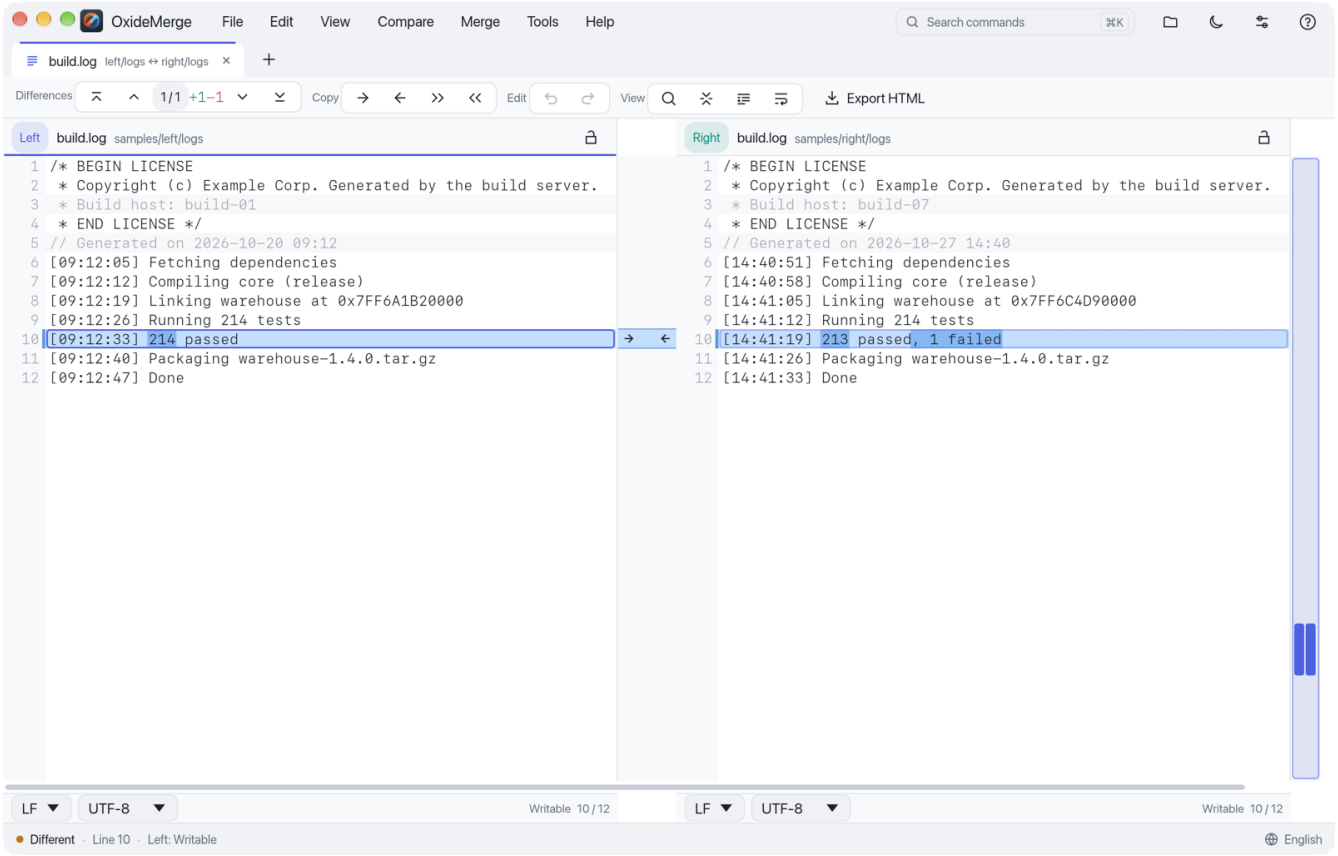


Figure 16. The same build logs with the Build logs rules

Click **Edit** next to **Build logs** in **Options** → **Compare rules** to see the same rules in the rules editor, one on each line. A substitution is written as `find => replace`; there are two spaces before ⇒ in the first substitution, because the pattern ends with a space.

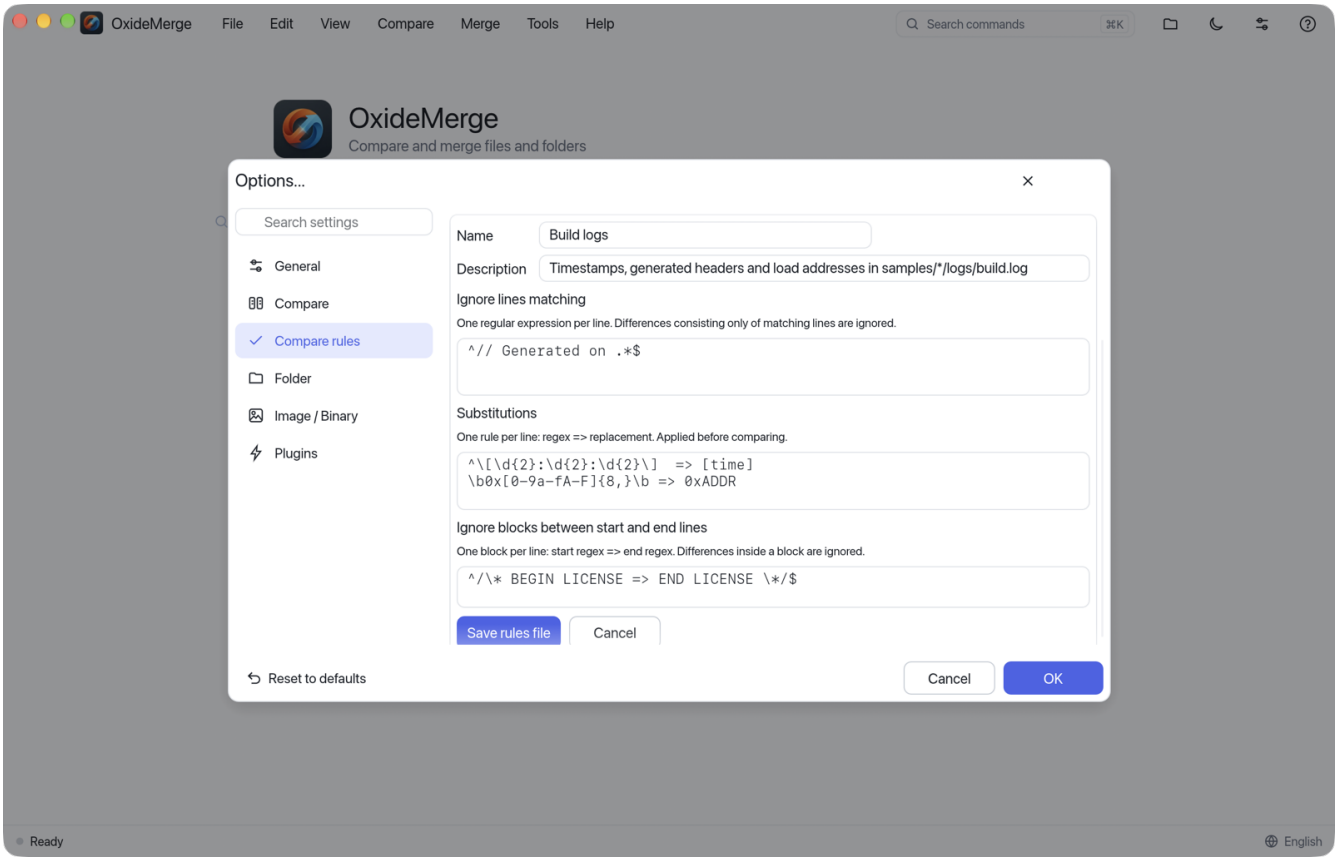


Figure 17. The Build logs rules in the rules editor

In scripts, the same rules decide the exit code. `oxidemerge --cli --rules "samples/rules/Build logs.oxrules" ⋯` exits with `1` for these logs, because of line 10. When the only differences are those the rules ignore, it prints nothing and exits with `0`.

10.3. Creating a rule from a text comparison

1. In a text comparison, right-click a line and choose **Add compare rule**.
2. Choose the kind of rule:

   Ignore lines like this

   A pattern that matches the line, or each side of a changed line.

   Substitute

   A substitution that changes the text of one side into the text of the other side.

   Ignore block

   A block that starts at this line.

3. Check the pattern that OxideMerge suggests, and change it if necessary.
4. Choose the **Rules file** for the rule, or enter a **File name** to create a new file.

The current comparison uses the rule at once. Other folder comparisons that use the same file use the rule after you refresh them (`F5`).

10.4. Filters

A filter decides which files and folders a folder comparison includes. Choose a filter from **Filter** on the folder comparison toolbar. To manage filters, choose **Tools** → **Filters...**.

A filter mask is a list of wildcards, separated by `;`.

- A `!` at the start excludes the matching items.
- A `\` or `/` at the end matches folders.

For example, this mask includes Rust and TOML files, and excludes the `target` folder:

```
*.rs;*.toml;!target\
```

OxideMerge also lists WinMerge `.flt` filter files. It reads them from the `Filters` folder in the configuration folder, and from a `Filters` folder next to the program. **Tools** → **Filters...** shows these filters, opens the filters folder and creates new filters.

10.5. Unpacker plugins

An unpacker plugin converts a file to text before the comparison. OxideMerge compares the text that the program writes to its standard output.

OxideMerge reads PDF, `.docx`, `.odt`, `.rtf`, `.xlsx`, `.ods` and `.ipynb` files, and ELF programs, itself (see Documents, spreadsheets and programs), so a plugin is needed only for a format it does not read — the older Word `.doc` and Excel `.xls` among them — or to read one of those formats better than it does. A plugin whose pattern matches is always used in preference.

To add a plugin:

1. Install the conversion program. Make sure that the command can find it.
2. In **Options** → **Plugins**, add an entry with a **File pattern**, such as `.docx, and a *Command line`. In the command line, `${file}` is replaced by the path of the file, with quotes.
3. Turn on the entry.

The preset buttons add entries for PDF text (`pdftotext`), Office and OpenDocument files (`pandoc`) and general document extraction (`tika`). These entries start turned off. Move the pointer over the checkbox of an entry to see which program it needs. This check looks only at the first program name of the command. If the command fails when it runs, OxideMerge shows the output of the converter.

For example, with Pandoc installed:

```
pandoc -t plain ${file}
```

Converters are used everywhere that files are compared as text: single files, folder comparisons with **Full contents**, the command line, the file compare pane, **All differences** and the differences in folder reports. The size, date, quick and binary folder methods still compare the original files.

Some limits and rules:

- The extracted text is read-only. It can never overwrite the original document.
- Each conversion can run for at most 30 seconds. Each output stream can be at most 16 MiB.
- If the program exits with a nonzero code, this is an error, even if it wrote some text.
- If you stop a folder scan, its conversions stop too.

Chapter 11. Saving, sharing and reporting

OxideMerge can save your work in several formats. Choose the format by what you need:

Format	File	Use it to
Project	`.oxmproj`	Open the same comparison again later. It stores paths and settings, not file contents. See Projects.
Workspace	(kept in your settings)	Reopen a named set of open tabs at once, alongside whatever is open already. See Workspaces.
Saved comparison	`.oxmcmp`	Keep the text of a text comparison, with bookmarks and comments. See Saved comparisons.
Folder review	`.oxmreview`	Keep a complete folder comparison, with file contents and review notes. See Folder reviews.
HTML report	`.html`	Share the differences with people who do not have OxideMerge. See HTML reports.
Other reports	`.csv`, `.txt`, `.xml`	Summaries for spreadsheets, text files and other programs. See Other reports.
Slideshow	`.html`	Present the differences one by one in a meeting. See Slideshows.
Patch	`.patch`	Send the changes in a format that other tools can apply. See Patches.



Saved comparisons and folder reviews contain the full content of the files. Reports contain the differences and the lines around them. Check these files for confidential content before you share them.

11.1. Projects

File → Save as project... saves what the current tab compares in an `.oxmproj` file. A project stores:

- the paths, the descriptions and the read-only sides;
- the comparison type, the filter, **Include subfolders** and the compare method;
- the compare rules file.

File → Open project... opens a project again. OxideMerge can also open WinMerge projects (`.WinMerge`).

Paths are stored relative to the project file, so you can move a project together with the files. Paths on another Windows drive stay absolute, and web addresses do not change.

A project does not contain the files. When you open it, OxideMerge loads the files that are at those paths now. To move a project to another computer, copy its files and the external rules files that it uses too. To share the text as it is now, with your review notes, save a comparison instead.

11.2. Workspaces

File → **Workspaces** → **Save as workspace…** saves every open text, binary, image, web and folder comparison tab that has files of its own under a name you choose (up to 80 characters). The name and the tabs' paths and settings are kept in your settings, not in a file of their own. Tabs with nothing of their own to reopen from are left out, the same as the session OxideMerge reopens automatically leaves them out: **All differences** and review tabs, the clipboard, and tabs of temporary copies, such as a file or folder fetched over SFTP or FTP (see Comparing over SFTP or FTP), a revision, or a file opened from inside an archive compare. A message says how many tabs were saved and how many were left out. When none of the open tabs can be saved, the dialog says so and nothing is saved.

Names are told apart regardless of case. Saving under the name of a workspace that already exists says so, and replaces its tabs only once you click **Replace**.

File → **Workspaces** then lists the saved workspaces (the first 15; **Manage workspaces…** lists them all): choose one to reopen all of its tabs at once, alongside whatever is open already. A tab whose files are gone is not reopened, and a message says how many could not be. **Manage workspaces…** lists the saved workspaces, to open or delete one; a deleted workspace can be brought back with **Undo** in the message that follows.

A workspace is unlike the one, unnamed session OxideMerge reopens on its own when it starts: that session is kept and replaced automatically, while a workspace is kept, and reopened, only by the name you gave it. When two OxideMerge windows run at once, a workspace saved or deleted in one is kept when the other saves its settings.

11.3. Saved comparisons

File → **Save comparison…** saves a text comparison in an `.oxmcmp` file. The file contains the text of each side, the bookmarks and comments, and the synchronization points.

Open the file later, or send it to another reviewer. It opens read-only, and it does not need the original files. It is a snapshot of the text: unlike a project, it does not load the current files from their paths.

11.4. Folder reviews

When a folder comparison has finished, **File** → **Save comparison…** saves an `.oxmreview` file. This file is a portable bundle that contains all items that the filter includes.

- To save only some items, select files or folders, right-click and choose **Save selected folder review…**. A selected folder includes all of its scanned contents.
- The **Show** settings and the chips only change what you see. They do not remove items from a complete bundle.

- Finish all file operations first. If the files changed, refresh the comparison before you save.

To add notes, right-click a file or folder and choose **Toggle bookmark** or **Bookmark comment**. The name of a bookmarked item starts with *, and its comment appears when you move the pointer over it. The notes are saved in the bundle, and they appear in HTML, XML and plain-text folder reports. CSV summaries do not include them. If you change the notes in an opened bundle, save the comparison again to keep them.

To open a bundle, use any of these:

- the command line: `oxidemerge review.oxmreview`;
- drop the file on the window;
- **File → Open project...**.

The bundle opens as a read-only snapshot, with the comparison options that were used. It does not need the original files, named filters or external rules files. Converted documents are stored as their extracted text, and the converter commands are not stored, so the reviewer does not need the converter programs. In the tree view, expand the folders to see their contents.

Limits:

- Two or three sides, at most 10,000 items and 256 MiB of uncompressed content.
- OxideMerge refuses symbolic links, special files, paths that are not safe on other systems, and files that changed after the comparison.
- A bundle contains the full content of each file, including binary files.

The review workspace adds review decisions to folder reviews. See Review workspace.

11.5. HTML reports

To create an HTML report:

1. Open a text or folder comparison, and wait until it has finished.
2. Check the comparison options, compare rules and folder filters. They change which differences the report shows.
3. Click **Export HTML** on the toolbar, or choose **File → Export → HTML report...** (`Ctrl` + `Shift` + `E`).
4. Choose the output file. The suggested name is made from the compared names, with `_` in place of spaces.
5. OxideMerge opens the report in your browser. Check what it contains before you share it.

The report is one HTML file that contains everything it needs. The recipient can open it in any browser, without OxideMerge.

- Each difference is a card, with line numbers, three lines of context and the changed words highlighted.
- The report has summary cards, an index, a side-by-side or unified view, light and dark themes, a

file filter and search.

- `N` and `P` go to the next and the previous difference, which pulses briefly.
- The header carries the OxideMerge icon, and the page animates as it opens: the sections fade in and the counts in the summary cards count up. A browser set to reduce motion shows a still page, and so does printing.
- The report names the compare rules file and counts the ignored differences.
- A folder report includes the differences of each changed text file.

To save reports without opening them, turn off **File** → **Export** → **Open reports in the browser after saving**.

Exporting a report never saves your edits to the compared files.

11.6. Other reports

File → **Export** → **HTML report...** also saves other formats: choose the file type in the save dialog.

- A folder comparison: HTML (with the differences of each changed text file), CSV, plain text or XML. CSV and plain text list the files with their status, sizes and dates.
- A text comparison: HTML or XML.

Tools → **Statistics...** shows numbers about the current comparison:

- for a folder comparison: the numbers of identical, different and unique files, the errors, and the total size of each side;
- for a text comparison: the identical, changed, added, removed and ignored lines, for each pair of sides.

11.6.1. XML reports

XML reports are for other programs, such as scripts, review tools or a build that checks what changed. Choose the XML type in the save dialog of **File** → **Export** → **HTML report...**, or use `--report <name>.xml` on the command line.

- A text comparison report lists each difference. For each one, it gives the kind, whether it is ignored, and the lines of each side with their numbers. The bookmarks and their comments come last.
- The kinds are `changed`, `added` and `removed`. Three-way comparisons also use `left-only`, `right-only`, `both-same`, `compatible` and `conflict`.
- A folder report lists each file and folder, with its status and its date and size on each side.

```
<comparison format="1" tool="OxideMerge" version="0.9.8" type="text" created="…">
  <side index="0" role="left" name="a.txt"/>
  <side index="1" role="right" name="b.txt"/>
  <statistics differences="1" ignored="0" identical-lines="1" changed-lines="1" added-
lines="0" removed-lines="0"/>
```

```
  <differences>
    <difference index="1" kind="changed" ignored="false">
      <lines side="0" start="2" count="1"><line number="2">two</line></lines>
      <lines side="1" start="2" count="1"><line number="2">TWO</line></lines>
    </difference>
  </differences>
</comparison>
```

- Line numbers start at 1.
- If a side has no lines in a difference, its `count` is `0`, and `start` is the line that the text would go before.
- The `format` number changes only if the layout changes in a way that could break a program that reads it.

11.7. Slideshows

File → **Export** → **Slideshow report...** saves an HTML page that shows one difference at a time. Use it to present a change in a meeting or a review.

- The title slide names the compared sides and counts the differences.
- Each difference then has its own slide, with its lines, the lines around it, the changed words and any bookmark comments.
- For a folder comparison, there is a slide for each difference of each changed text file. The last slide lists the files that are different in other ways, such as binary files or files on one side only.
- Ignored differences are not included.

In the browser:

- `→`, `←`, `Space`, `PgUp` and `PgDn` go to another slide.
- `Home` and `End` go to the first and the last slide.
- `F` switches to full screen.
- Each slide has its own address, such as `#3` for the third slide, so you can link to it.
- When you print the page, each slide is printed on its own page.
- The header carries the OxideMerge icon, which links to the project page, as does the last slide.

11.8. PDF reports and printing

File → **Export** → **PDF report...** saves the report of a text or folder comparison as a PDF file. **File** → **Print...** (`Ctrl` + `P`) writes the same PDF to a temporary file that only you can read and opens it in your system's PDF viewer, where you print it. The temporary file is deleted when OxideMerge closes. No browser is used.

A PDF report holds what the HTML report holds:

- The compared files, the compare rules file, the counts of differences and of added, removed and changed lines, and bookmark comments.
- Every difference with three lines of context, line numbers and the changed words marked, side by side. With the inline view switched on, a two-way report has one column, with the removed lines above the added ones, on portrait pages.
- For a folder comparison, the list of files with their status, sizes and dates, then the differences of every changed text file, review notes and the review summary.
- What matching by key found, for a table compared by key.

Each page has a header with the OxideMerge icon and the report title, and a footer with **Page n of m** and **OxideMerge**, which links to the project page (https://github.com/kuopinghsu/OxideMerge). The PDF has a bookmark for each file and each difference. The fonts OxideMerge uses are embedded, only with the characters the report needs, so Chinese, Japanese and Korean text prints as it shows on screen. Each column is labeled with its side, and on a page a difference continues on, with its file as well. In a folder report, the header of the file list is repeated on each page.

A report draws at most 20,000 lines of compared text (about 400 pages), all files together; a long line that wraps counts once for each line it takes. A difference shows at most 400 rows, unless the whole file is shown. When one is cut, at most 100 unchanged lines are kept above the change, so the change itself is always shown, and a line such as **… 1,200 more lines** says what is left out. The differences after the limit are only counted.

Lines read left to right, and Hebrew and Arabic text in them right to left, with the numbers and brackets in it in their places (a date, phone number, percentage or price stays whole) and Arabic and Persian letters joined. This is simpler than a browser's layout, so a line mixing the two directions in unusual ways can differ from it. Code fonts do not join `!=` or `->` into one sign. Hebrew, Arabic and Thai print with an installed font whose licence allows embedding a subset of it, such as SF Hebrew, Geeza Pro, Tahoma, Segoe UI or Noto. A character none of the fonts has prints as `?`; a note at the end of the report (and on the command line, on standard error) lists such characters by code point.

Color is never the only cue, so a report printed in black and white still shows what changed:

- Each changed line starts with a sign: `-` removed, `+` added, `~` changed, `!` conflict, `=` ignored.
- Changed words are underlined as well as highlighted.
- The place of a line missing on one side is hatched.
- With **Color-blind friendly diff colors** on in **Options** → **General**, blue and orange take the place of green and red, changes are purple and conflicts yellow.

Pages are A4 and landscape. Select **File** → **Export** → **US Letter pages for PDF reports** for Letter pages. On the command line, `--page-size`, `--portrait`, `--unified`, `--whole-file` and `--context` set the layout (see Command line):

```
oxidemerge old.c new.c --report changes.pdf
oxidemerge -r old new --report review.pdf --report-diffs --page-size letter
oxidemerge old.c new.c --report full.pdf --whole-file --unified --portrait
```

11.9. Patches

File → Export → Generate patch… writes the differences between two files or folders as a patch. Choose the format (**Normal**, **Context** or **Unified**) and the number of context lines.

- You can create patches from two-way comparisons only.
- The ignore options still decide which differences are exported.
- Patches support UTF-8 files, with or without a byte order mark, with LF, CRLF or mixed line endings, and files without a final newline.
- Files in other encodings, or with CR-only line endings, must be converted first. For these files, OxideMerge shows an error instead of creating a patch that would not reproduce the changes.

File → Apply patch… applies a unified diff, including the output of `git diff` and `svn diff`.

- OxideMerge finds each hunk even when its lines have moved. A hunk is one block of changes in the patch.
- Each patched file opens next to its original, so you can check the result and save it.
- Hunks that cannot be applied are reported. Read these messages and check the comparisons before you save. Applying a patch does not save any result automatically.

Chapter 12. Analysis tools

OxideMerge has optional tools that help you check merges and review larger changes:

- Merge verification previews a merge and checks a saved result.
- Semantic comparison compares JSON, XML, TOML and YAML files by their structure, and Tree view shows two of them as one aligned tree.
- Review workspace records review decisions for each file and difference.
- Release summary / Compare baselines summarizes the changes between two releases.
- Potential impact lists the places where changed files may be used.

These tools are read-only: they never change your files. They do nothing until you open them. In the window, they are in the side drawer: click **Side panel** at the right end of the toolbar, then choose a tab. You can also run them from the command line (see Running analyses from the command line).

For the exact rules of each analysis, see the analysis guide.

12.1. Merge verification

12.1.1. Text files

In a three-way text comparison, open the **Merge verification** tab.

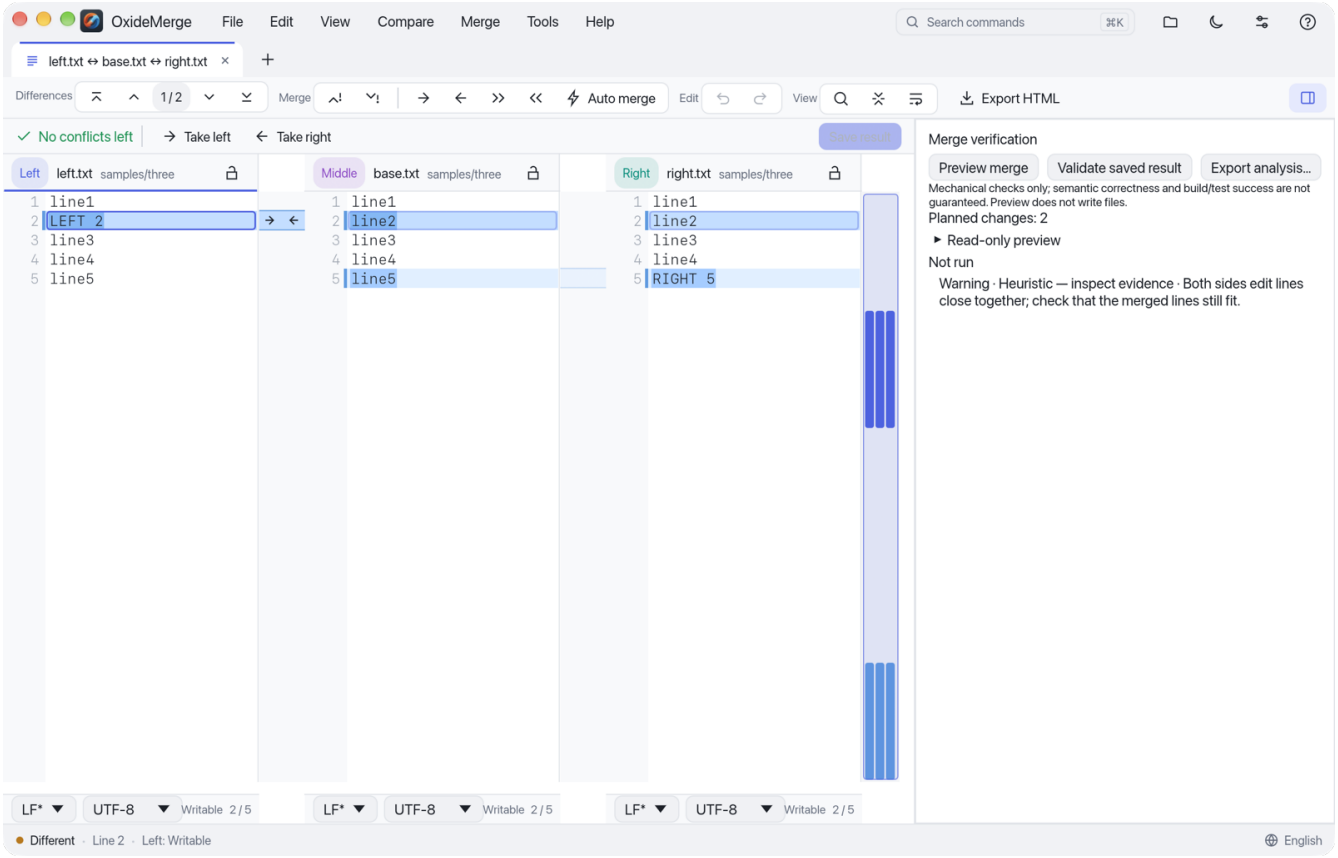


Figure 18. Merge verification after Preview merge

- **Preview merge** shows the result of the merge as a **Read-only preview**. It does not change the

middle pane. Conflicts keep the base text in the preview; you must resolve them yourself.

- **Validate saved result** reads the saved result file again. It compares the bytes, the encoding, the byte order mark, the line endings and the final newline with the preview.
- If you edit the text, the validation status that is shown is no longer valid.

The preview also warns about merges that apply without conflicts but need a careful review:

- the two sides edit lines that are at most two lines apart;
- both sides change the same setting in different places, such as `name = value`, `name: value` or `"name": value`.

A warning does not make the check fail.



These checks compare text and bytes only. They never prove that the merged result is correct.

12.1.2. Folders

`--merge-plan` lists the operations and the conflicts of a folder merge. It does not carry them out.

`--validate-merge` checks a merge result against the plan. It checks the planned copies, merges and deletions and the files that stay unchanged, and it checks again that the source files did not change. The result can be in a separate folder:

```
oxidemerge --validate-merge result-folder left-folder base-folder right-folder
```

Recommended order: preview the merge, run the merge, then validate the result against the plan.

12.2. Semantic comparison

For JSON, XML, TOML and YAML files, choose **Semantic** to compare the structure instead of the lines. Comparing never writes to the files; only the copy buttons described below change a pane. If OxideMerge cannot read a file, the view shows why, and one click takes you back to the text view.

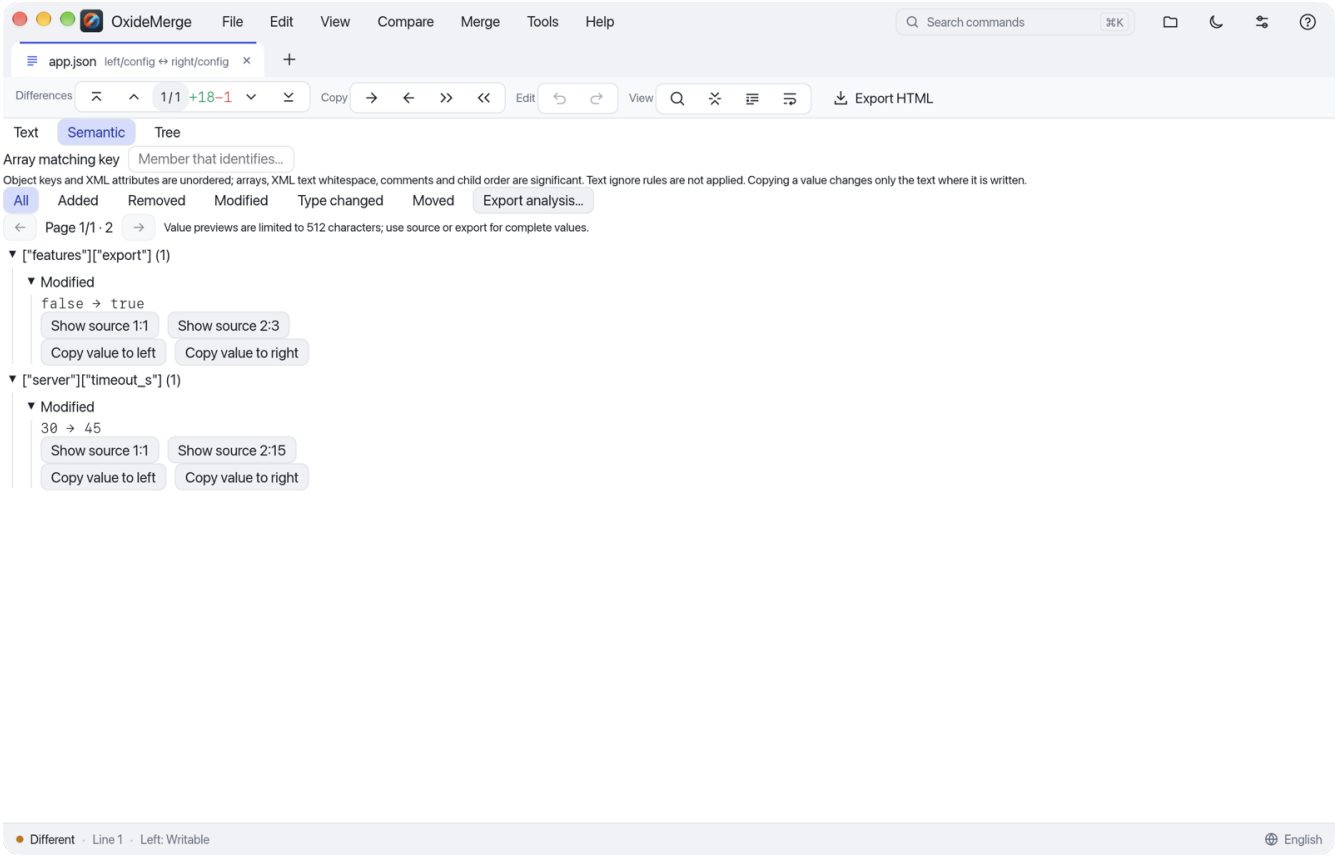


Figure 19. Semantic comparison of two JSON files

12.2.1. What counts as a change

JSON

- The order of object keys does not count.
- The order of array elements counts.
- A missing value and `null` are different.
- Duplicate keys are an error.

XML

- The order of attributes does not count.
- Names are compared with their full namespace.
- The order of child elements, whitespace in text, mixed content and comments all count.
- DTDs, declared and unknown entities, and XML 1.1 are not supported. Built-in character references are supported.

TOML and YAML

- They are compared like JSON: the order of tables and mappings does not count.
- Numbers are compared by value, also between integers and decimals. For example, `1` equals `1.0`, and in TOML `8_080` equals `8080`.
- Comments and layout do not count.
- TOML date-times with a time zone are compared by the moment they name. For example, `10:00+02:00` equals `08:00Z`.

- YAML follows the YAML 1.2 core schema, so `yes` and `on` are text, and `0777` is the number 777, not an octal number as YAML 1.1 reads it. Keys are compared by value too, so `0x10:` and `16:` are the same key.

Limits:

- A TOML or YAML file can be at most 4 MiB.
- A YAML file must contain one document, without aliases, complex keys or custom tags. A core tag such as `!!int` must match its value.

Compare rules for text are not used as filters in the semantic view.

12.2.2. Reading the results

Changes are shown as a tree of their paths. Each branch shows how many changes it contains. Click the source button of a change to go to its line in the file.

Arrays need special care:

- If an array has the same elements in a new order, the elements that moved are listed as **Moved**, with their old place.
- To pair the objects of an array by a member instead of by position, type the member name, such as `id`, in **Array matching key** (JSON, TOML and YAML). An insertion or a new order is then shown as added and moved elements, not as a change at every later position.
- If the key cannot pair an array, because an element has no unique simple value for it, a note above the changes says so. That array is then compared by position.

Each change tells how its array elements were paired when this was not by path alone: by the array matching key, as an exact new order, or by position in arrays of different lengths.

A change is marked **ambiguous** when another explanation fits equally well. Check these changes yourself. Examples:

- In a swap of two elements, either element could be the one that moved.
- A moved element has duplicates.
- An insertion moved the elements, and the change was found by position.

Reports record for each change its `confidence` (`parser_exact`, `array_key`, `reordered` or `position`), `ambiguous` when it is set, and its `notes`.

12.2.3. Copying values

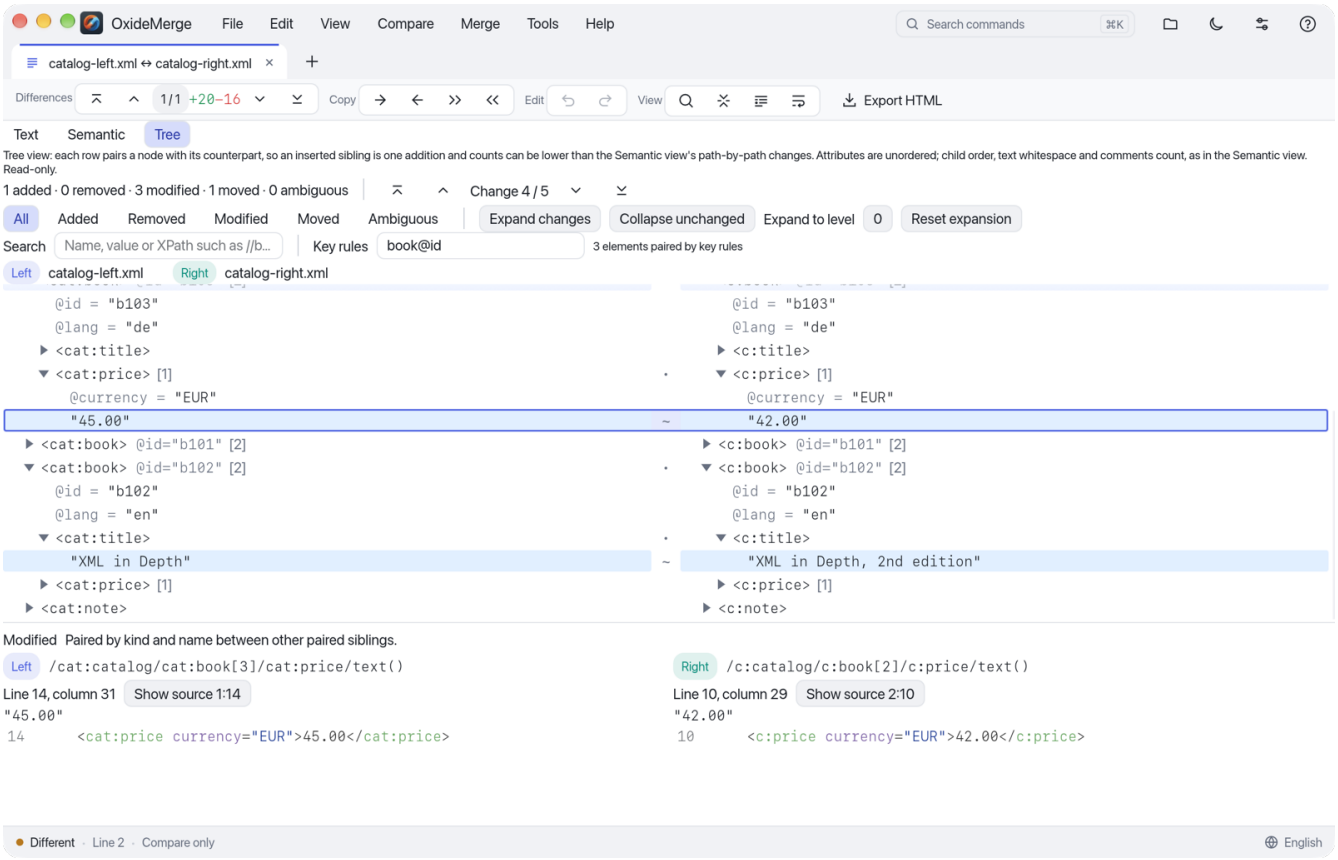
Each change has **Copy value to left** and **Copy value to right** buttons for the panes that are not read-only. A copy replaces the value on the other side, adds it where it belongs, or removes it when the source side does not have it.

- Only the text where the value is written changes. Comments, layout and the rest of the file stay as they are.

- The edited file is read again before the copy is kept. If the result is not exactly the expected structure, nothing changes and the view says why. Edit such a value in the text view.
- Moves are not copied, because a move changes the order of the whole array.
- A copy is one step of **Undo**. The file is written only when you save it.

12.2.4. Tree view

For two JSON, YAML, TOML or XML files, **Tree** shows both documents as one aligned tree. A node and its counterpart share a row: the left file on the left, the right file on the right, so both sides scroll together. A node that exists on one side only has an empty placeholder row on the other side. The view only reads the files.



The column between the sides tells what each row is, also without color: `+` added, `−` removed, `~` modified, `↕` moved, `•` changes below, and `?` ambiguous. A folded node with changes inside shows how many, such as **3 changes**. What a row shows depends on the format:

- **JSON, YAML and TOML**: a key and its value, such as `port: 8080`; an object or array with the number of members, such as `env {2}` or `ports [2]`; and an array element by its position, with its `id`, `name` or `key` member, such as `[0] id="report"`.
- **XML**: elements by their name as written, with their `id`, `name`, `key` and `ref` attributes and any changed attributes as short labels. Expand an element to see its attributes, text, comments and processing instructions.

Rows are paired below paired parents, so a node is never shown under another parent:

- Object members, and XML attributes, are paired by their key or name, whatever order they are

written in.

- Array elements and XML child nodes are paired sibling by sibling:
  1. by a key rule you give, or for an array of objects that all have an `id`, `name` or `key` member with a different value, by that member (see **Key rules** below);
  2. as identical content, keeping the order;
  3. as identical content out of order, when that content occurs once on each side: a **Moved** row;
  4. by kind and name between the rows paired so far.

Inserting an element is one addition: the elements after it stay paired with their counterparts. Because the Semantic view compares by position, it can list more changes for the same files. What counts as a change is the same as in the Semantic view:

- **JSON, YAML and TOML**: key order, comments, quoting and block or flow style do not count. In YAML and TOML, how a number is written (`8_080`, `0x1F90`, `1.0`) does not count either; in JSON, `1` and `1.0` differ. A value that changed type, such as a number that became a string, is shown as **Type changed**.
- **XML**: attribute order and namespace prefixes do not count, CDATA is text, and child order, whitespace in text and comments count. Rows of whitespace-only text that did not change are left out, and whitespace added or removed next to an added or removed element, such as its indentation, is part of that element.

Moving between changes

The previous and next difference commands (`Alt+↑` and `Alt+↓`, `Alt+Home` and `Alt+End`) and the buttons above the tree go through the changes in tree order. A change inside a folded node is not skipped: its parents unfold. **Change 3 / 12** shows where you are. The filters show and visit only added, removed, modified, moved or ambiguous changes, and the change counts on folded rows follow the filter.

Keyboard

The tree is one stop in the tab order, as a tree is: press `Tab` until it has an outline, or click a row. The arrow keys then move: ↑ and ↓ from row to row, → to unfold a row, and ← to fold it or to go to its parent. `Tab` leaves the tree rather than stepping to the next row, and `Esc` lets go of it. While the tree has the keyboard, a screen reader reads out the selected row: what it is, then each side. Every row is in the accessibility tree to be read one by one, whether the tree has the keyboard or not.

Folding

Expand changes (also the starting state) unfolds every node with changes inside. **Collapse unchanged** folds the rest, **Expand to level** unfolds to a depth, and **Reset expansion** returns to the start.

Details and source

Below the tree, the selected row shows its path on each side (such as `$.server.port` or `/catalog/book[2]/@id`), how it was paired, its line and column, its value and the lines of the file where it is written, as they are in the file. **Show source** opens the Text view at that line. Choosing **Tree** while the text cursor is in a node, including on the line of its key, selects that

node.

Search

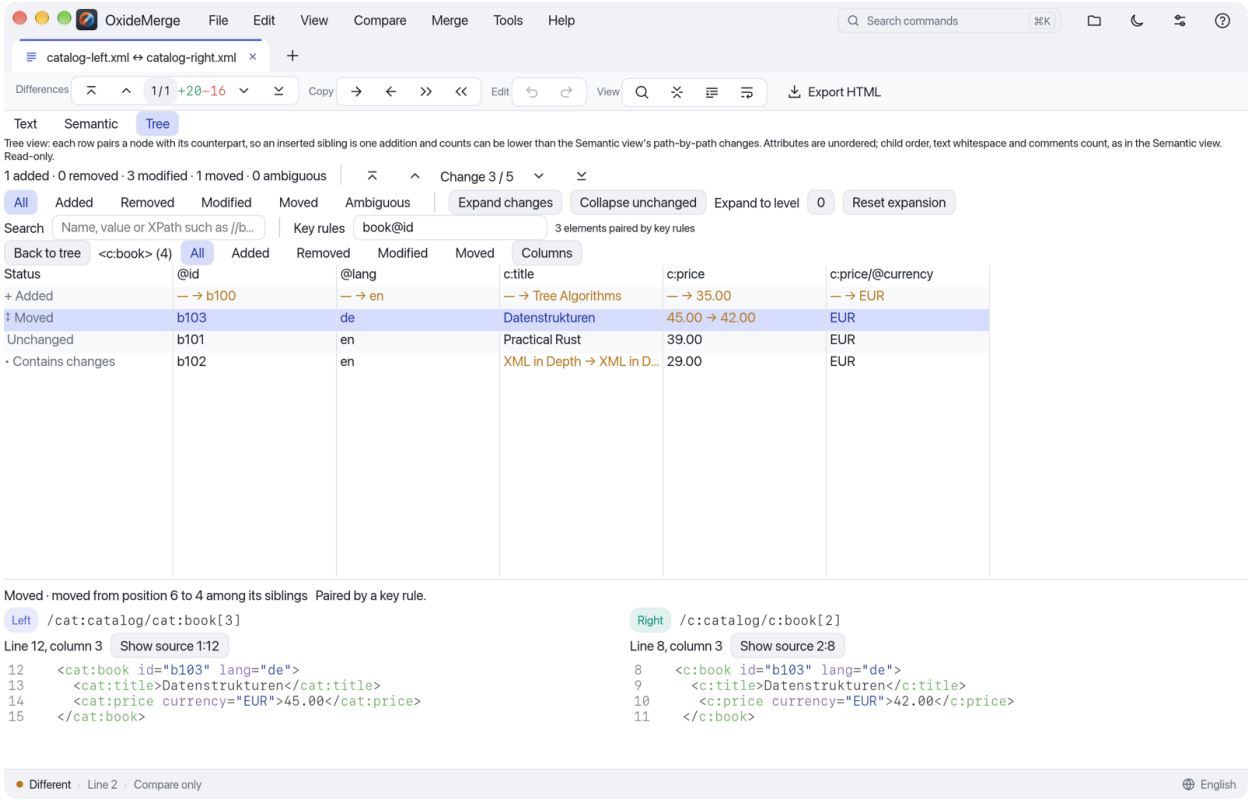
Type a key, name or value to show the matching rows with their parents. In JSON, YAML and TOML, text starting with `$` is a path, such as `$.server.port`, and shows that node and what is below it. In XML, text starting with `/` is a small XPath subset: `/` and `//` steps of element names or `*`, `@name`, `text()` and `comment()`, with `[@a]`, `[@a='value']` and `[n]` conditions, such as `//book[@id='b102']/title`. Other XPath is refused with a message.

Key rules

Repeated nodes are paired by position unless they can be told apart. In JSON, YAML and TOML, the objects of an array that all have an `id`, `name` or `key` member with a different value on each side are paired by it without a rule, and the toolbar says how many were, such as **3 objects paired by their id member**. Otherwise, write a rule as `name@key`. In JSON, YAML and TOML, `servers@name` pairs the objects of arrays named `servers` by their `name` member, `ports@name+protocol` by two members, and `spec/ports@name` only below `spec`; `*@id` covers every array, including a file that is a single array and an array directly inside another array, which has no name. In XML, `book@id` pairs `book` elements by their `id` attribute, `book@id+lang` by two attributes, and `catalog/book@id` only below `catalog`; a rule matches local names, but never pairs elements of different namespaces. Separate several rules with `;`; they apply once you stop typing. Entries that swapped places are then shown as a move plus the content that changed, not as two replacements. A rule is used for a group of siblings only when every node it covers on both sides has a complete and unique key. Otherwise a warning gives the number of duplicate and missing keys, and those nodes are paired as without the rule and marked ambiguous. Key rules belong to this view: they do not change the Semantic view or `--semantic` reports.

Collections

When the selected row holds an array of objects or repeated XML elements, **View as collection** lists them as a table: one row each, with the status and the value on each side in a column per member that holds a single value, or per attribute, child element that holds only text and attribute of such a child (and one for the text when they hold only text). Choose the columns and filter by status; click a row to select it in the tree. Entries that hold deeper objects, arrays, elements or mixed content are flagged, as the table does not show those.



Ambiguous rows

A row is marked ambiguous when another pairing fits as well: several siblings of the same name on one side with fewer on the other, a key rule that could not be used, an addition next to an identical sibling, or a swap where either node could be the one that moved. Check these yourself.

The Tree view is built in the background with progress and **Cancel**, within the same limits as the Semantic view (100,000 nodes and 64 levels). It is available for two files; three-way comparisons use the Semantic view.

12.2.5. Three files

In a three-way comparison, **Semantic** compares the left and right files with the middle one. Each path that differs is listed once, by its path in the middle file:

- **Only left differs** or **Only right differs**: one side changed it.
- **Both differ the same way**: both sides made the same change.
- **Conflict**: the sides changed it differently, or one side changed a value inside a value that the other side changed. Changes to different members of one object do not conflict. When both sides changed an array whose elements could be paired in more than one way, such as arrays of different lengths paired by position, the whole array is one conflict.

Use the filters to show only the conflicts. **Use left** and **Use right** copy that side's value into the middle pane, in the same way as **Copy value**. The middle pane is compared as it is now: after you take a side, that path no longer differs from the side you took.

12.3. Review workspace

The **Review workspace** tab of a folder comparison records your review:

- a state for each file and each difference (hunk);
- a **Review comment**, a **Rationale** and optional **Tags (comma-separated)**;
- a name in **Reviewer (free-form name)**. This is free text, not a verified identity.

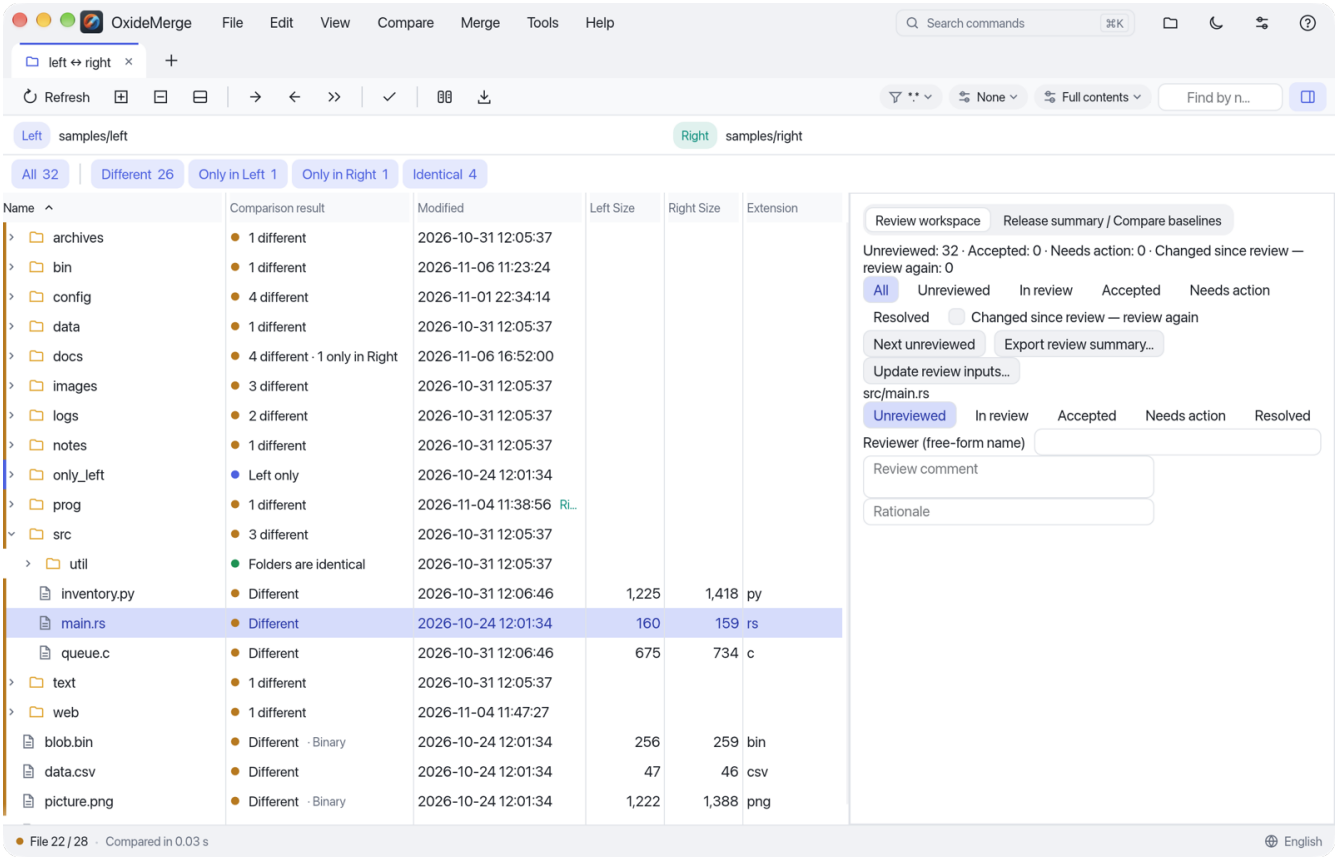


Figure 20. The review workspace

Next unreviewed goes to the next file that still needs a review. To review single differences, open the file from its folder comparison.

To keep your decisions and your position, save the folder review (see Folder reviews). OxideMerge saves only after the comparison, the file operations and the check of earlier decisions have finished. It refuses to save after a comparison that was stopped or that failed.

12.3.1. Updating the review

Update review inputs... chooses new folders to review against your earlier decisions. It keeps the selected compare rules, opens the folders read-only and marks the decisions that no longer match.

- A decision stays with a file that was renamed without changes, and notes the old name.
- If several new files have the same content, each one is marked for a new review.
- Changes to converted content, to converters or to rules can make earlier decisions invalid.
- A decision that is out of date counts as needing action. It never counts as accepted.

- Editing a reviewed file puts the file's decision out of date, and the decision on each difference whose lines, or the two lines around them, the edit changed. Decisions on other differences stay.
- If the context of a difference repeats or is not clear, you must review it by hand.

12.3.2. File versions and safety

- OxideMerge saves `.oxmreview` files in version 2. It opens version 1 files with every item unreviewed, and refuses unknown versions.
- When you open a bundle, OxideMerge checks the digests of the stored files.
- OxideMerge refuses a bundle with paths that lead outside it, with missing, unlisted or linked items, or that a newer version of OxideMerge saved.

12.4. Release summary / Compare baselines

The **Release summary / Compare baselines** panel compares two releases. It is on the start page, and in the side drawer of a folder comparison.

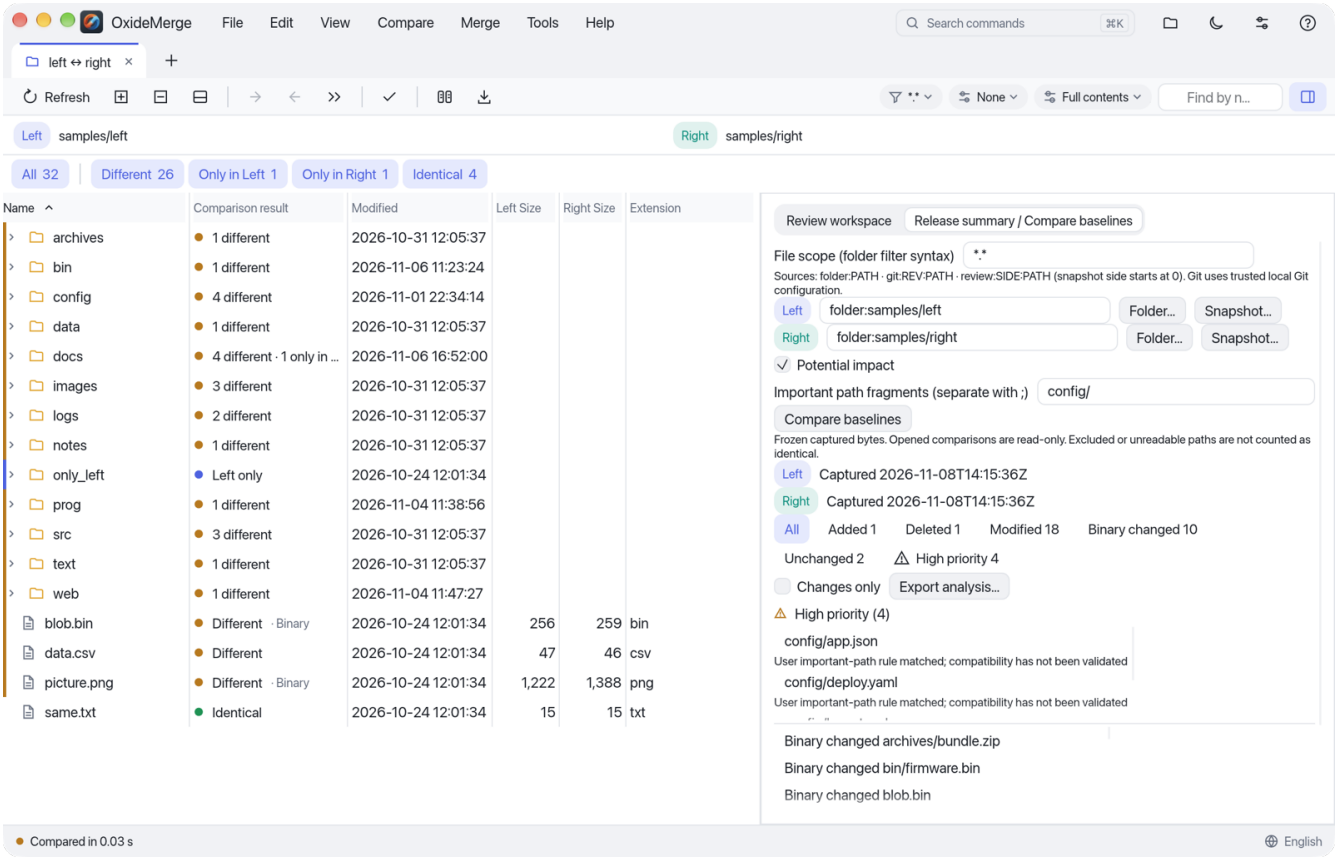


Figure 21. A release summary with potential impact turned on

Choose each side directly. You do not need to compare two folders first. Each side is one of these:

Source	Meaning
`folder:PATH`	The current content of a folder.

Source	Meaning
`git:REV:PATH`	A path in a Git revision. OxideMerge resolves the revision once and records its commit ID. Uncommitted changes are not included.
`review:SIDE:PATH`	One side of a saved folder review. The first side is `0`.

For Git sources, your Git configuration and filters must be trusted, as for Git revision comparisons.

12.4.1. The summary

The summary counts these kinds of changes:

- added, deleted, modified and unchanged files;
- moves of unique files with exactly the same content;
- suspected renames with edits;
- binary changes and changes of file type;
- semantic changes in JSON, XML, TOML and YAML files;
- excluded files.

It also shows when each side was captured.

- The counts above the list are filters. Click one or more categories to show only those changes.
- **High priority** shows the changes with a risk flag: an important path, or conflict markers that are still in a file. These changes are also listed first.
- Select a row to open the captured files read-only. The files come from the baseline, even if the sources changed after the capture.

12.4.2. Renames

OxideMerge suspects a rename with edits when all of these are true:

- A deleted file and an added file are text files of the same type.
- At least half of their lines, and at least three lines, are the same.
- They are clearly the best match for each other.

The row shows the old name and the share of matching lines.

- Click **Not a rename** to keep the two files as deleted and added. On the command line, use `--split-rename PATH`. OxideMerge warns if the path is not an added file.
- Click **Detect renames again** to undo this.

12.4.3. Important paths

An important path is text that you want to watch, such as `config/` or `api`. A changed path that contains this text is flagged as high priority. A file moved or renamed from such a path is flagged too.

Enter important paths in **Important path fragments (separate with ;)**. On the command line, use `--important-path TEXT`.

`--important-path` and `--split-rename` can be repeated. They need `--impact`, `--release-report` or `--baseline-left`.



Important paths only help you choose what to review first. They do not guarantee compatibility.

12.5. Potential impact

Turn on **Potential impact** to list the places where the changed files may be used. Each finding shows the file, the line, a digest of the content and an excerpt.

12.5.1. Exact findings

A finding is **Exact** when an import names exactly a file that was changed, deleted or renamed. OxideMerge recognizes these imports:

- JavaScript and TypeScript: relative imports, re-exports and `require`. An import of `./util.js` also finds a changed `util.ts`.
- Python imports.
- C and C++ `#include "…"`, and Objective-C `#import "…"`.
- CSS `@import`.
- Rust `mod name;`, and `use` paths that start with `crate::`, `super::` or `self::`.
- JSON `$ref` file references.

For JavaScript, TypeScript, Python, C, C and Rust, OxideMerge reads the imports from a syntax tree made by Tree-sitter. A `.h` header is read as C, or as C when it can only be read as C++. For the other files, it reads the tokens of the code. In Vue and Svelte components, only the `<script>` blocks are read. So an import can span several lines, and text in a comment or a string is not an import. If the parser cannot read part of a file, the file is listed as partly assessed, because some imports and definitions may be missed.

OxideMerge also compares the top-level functions, types and constants of changed JavaScript, TypeScript, Python, C, C and Rust files. C and C global variables are not included. Rust attributes, such as `#[derive(…)]`, are part of the definition that follows them.

A file that imports a changed or removed definition by name gets an **Exact** finding at the first place where it uses the name. The finding tells whether the definition changed or was removed. Imports under another name count too. Examples of such imports:

- `import { parse } from "./parser"`
- `from .parser import parse`
- `use crate::parser::parse`

12.5.2. Heuristic findings

Heuristic findings are possible uses that need a check:

- the first use of a changed definition name in other files of the same language;
- the name or path of a changed file in the text, as a separate word;
- changed JSON, TOML and YAML keys, also as dotted paths such as `db.timeout`.

If a large change gives more than 500 matches of names and keys, only the first 500 are listed. A note above the findings counts the others. The other kinds of findings are always complete.

12.5.3. What impact analysis does not find

- Imports that are conditional, generated or built by macros.
- Methods, and names used through a namespace, such as `ns.parse`.

OxideMerge never runs builds, tests or scripts of the repository.

12.5.4. Working with findings

- Click **Dismiss finding** to hide a finding. This does not change the files.
- In a folder comparison, **Review** opens the file of the finding in the review workspace.
- If there is no result, the panel says that the impact was not assessed.
- Changing a source clears the old findings.
- If you cancel the analysis, no partial result is shown.

12.6. Limits

- Each file can be at most 16 MiB.
- Each baseline can have at most 10,000 items and 256 MiB of captured content.
- Structured files can have at most 64 levels of nesting and 100,000 nodes.
- An impact analysis with more than 2,000 findings fails and asks you to make the scope smaller. Only the matches of names and keys are cut to fit.
- Files over these limits are refused or listed as excluded. They are never reported as identical.

Long analyses show their progress and can be canceled. Some steps, such as parsing one file or merging one file, check for cancellation only before and after they run, so a cancellation can wait until that step finishes. These limits do not change the normal comparisons.

12.7. Running analyses from the command line

```
oxidemerge --merge-plan left.txt base.txt right.txt --report merge-plan.json
oxidemerge --validate-merge merged.txt left.txt base.txt right.txt --report
validation.json
```

```
oxidemerge --semantic --format json old.json new.json --report semantic.html
oxidemerge --semantic --array-key id old.yaml new.yaml
oxidemerge --semantic left.yaml base.yaml right.yaml --report conflicts.json
oxidemerge --review-summary review.oxmreview --report review-summary.json
oxidemerge --baseline-left git:v1.0:/repo --baseline-right git:v1.1:/repo --release
-report release.html
oxidemerge --impact old-folder new-folder --report impact.json
```

Output:

- Without a report path, an analysis writes versioned JSON to the standard output.
- `.xml` reports, and `.html` reports of most analyses, contain this JSON. Their format is separate from the comparison reports.
- An `.html` release report is a readable summary, with the counts, the high-priority changes and a section for each change. It also contains the JSON.
- A report cannot replace an input file, and cannot be inside a selected input folder.

Exit codes:

Code	Meaning
`0`	Identical, or the plan or summary was created successfully.
`1`	Different, a warning, conflicts left in a plan, or a review summary with decisions that are out of date.
`2`	An error, or the baseline scope is not complete. Files that a filter excludes on purpose are outside the scope, so they are not errors.
`3`	`--validate-merge` only: the check failed.

Other notes:

- `--format` accepts `json`, `xml`, `toml` and `yaml`. The default comes from the extension of the first file.
- Use one analysis option for each run.
- `--semantic` only reports. To copy values, open the files and use the semantic view (see Semantic comparison).
- A three-file `--semantic` report lists each path with its `state` (`left`, `right`, `both` or `conflict`) and the changes from each side. It exits with `1` when any path differs, including conflicts.
- Source positions in semantic results are byte ranges of the decoded UTF-8 text and line numbers of the file. They are not offsets in a UTF-16 or legacy-encoded file.

Chapter 13. Command line

13.1. Opening comparisons

Run `oxidemerge` without arguments to open the start page. Give two or three paths to compare them. Give one path to open a conflict file, a project or a saved comparison. Put quotes around paths that contain spaces.

The examples use POSIX shell syntax. On Windows, use Git Bash, or change the examples for your shell.

```
oxidemerge left.txt right.txt                    # two-way text comparison
oxidemerge mine.txt base.txt theirs.txt          # three-way comparison (base in the
middle)
oxidemerge -r old-folder new-folder              # include subfolders
oxidemerge -r -f "*.rs;!target/" old new         # folder comparison with a filter
oxidemerge conflict.txt                          # resolve a file with conflict markers
oxidemerge -t image a.png b.png                  # choose the comparison type
oxidemerge https://example.com https://example.org   # web page comparison
oxidemerge left.txt right.txt -o merged.txt      # Save writes the right pane to
merged.txt
oxidemerge release.oxmproj                       # open a saved project
oxidemerge -r old.tar.gz new.tar.gz              # compare archives as folders
oxidemerge --git-rev HEAD~1 src/main.rs          # compare with an earlier Git revision
oxidemerge review.oxmcmp                         # open a saved comparison
```

13.2. Options

Table 1. Paths and sides

Option	Description
`PATHS…`	Two or three files, folders, web addresses or server paths (`sftp://`, `ftp://`, `ftpes://`, `ftps://`; see Server paths on the command line and in jobs); or one conflict file, project file or saved comparison.
`-t`, `--compare-as <T>`	The comparison type: `text`, `binary`, `image` or `web`.
`-o`, `--output <PATH>`	Where **Save** writes the result: the right pane in a two-way comparison, the middle pane in a three-way comparison.
`--dl`, `--dm`, `--dr <TEXT>`	Descriptions to show instead of the left, middle and right paths.
`-L`, `--label <TITLE>`	A title for each side in turn, as `diff -L` takes them (see Subversion, Mercurial and Perforce (experimental)); `--dl`, `--dm` and `--dr` take precedence.
`--wl`, `--wm`, `--wr`	Open the left, middle or right side read-only.

Option	Description
`--ssh-key <FILE>`	The private key to sign in to `sftp://` paths with (see Server paths on the command line and in jobs).
`--trust-fingerprint <FINGERPRINT>`	Trust a server path's server not trusted yet when its key or certificate has this fingerprint, and remember it (see Server paths on the command line and in jobs).
`--proxy <URL>`	Reach server paths through a SOCKS5 (`socks5://[user@]host:port`) or HTTP CONNECT (`http://[user@]host:port`) proxy, its password from `OXIDEMERGE_PROXY_PASSWORD` (see Proxies, active mode and client certificates).
`--ftp-active`	Use active mode for `ftp://`, `ftpes://` and `ftps://` paths (see Proxies, active mode and client certificates).
`--client-cert <FILE>`, `--client-key <FILE>`	The client certificate (PEM) to present to FTPS servers, and its private key when not in the same file (see Proxies, active mode and client certificates).
`--rev`, `--git-rev <REV>`	Compare the one path given with its version in a Git, Subversion or Mercurial revision (see Comparing with a revision).

Table 2. Folder comparisons

Option	Description
`-r`, `--recursive`	Include subfolders.
`-f`, `--filter <MASK>`	A filter mask, such as `.c;.h` or `!*.bak`, or `"[F] name"` for a named filter.
`-m`, `--method <M>`	The compare method: `full`, `quick`, `binary`, `date`, `datesize` or `size`.

Table 3. Comparison options

Option	Description
`--ignore-ws <MODE>`	Whitespace: `compare`, `change` or `all`.
`--ignore-case`, `--ignore-blank`, `--ignore-eol`, `--ignore-numbers`	Ignore case, blank lines, line-ending differences or numbers.
`--ignore-comments`	Ignore changes in comments.
`--prettify`	Format JSON and XML before comparing.
`--structured`	In folder comparisons, JSON, XML, TOML and YAML files with the same structure are identical.
`--numeric-tolerance <AMOUNT>`	Numbers that are at most this far apart are the same.
`--rules <NAME>`	The compare rules file: a name from the rules folder, or the path of an `.oxrules` file. `""` means no rules file.

Table 4. Reports and patches

Option	Description
`--cli`	Compare without a window. See Scripting and exit codes for the exit codes.
`--report <FILE>`	Write a report. Folder comparisons: `.html`, `.pdf`, `.csv`, `.txt` or `.xml`. Text comparisons: `.html`, `.pdf` or `.xml`. See PDF reports and printing.
`--report-diffs`	Add the differences of each changed text file to an HTML or PDF folder report.
`--slideshow`	Write the `.html` report as a slideshow, with one difference on each slide.
`--patch <FILE>`	Write a patch for two files or folders.
`--patch-style <S>`	`normal`, `context` or `unified` (the default).
`--context <N>`	The number of context lines in context and unified patches and in PDF reports. The default is 3.
`--page-size <SIZE>`	The pages of a PDF report: `a4` (the default) or `letter`.
`--portrait`	Portrait pages in a PDF report. The default is landscape.
`--unified`	One column in a two-way PDF report, with the removed lines above the added ones.
`--whole-file`	Show the whole files in a PDF report, not only the differences with their context.

Table 5. Analysis

Option	Description
`--merge-plan`	Write a read-only merge plan for three files or folders (left, base, right).
`--validate-merge <RESULT>`	Check a merge result against three files or folders (left, base, right).
`--semantic`	Compare two JSON, XML, TOML or YAML files by their structure. With three files (left, base, right), list what each side changed from the base, with the conflicts.
`--format <FORMAT>`	The semantic format: `json`, `xml`, `toml` or `yaml`. Needs `--semantic`.
`--array-key <MEMBER>`	Pair the objects of arrays by this member. Needs `--semantic`.
`--review-summary <SNAPSHOT>`	Summarize the decisions in a saved `.oxmreview` folder review.
`--impact`	List where the changed files of two folders or baselines may be used.

Option	Description
`--baseline-left <SPEC>`, `--baseline-right <SPEC>`	Release baselines: `folder:PATH`, `git:REV:PATH` or `review:SIDE:PATH`. Give both, instead of the paths.
`--release-report <FILE>`	Write a release baseline report as `.json`, `.html` or `.xml`.
`--important-path <TEXT>`	Flag changed paths that contain this text. Can be repeated. Needs `--impact`, `--release-report` or `--baseline-left`.
`--split-rename <PATH>`	Keep this added path as added, not as a suspected rename. Can be repeated. Needs `--impact`, `--release-report` or `--baseline-left`.
`--sync-plan <MODE>`	Write a read-only plan to synchronize two folders (source, destination). `MODE` is `mirror` or `update`. Writes JSON to the standard output, or to the `--report` file.
`--sync-links`	With `--sync-plan`: synchronize symbolic links as links, never following them (macOS and Linux). The plan records it, so `--sync-apply` needs no flag. Without it, links are left alone.
`--sync-apply <PLAN>`	Apply a plan to the folders it was made for, given in the same order. Needs `--yes`.
`--yes`	Confirm that `--sync-apply`, or the sync jobs of `--job`, may replace and delete files in the destination, and that the jobs of `--job` may write reports and patches outside the job file's folder.
`--sync-recover`	Undo the folder synchronizations that did not finish.
`--job <FILE>`	Run the jobs of a job file in order and print a JSON summary (see Jobs). Takes no paths.

Use one analysis option for each run. Analysis options cannot be combined with `-o`, `--patch`, `--git-rev`, `--slideshow` or `--report-diffs`.

Table 6. Other options

Option	Description
`--lang <CODE>`	The interface language, such as `ja` or `zh-TW`.
`--list-languages`	Print the available languages.
`--register-shell`, `--unregister-shell`	Add or remove the file manager entries (see File manager).
`--select-left <PATH>`, `--select-middle <PATH>`	Remember a side for a later `--compare-to`.
`--compare-to <PATH>`	Compare a path with the remembered left side (and middle side).
`-h`, `--help`	List the options. `-h` gives a short summary.
`-V`, `--version`	Print the version.

On Windows, WinMerge-style switches also work, such as `/r`, `/f`, `/o`, `/dl`, `/dr`, `/wl` and `/wr`. Some

WinMerge switches are ignored or not supported, so test your scripts before you rely on them.

13.3. Scripting and exit codes

`--cli`, `--report` and `--patch` run without a window. They return these exit codes:

Code	Meaning
`0`	The content is identical with the current options and rules.
`1`	Differences were found. This code is also returned when a report or patch was written successfully.
`2`	An error occurred.



Exit code `1` is a comparison result, not an error. Make sure that your scripts treat it that way.

Folder synchronization (see Synchronizing two folders) has its own codes:

Code	Meaning
`0`	`--sync-plan`: nothing to do. `--sync-apply`: every change was applied and checked. `--sync-recover`: everything was undone.
`1`	`--sync-plan`: the plan has changes or conflicts.
`2`	An error occurred, or the plan was refused and nothing was changed, for example because a file changed after the plan was made.
`4`	`--sync-apply` failed or was stopped, and every change was undone.
`5`	Partly applied: some changes could not be undone, also by `--sync-recover`. The journals named in the output keep the original files.

```
oxidemerge --sync-plan mirror source destination --report plan.json
oxidemerge --sync-apply plan.json --yes source destination
```

`--sync-apply` compares the folders again and refuses a plan that no longer matches them, one made with another filter or for other folders, or one written by a newer version.

`--sync-apply` and `--sync-recover` print a JSON result with a `status` (`applied`, `not_applied`, `rolled_back`, `recovery_required` or `recovered`). When the result is not a plain success it also has a `problem`: `stale` (a file or folder changed after the plan), `unsupported`, `read_failure`, `write_failure` (for example no permission or not enough free space), `canceled` or `recovery_needed`. These seven names are the same throughout OxideMerge: a view that leads with the kind of problem before its detail uses them too, so the same situation is called the same thing wherever it appears (`ambiguous` is the seventh: another reading of the input is as likely, as with repeated items that cannot be told apart).

- The command line uses your saved options, unless you change them with options. For runs that must not depend on your settings, set `OXIDEMERGE_CONFIG_DIR` to a separate folder (see Configuration files).

- Reports are available for text and folder comparisons. For a binary or image comparison, OxideMerge prints a message instead of an HTML report.

```
# Compare without a window. The differences are printed too.
oxidemerge --cli -r dirA dirB

# HTML report of a folder comparison, with the differences of each changed file
oxidemerge -r dirA dirB --report report.html --report-diffs

# HTML report of two files, ignoring changes in comments
oxidemerge old.c new.c --ignore-comments --report diff.html

# Unified diff with 5 lines of context
oxidemerge old.c new.c --patch changes.patch --context 5

# Use a compare rules file
oxidemerge --cli -r build1 build2 --rules "Build logs"
```

In a shell that uses `set -e`, save the exit code explicitly, so that differences do not stop the script:

```
result=0
oxidemerge --cli -r old-folder new-folder || result=$?
case "$result" in
  0) echo "No differences" ;;
  1) echo "Differences found" ;;
  *) echo "Comparison failed" >&2; exit "$result" ;;
esac
```

13.4. Jobs

A job file keeps a set of runs in one place, so a nightly report or backup is one command. It is a JSON file, usually with the `.oxjob` extension:

```
{
  "version": 1,
  "continue_on_error": false,
  "jobs": [
    {
      "name": "nightly report",
      "left": "a",
      "right": "b",
      "recursive": true,
      "report": "report.html"
    },
    {
      "name": "plan",
      "kind": "sync_plan",
```

```
      "source": "a",
      "destination": "${env:BACKUP}",
      "mode": "mirror",
      "report": "plan.json"
    },
    {
      "name": "backup",
      "kind": "sync",
      "source": "a",
      "destination": "${env:BACKUP}",
      "mode": "mirror",
      "apply": true
    }
  ]
}
```

```
oxidemerge --job nightly.oxjob          # runs the first two jobs, refuses the sync
oxidemerge --job nightly.oxjob --yes    # runs all three
```

Each job is turned into the options of a command line and run as that command line would be, so a job and its command line give the same result. Your saved options apply, as they do on the command line. `--job` takes no paths and no other options except `--yes` and `--lang`; the options of each job are in the file.

`version`

Required; `1`. A file of a newer version is refused.

`continue_on_error`

When `false` (the default), the jobs after one that ends with an error (code 2 or higher) are not run. When `true`, every job runs.

`jobs`

The jobs, in order. A field OxideMerge does not know is an error, so a misspelled name is never ignored.

Every job has an optional `name`, shown in the summary and in error messages, and a `kind`:

`compare` **(the default)**

`left` and `right` (and `base` for a three-way compare) are required. Options: `recursive`, `report`, `report_diffs`, `slideshow`, `patch`, `patch_style`, `context`, `compare_as`.

`sync_plan`

Writes a sync plan. `source`, `destination`, `mode` (`mirror` or `update`) and `report` (the plan file) are required.

`sync`

Plans and applies a synchronization at once. `source`, `destination`, `mode` and `"apply": true` are required; the run also needs `--yes`. A sync that would delete many items is refused unless the job

has `"allow_large_deletion": true`. Both kinds take `"copy_links": true` for `--sync-links`.

All kinds also take `filter`, `method`, `rules`, `whitespace`, `ignore_case`, `ignore_blank`, `ignore_eol`, `ignore_numbers`, `ignore_comments`, `prettify`, `structured` and `numeric_tolerance`, and the compare kind `page_size`, `portrait`, `unified` and `whole_file` for a `.pdf` report, with the meanings of the command-line options of the same names. A field that does not fit the kind, such as `apply` in a compare, is an error. For server paths, all kinds take `ssh_key`, `trust_fingerprint`, `proxy`, `ftp_active`, `client_cert` and `client_key`, as `--ssh-key`, `--trust-fingerprint`, `--proxy`, `--ftp-active`, `--client-cert` and `--client-key` (see Server paths on the command line and in jobs); a `sync` job whose `destination` is a server path writes the changes back to the server.

Paths, including `rules`, are relative to the folder of the job file, unless they are absolute, URLs or server paths. In paths and `rules`, `${env:NAME}` is replaced by the environment variable `NAME`; an unset variable is an error. Nothing else is expanded, and a job file names no command and no shell. Comparing a file that one of your unpackers reads (see Unpacker plugins) runs that unpacker, as any comparison does.

A job writes its report or patch as a new file or over a plain file, never through a symbolic link. A report or patch outside the job file's folder, such as `../report.html` or a path under `${env:HOME}`, is written only with `--yes`, so a job file received from someone else cannot replace your files.

`--job` prints a JSON summary. Its `exit_code` is the worst of the jobs' codes, which is also the exit code of the run. Each entry of `jobs` has the `name`, the `kind`, the `exit_code`, an `error` when the job failed, and for a sync job the same `sync` result that `--sync-apply` prints. `not_run` lists the jobs skipped after a failure, and `stopped` is `true` when the run was stopped in the program:

```
{
  "exit_code": 2,
  "jobs": [
    { "name": "nightly report", "kind": "compare", "exit_code": 1 },
    { "name": "plan", "kind": "sync_plan", "exit_code": 1 },
    { "name": "backup", "kind": "sync", "exit_code": 2,
      "error": "This job changes … and runs only when allowed: --yes on the command
line" }
  ]
}
```

13.4.1. Saving and running jobs in the program

File → Save as job… saves the active folder comparison as a job file. The job names the compared folders, the filter, the compare method, the compare rules file and the comparison options that are on, so it compares as the tab does. Choose what the job does:

- **Compare and write a report**: the report file is relative to the job file, and its extension (`.html`, `.pdf`, `.csv`, `.txt` or `.xml`) sets the format.
- **Plan a synchronization**: writes a JSON plan and changes nothing.
- **Synchronize**: plans and applies at once, left to right or right to left, as a mirror or an update. The job has `"apply": true`; **Allow deleting many files** adds `"allow_large_deletion": true`.

Select **Store paths relative to the job file** to move the job and its folders together. Only a comparison of two folders can be saved as a sync job. A comparison of a revision, a web page or an archive goes through temporary copies, so it cannot be saved as a job.

File → Run job… runs a job file and lists how each job ended, with **Open** for the report or plan it wrote and **Compare** to open its folders in a tab. **Open** is offered only for report files (`.html`, `.pdf`, `.csv`, `.txt`, `.xml`, `.json`, `.diff` or `.patch`). The jobs run as `--job` runs them, from your saved options, not the options OxideMerge was started with.

A file that holds a sync job, or writes a report or patch outside its folder, first lists the folders it changes and the files it writes. A sync job allowed to delete many files says so. The file runs only after you click **Run and allow changes**; that stands for `--yes`, for this run only. What runs is the file as it was listed, even if it changes on disk meanwhile; **Run again** reads it again and asks again. **Stop** stops a synchronization being applied, undoing what it changed, and skips the jobs after it. While a job is synchronizing, OxideMerge waits for it before it quits.

Chapter 14. Integrating with other tools

This chapter sets OxideMerge up as the diff and merge tool that Git, Subversion, Mercurial or Perforce run on their own. To instead open a revision from within OxideMerge, see Comparing with a revision.

14.1. Git

Install Git and make sure that `oxidemerge` is in your `PATH`. Then run these commands in a POSIX shell, such as Bash or Git Bash. They make OxideMerge the diff tool and the merge tool for your user account:

```
git config --global diff.tool oxidemerge
git config --global difftool.oxidemerge.cmd 'oxidemerge "$LOCAL" "$REMOTE"'
git config --global merge.tool oxidemerge
git config --global mergetool.oxidemerge.cmd \
  'oxidemerge "$LOCAL" "$BASE" "$REMOTE" -o "$MERGED" --dl Local --dr Remote --wl
--wr'
git config --global mergetool.oxidemerge.trustExitCode false
```

On macOS, use the full path `/Applications/OxideMerge.app/Contents/MacOS/oxidemerge`, unless you linked the program into your `PATH`. On Windows, use the path of `oxidemerge.exe`.

`git difftool`

Opens each changed file in turn. `git difftool --dir-diff` opens one folder comparison with all changed files. In a file, `Alt` + `↓` after the last difference opens the next changed file.

`git mergetool`

Opens a three-way merge with the base in the middle. Use **Auto merge**, or copy differences to the middle, then save. The middle pane is written to the merged file, never to the base. Files OxideMerge would otherwise read as text, such as Jupyter notebooks, are merged as the files they are here, so what is saved is still a valid file.

With `trustExitCode false`, Git checks whether the output file changed. If it looks unchanged, Git can ask you whether the merge was successful. Answer yes only after you saved and checked the result. Closing the window alone does not mean that the merge was successful.

14.2. Subversion, Mercurial and Perforce (experimental)



Support for Subversion, Mercurial and Perforce is experimental and not fully tested. The settings below were tried on macOS only, with local repositories and a local Perforce server. Check the result of every merge before you accept it, and report what does not work.

OxideMerge accepts the arguments that `diff` takes for titles: `-u` is ignored, and each `-L <TITLE>` names the next side, in the order of the paths. So the pane headers show revision names rather than

the paths of temporary files. `--dl`, `--dm` and `--dr` take precedence over `-L`.

Subversion and Perforce pass the files of a merge in an order that OxideMerge cannot take directly, so a merge goes through a small wrapper script. The wrapper fails when the merged file was not saved, so that the conflict stays unresolved when you close the window without saving.

14.2.1. Subversion

`svn diff --diff-cmd oxidemerge` compares each changed file in turn. Subversion passes `-u -L <old title> -L <new title> <old> <new>`, so the headers show, for example, `f.txt (revision 12)` and `f.txt (working copy)`. To use OxideMerge for every `svn diff`, set it in `~/.subversion/config` (`%APPDATA%\Subversion\config` on Windows):

```
[helpers]
diff-cmd = oxidemerge
merge-tool-cmd = /path/to/oxidemerge-svn-merge
```

Do not pass `-x` options to `svn diff`: Subversion then passes them instead of `-u`.

For merges, Subversion passes five paths: base, theirs, mine, merged file and working file. Save this wrapper as `oxidemerge-svn-merge`, make it executable and name it in `merge-tool-cmd`:

```
#!/bin/sh
# Subversion passes: base, theirs, mine, merged, working file.
before=$(cksum < "$4")
oxidemerge "$3" "$1" "$2" -o "$4" --dl Mine --dm Base --dr Theirs --wl --wr
[ "$(cksum < "$4")" != "$before" ]
```

Choose **l** (launch) at Subversion's conflict prompt, or run `svn update --accept launch`. Subversion marks the conflict resolved as soon as the wrapper succeeds, that is when you saved.

14.2.2. Mercurial

Add this to `~/.hgrc` (`%USERPROFILE%\mercurial.ini` on Windows):

```
[extensions]
extdiff =

[extdiff]
cmd.oxidemerge = oxidemerge

[diff-tools]
oxidemerge.diffargs = -L $plabel1 -L $clabel $parent $child

[ui]
merge = oxidemerge
```

```
[merge-tools]
oxidemerge.args = $local $base $other -o $output --dl $labellocal --dm $labelbase --dr
$labelother --wl --wr
oxidemerge.checkchanged = true
```

`hg oxidemerge` compares the changed files: one file directly, several as a folder comparison. `hg oxidemerge -c REV` shows one changeset. Do not also set `opts.oxidemerge`: Mercurial then ignores `diffargs`, and the titles are lost. `hg merge` and `hg resolve` open a three-way merge with the base in the middle. With `checkchanged`, Mercurial asks whether the merge was successful when the merged file did not change.

14.2.3. Perforce

Perforce can use OxideMerge from the p4 command line, from P4V, or from both. The two are set up separately: p4 reads `P4DIFF` and `P4MERGE`, while P4V keeps its own preferences.

Both need a working connection first. Run `p4 info` in your workspace: it should show your **User name**, your workspace as **Client name** and your server as **Server address**. If it reports **Client unknown** or cannot connect, set `P4PORT`, `P4USER` and `P4CLIENT` first, or the `P4CONFIG` file your team uses.

The p4 command line

p4 runs the program named by `P4DIFF` with two files: the depot revision (in a temporary file) and your workspace file. It runs the program named by `P4MERGE` with four: base, theirs, yours and the result. OxideMerge takes the merge files in a different order, so the merge goes through a wrapper.

To set it up:

1. Save this script as `oxidemerge-p4-merge` in a folder on your `PATH`, for example `~/bin/oxidemerge-p4-merge`:

   ```
   #!/bin/sh
   # Perforce passes: base, theirs, yours, result.
   before=$(cksum < "$4")
   oxidemerge "$3" "$1" "$2" -o "$4" --dl Yours --dm Base --dr Theirs --wl --wr
   [ "$(cksum < "$4")" != "$before" ]
   ```

   The last line makes the wrapper fail when the result was not saved, so the file stays unresolved.

2. Make it executable:

   ```
   chmod +x ~/bin/oxidemerge-p4-merge
   ```

3. Tell p4 to use OxideMerge, with `p4 set` (or as environment variables of the same names):

   ```
   p4 set P4DIFF=oxidemerge
   ```

```
p4 set P4MERGE=$HOME/bin/oxidemerge-p4-merge
```

If `oxidemerge` is not on your `PATH`, give its full path, for example `/Applications/OxideMerge.app/Contents/MacOS/oxidemerge` on macOS. Do not add `--cli` to `P4DIFF`: it exits with 1 when the files differ, which p4 reports as **Execution Failed**.

`p4 set` stores the values in `~/.p4enviro` on macOS and Linux, and in the registry on Windows.

4. Check the settings with `p4 set`; both lines should name OxideMerge:

```
P4DIFF=oxidemerge (enviro)
P4MERGE=/Users/you/bin/oxidemerge-p4-merge (enviro)
```

The text in parentheses tells where p4 found each value: `(enviro)` for `p4 set` on macOS and Linux, `(set)` for `p4 set` on Windows, `(config '…')` for a `P4CONFIG` file. A value with nothing after it comes from an environment variable, which takes precedence over `p4 set`; if a line names another tool, change it where it is set.

To compare a file with the depot, run `p4 diff f.txt`; OxideMerge opens with the depot revision on the left and your workspace file on the right. The depot revision is a temporary file with a name such as `tmp.63484.0x20d2d0f40.90`, and only your workspace file can be edited. p4 passes the words in `P4DIFF` on to OxideMerge, so you can name the panes instead:

```
p4 set P4DIFF="oxidemerge --dl Depot --dr Workspace"
```

p4 splits `P4DIFF` at spaces, so keep each title to one word.

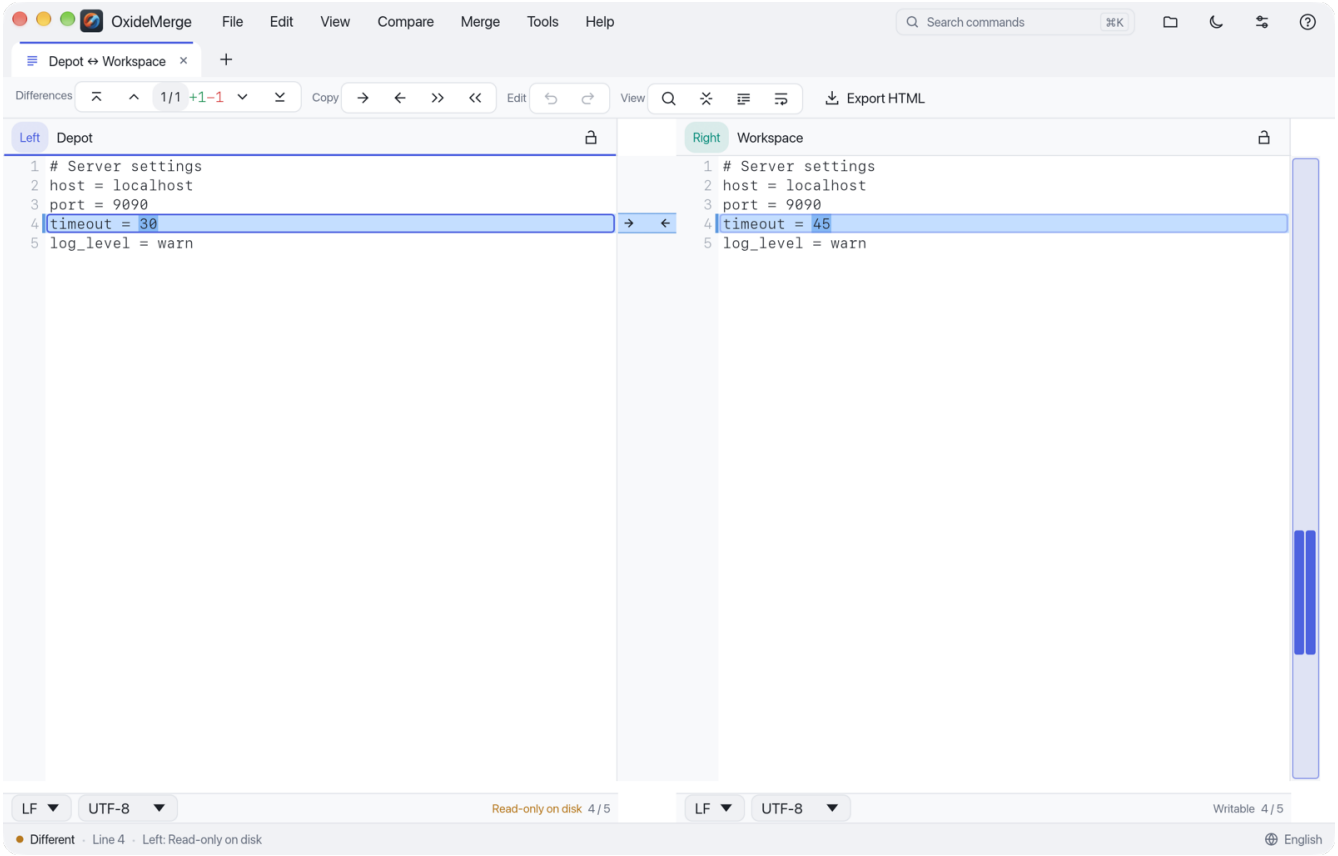


Figure 22. `p4 diff` with the panes named Depot and Workspace

To merge a conflict:

1. Run `p4 resolve f.txt`. p4 shows what it found, and suggests an action:

   ```
   /home/you/ws/f.txt - merging //depot/main/f.txt#2
   Diff chunks: 0 yours + 1 theirs + 0 both + 1 conflicting
   Accept(a) Edit(e) Diff(d) Merge (m) Skip(s) Help(?) e:
   ```

2. At the prompt, type **m** (merge). OxideMerge opens with **Yours** on the left, **Base** in the middle and **Theirs** on the right. The middle pane starts as the base, and **Save result** writes it to the result file p4 passed, never to the base.
3. Resolve each conflict, with **Next conflict** and then **Take left**, **Take right** or **Take both**, or by editing the middle pane. Check that the toolbar says **No conflicts left**: **Save result** also saves a result that still has conflicts, with the base text in their place.
4. Click **Save result** and close the window.
5. Back at the prompt, p4 suggests **ae** (accept edit), or **at** (accept theirs) when the result is the same as theirs. Type **ae** to accept the merged file as you saved it. p4 then reports, for example, `//ws/f.txt - edit from //depot/main/f.txt`.

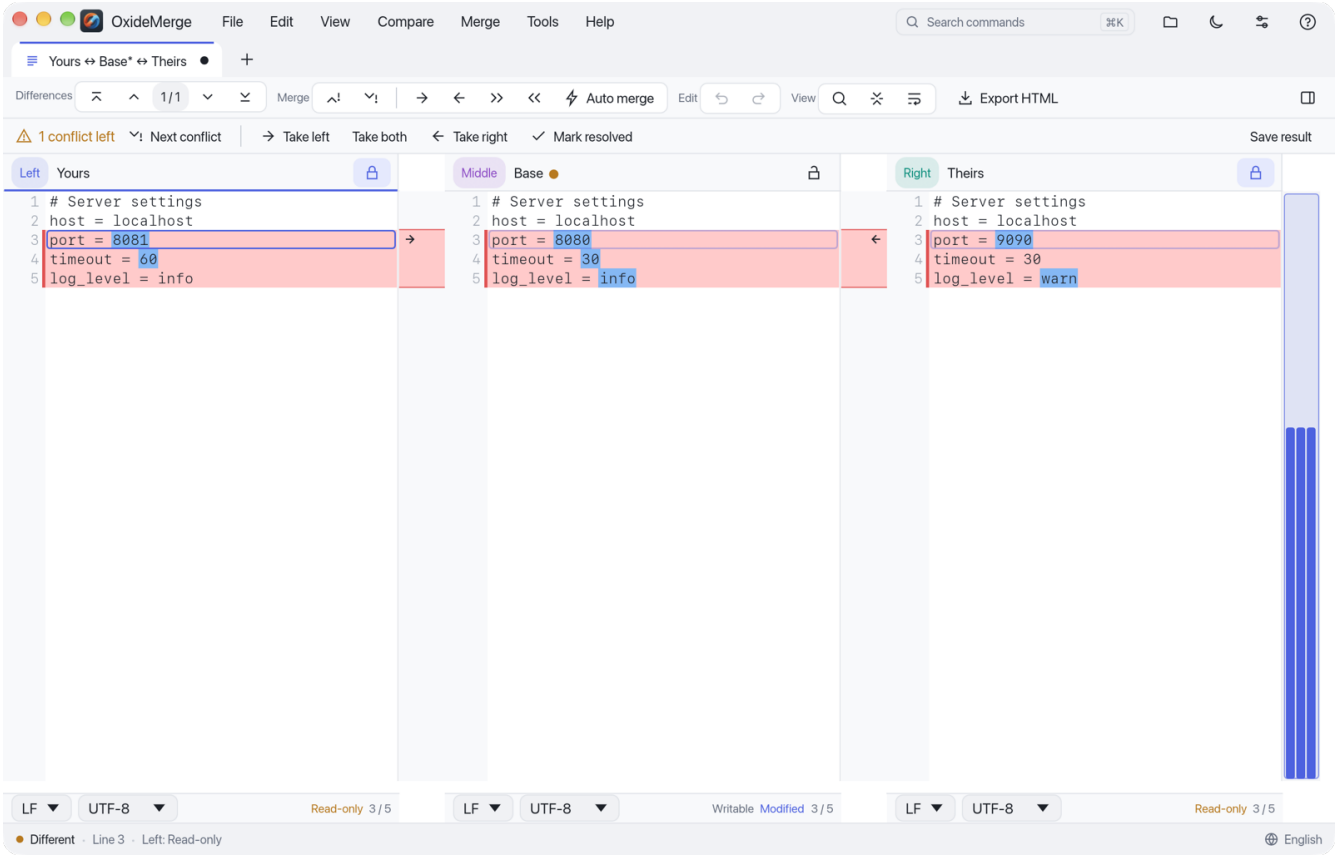


Figure 23. A conflict opened by `p4 resolve`

If you close the window without saving, p4 reports **Execution Failed** and the file stays unresolved; typing **ae** then warns that the file still has change markers. Run `p4 resolve` again to start over.

P4V

P4V does not read `P4DIFF` or `P4MERGE`; it has its own diff and merge settings. It runs OxideMerge directly, so no wrapper is needed.

To set it up:

1. In P4V, open **Preferences** (**P4V** → **Preferences** on macOS, **Edit** → **Preferences** on Windows and Linux).
2. Select **Diff** in the list on the left.
3. Select **Other application**. Filling in the fields below it is not enough: while **P4Merge** stays selected, P4V keeps using P4Merge.
4. In **Location**, give the path of the `oxidemerge` program itself, not of the app or a wrapper:

macOS	`/Applications/OxideMerge.app/Contents/MacOS/oxidemerge`
Windows	the path of `oxidemerge.exe` in the folder you extracted it to
Linux	the output of `command -v oxidemerge`, for example `/usr/bin/oxidemerge`

5. In **Arguments**, type `%1 %2`.

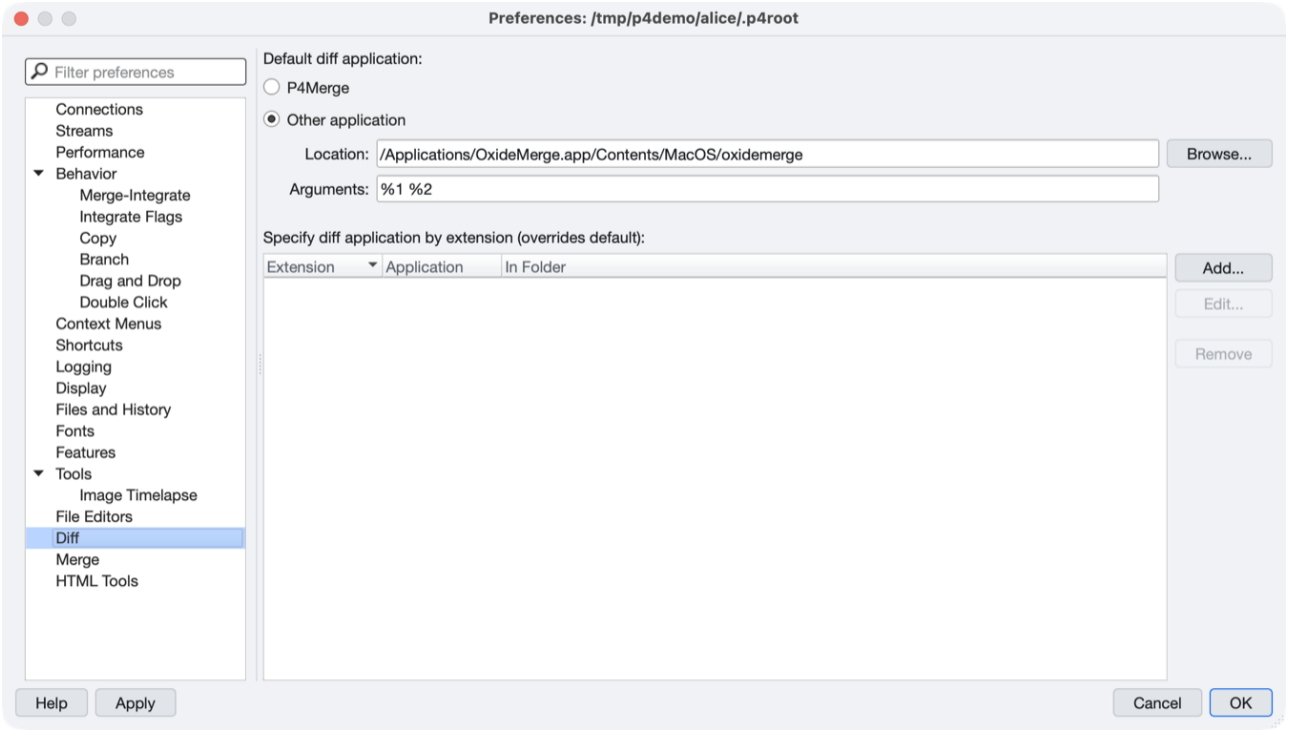


Figure 24. P4V Preferences → Diff

6. Select **Merge** in the list on the left, select **Other application** there too, and give the same **Location**.
7. In **Arguments**, type:

```
%2 %b %1 -o %r --dl Yours --dm Base --dr Theirs --wl --wr
```

In P4V's merge arguments, `%1` is theirs, `%2` is yours, `%b` the base and `%r` the result.

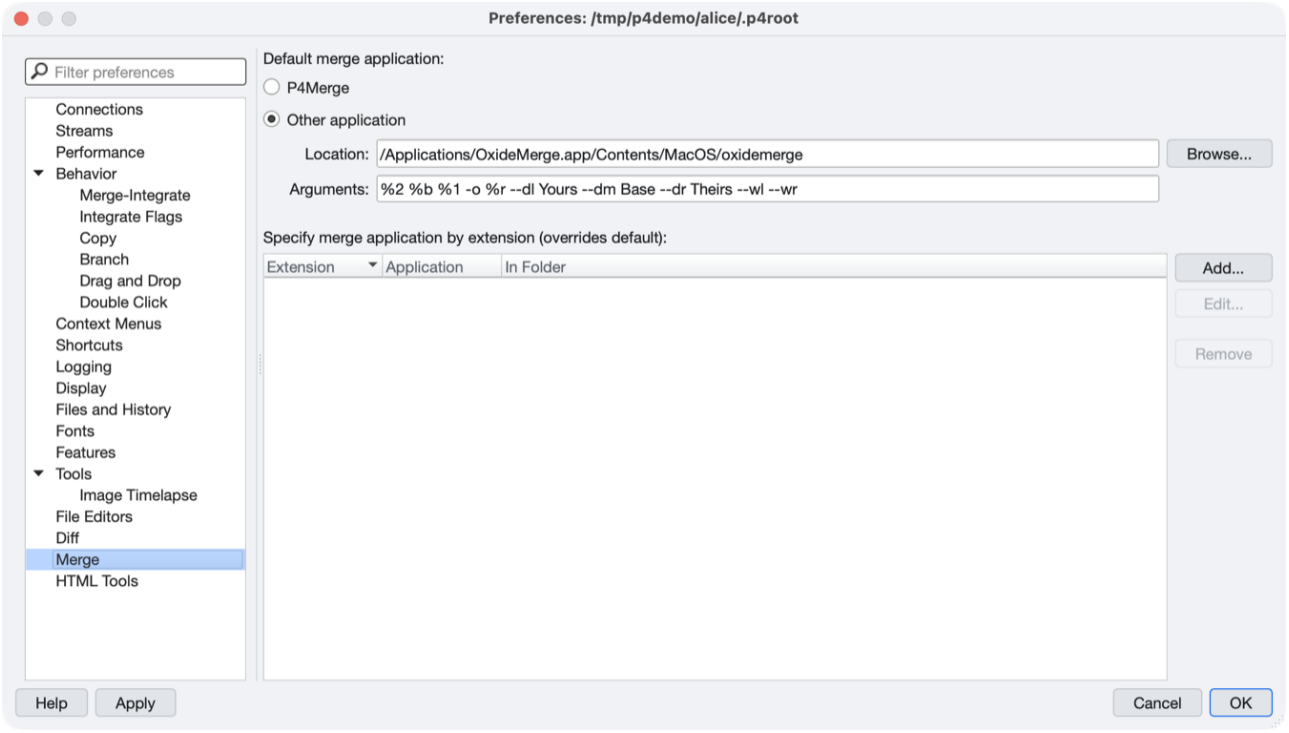


Figure 25. P4V Preferences → Merge

8. Click **OK**.

The list under **Specify diff application by extension** and **Specify merge application by extension** overrides the default for the extensions in it. Leave it empty to use OxideMerge for every file; if one kind of file still opens P4Merge, look for its extension there.

To compare a file with the depot:

1. Select the file in the **Workspace** or **Depot** tab and click **Diff** in the toolbar.
2. In the Diff Files dialog, the first path is the latest revision and the second is your workspace file; click **Diff**. OxideMerge opens with the depot revision on the left, in a temporary file with a name such as `server#head.conf`, and your workspace file on the right.

To merge a conflict:

1. Select the file in the **Workspace** or **Pending** tab; a file that needs resolving shows a question mark.
2. Choose **Actions** → **Resolving** → **Resolve…**, or right-click the file and choose **Resolve…**.
3. In the Resolve dialog, click **Run Merge Tool**. OxideMerge opens with **Yours** on the left, **Base** in the middle and **Theirs** on the right. If P4Merge opens instead, go back to **Preferences** → **Merge** and check that **Other application** is selected.

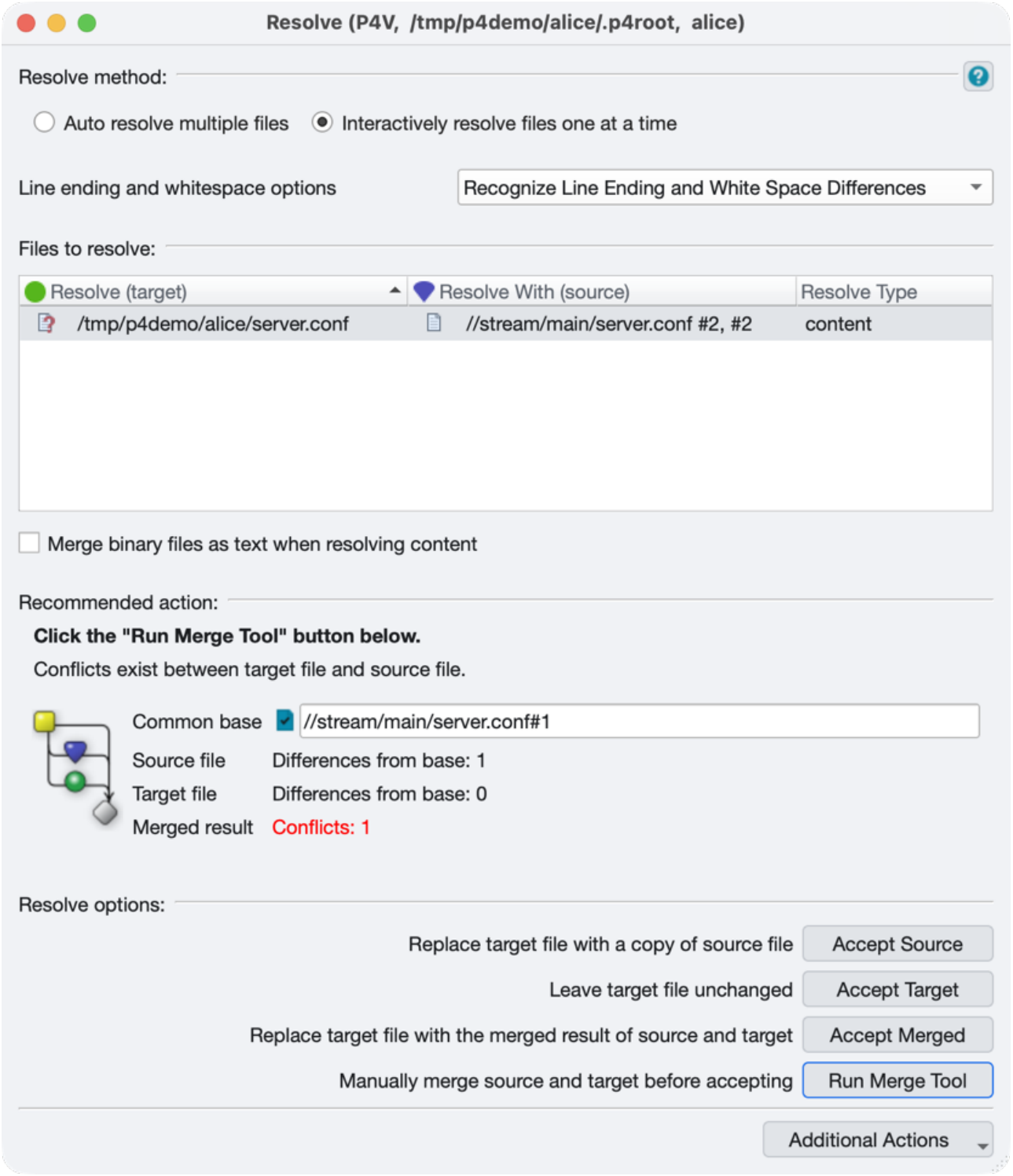


Figure 26. The P4V Resolve dialog

4. Resolve each conflict, click **Save result** and close the window. The result file P4V passes already holds P4V's own conflict markers; saving replaces them. **Yours** is your workspace file itself rather than a copy, so make your changes in the middle pane: what you save in the left pane is written to your workspace file at once.
5. In the Resolve dialog, click **Accept Merged**, and **Yes** when P4V asks whether to replace the file with the merged result. The dialog still shows the conflict count from before the merge; P4V does not read the result again until you accept it. Check the result first: OxideMerge writes the result only when you save, so if you closed it without saving, the result still holds P4V's markers. Use **Accept Source** or **Accept Target** instead to take theirs or yours whole.

On Windows, write a wrapper as a `.cmd` file with the line `oxidemerge.exe "%3" "%1" "%2" -o "%4" --dl Yours --dm Base --dr Theirs --wl --wr` (**Mine** rather than **Yours** for Subversion). It cannot tell whether you saved, so check the result before you accept it.



What was tried: the Subversion commands and wrapper with Subversion 1.15, and the Mercurial settings with Mercurial 7.2, on macOS, with local repositories only. Perforce: `P4DIFF`, with and without pane titles, and the `oxidemerge-p4-merge` wrapper with p4 2026.1, and P4V 2026.3's diff and merge settings, from **Diff** to **Accept Merged**, against a local personal server. Not tried: Windows and Linux, the Windows `.cmd` wrappers, Subversion servers reached over the network (`svn://`, `https://`) and their sign-in, and Perforce servers reached over the network.

14.3. File manager

The **Tools** menu, or `oxidemerge --register-shell`, adds OxideMerge to your file manager:

Windows

Tools → **Add to Explorer context menu** adds an **OxideMerge** submenu for files, folders, drives and folder backgrounds. On Windows 11, it is under **Show more options**.

macOS

Tools → **Add Finder Quick Actions** adds **OxideMerge: Compare**, **OxideMerge: Select Left** and **OxideMerge: Select Middle** to the Quick Actions and Services menus of Finder.

Linux

Tools → **Add to file manager context menu** adds the same entries to Nautilus, Nemo and Dolphin.

The entries work like this:

- **Compare** compares the two or three selected files or folders. With one item selected, it compares the item with the side that you selected before. If you did not select a side, it opens the **Select files or folders** dialog with the item.
- **Select Left** and **Select Middle** remember an item, for example from another folder. **Compare** then reads **Compare to "name"**.
- **Compare as** (Windows, files only) compares as text, binary, image or web page.

The entries point to the program that added them. If you move the program, add the entries again. To remove them, use the matching **Tools** command, or run `oxidemerge --unregister-shell`.

Chapter 15. Options and configuration

15.1. Application options

To open the options, choose **Edit** → **Options...**, or click the settings button (two sliders) at the top right. **Search settings** finds an option by name. **Reset to defaults** restores the default values.

General

- **Language**, **Theme** (**Light**, **Dark** or **Match system**), **Font size** and **Tab size**.
- **Line numbers**, **Location pane**, **View whitespace** and **Syntax highlighting**.
- **Text layout** (see the view options in Comparing text files).
- **Keep a .bak copy when saving over a file**, **Use the recycle bin when deleting** and **Confirm copy operations**.
- **Reopen previous comparisons at startup**.
- **Accessibility**:
  - **Color-blind friendly diff colors**: blue for added, orange for removed, purple for changed lines and yellow for conflicts.
  - **High contrast**: stronger colors, with the text on every line color at a contrast of at least 4.5:1, as WCAG asks for text. In every look the text cursor has at least 3:1 on every line color.
  - **Reduce motion**: no animations.
  - **Compact density**: lower rows and less space.

    You can also switch these four options in the command palette.

Compare

The comparison options (see Comparison options).

Compare rules

The compare rules files (see Compare rules).

Folder

Compare method, **Include subfolders**, **Include contents of unique subfolders**, **Skip hidden files and folders**, **Treat JSON, XML, TOML and YAML files with the same structure as identical**, **Date tolerance (seconds)** and **Worker threads (0 = automatic)**.

The structure option works with the **Full contents** method. A file whose text differs is read as structured data, and it counts as identical when the values are the same. Key order, layout and comments do not count, as in Semantic comparison, and TOML and YAML numbers compare by value: `30` and `30.0` are the same, even where a program reading the file tells them apart. Opening such a file still shows its text differences; choose **Semantic** to see that the structure is the same. Folder synchronization still copies a file whose contents differ.

Image / Binary

Color distance threshold, **Block size** and **Bytes per line (binary)**.

Plugins

The unpacker plugins (see Unpacker plugins).

15.2. Configuration files

OxideMerge stores its options as JSON in your configuration folder:

System	Settings file
Windows	`%APPDATA%\OxideMerge\config.json`
macOS	`~/Library/Application Support/OxideMerge/config.json`
Linux	`~/.config/OxideMerge/config.json`

Help & Shortcuts shows where your settings file is. If OxideMerge cannot read the settings file, it saves it as `config.json.bad` and uses the default settings. You can then correct the file and put it back.

The configuration folder also contains these folders:

- `Rules`: compare rules files.
- `Filters`: WinMerge `.flt` filter files.
- `Languages`: added or changed language packs.

15.3. Portable and separate configurations

Portable use

Create an empty file named `oxidemerge.json` next to the program. OxideMerge then stores its options in this file, and the `Rules` folder is next to the program. The program folder must be writable.

Separate configuration

Set the environment variable `OXIDEMERGE_CONFIG_DIR` to a folder. OxideMerge then uses the `config.json` in that folder. This setting has priority over portable use and over the default configuration folder. Use it for scripts, for tests, or to keep separate work environments.

15.4. Adding a language

1. Copy `lang/en.json` from the source code to a file named `<code>.json`, such as `nl.json`.
2. Translate the values. Keep each `{placeholder}` exactly as it is.
3. Put the file in a `Languages` folder next to the program, or in the configuration folder.

Messages that are missing, and translations with missing or extra placeholders, are shown in English.

Chapter 16. Keyboard shortcuts

Help → **Help & Shortcuts** (`F1`) lists all shortcuts and mouse actions in the program. On macOS, use `⌘` for `Ctrl` and `⌥` for `Alt`.

Table 7. General

Keys	Action
`F1`, `?`	Show or hide Help & Shortcuts
`Ctrl` + `K`, `Ctrl` + `Shift` + `P`	Command palette
`Ctrl` + `N`	New text compare
`Ctrl` + `O`	Open
`Ctrl` + `S`	Save
`Ctrl` + `Shift` + `S`	Save all
`Ctrl` + `Shift` + `E`	HTML report
`Ctrl` + `P`	Print: open the PDF report in the system's viewer
`F5`	Refresh
`Ctrl` + `W`, `Ctrl` + `F4`, `Esc`	Close the tab. `Esc` first closes an open dialog, search bar or editor
`Ctrl` + `Tab`, `Ctrl` + `Shift` + `Tab`	Next or previous tab
`Alt` + `F4` (`Cmd` + `Q` on macOS)	Exit. OxideMerge asks about unsaved changes first
`Ctrl` + `+`, `Ctrl` + `-`, `Ctrl` + `0`	Zoom in, zoom out, reset the zoom

Table 8. Differences

Keys	Action
`Alt` + `↓`, `Alt` + `↑`	Next or previous difference
`Alt` + `Home`, `Alt` + `End`	First or last difference
`Alt` + `PgDn`, `Alt` + `PgUp`	Next or previous conflict
`Alt` + `→`, `Alt` + `←`	Copy the current difference or the selected lines to the right or left. In a folder comparison: copy the selected items
`Ctrl` + `Alt` + `→`, `Ctrl` + `Alt` + `←`	Copy all differences to the right or left
`Alt` + `S`	Set or remove a synchronization point

Table 9. Text editing and navigation

Keys	Action
`Ctrl` + `F`, `F3`, `Shift` + `F3`, `Esc`	Find, find next, find previous, close the find bar
`Ctrl` + `H` (`Cmd` + `Alt` + `F` on macOS)	Find and replace
`Ctrl` + `G`	Go to a line (an offset in a binary comparison)

Keys	Action
Ctrl + Home, Ctrl + End	First or last line
←, →	Make the left or right pane active
F2, Enter	Without a cursor, put it at the end of the current line
Ctrl + A	Select the whole file
Shift + ←, Shift + →, Shift + ↑, Shift + ↓, Shift + PgUp, Shift + PgDn	Select text
Ctrl + C, Ctrl + X	Copy or cut the selection or columns, or the whole line
Ctrl + V	Paste at the cursor, or over the selection or columns
Delete, Backspace	Delete a character, the selection or the columns
Ctrl + ←, Ctrl + → (Option + ←, Option + → on macOS)	Move by word; with Shift, select by word
Ctrl + Backspace, Ctrl + Delete (Option + Backspace, Option + Delete on macOS)	Delete the word before or after the cursor
Alt +drag (Option +drag on macOS)	Select columns
Ctrl +], Ctrl + [, Tab, Shift + Tab	Indent or unindent the selected lines
Ctrl + D	Duplicate the selected lines
Ctrl click, kbd:[Ctrl+Alt↑], Ctrl + Alt + ↑	Add a text cursor (see Several cursors)
Alt + Shift + ↑, Alt + Shift + ↓	Move the selected lines up or down
Ctrl + Z, Ctrl + Y / Ctrl + Shift + Z	Undo, redo
Ctrl + F2, F4, Shift + F4	Set or remove a bookmark, next bookmark, previous bookmark
Alt + C	Collapse unchanged lines
Alt + Z	Word wrap

Table 10. Folder comparison

Keys	Action
Enter	Compare the selected item
Space	Quick preview of the selected file
Ctrl + A	Select all items
→, ←	Expand or collapse a folder in tree mode
Ctrl + F	Find by name
Alt + P	Show or hide the file compare below the folder list

Chapter 17. Troubleshooting

First, read the error message and note the path that it names. Before you repeat a merge or a save, check the destination, the free disk space, the permissions, and whether another program changed the file.

17.1. Installation and startup

macOS refuses to open the app

Try to open it once, then choose **Open Anyway** in **System Settings** → **Privacy & Security**. Or remove the download flag in Terminal: `xattr -dr com.apple.quarantine /Applications/OxideMerge.app`.

macOS asks for access to a folder

macOS asks once before OxideMerge reads Desktop, Documents, Downloads, removable volumes or network volumes. Allow the access, or turn it on later in **System Settings** → **Privacy & Security** → **Files and Folders**.

The AppImage does not start

Make it executable with `chmod +x`. If the message mentions FUSE, start it with `--appimage-extract-and-run`.

The window stays empty or does not open on Linux

OxideMerge draws with OpenGL. Without a working graphics driver, for example in a virtual machine, start it with `LIBGL_ALWAYS_SOFTWARE=1 oxidemerge`. This uses the Mesa software renderer.

File dialogs do not open on Linux

File dialogs use the XDG desktop portal. Install `xdg-desktop-portal` and the portal for your desktop, such as `xdg-desktop-portal-gtk` or `xdg-desktop-portal-kde`.

The file manager entries are missing

On Windows 11, they are under **Show more options**. The entries point to the program that added them. If you moved or updated the program to another folder, add them again from the new program. On Linux, restart the file manager (for example, `nautilus -q`) to see new entries.

The settings are not saved

OxideMerge tells you why when it cannot save the settings. Check that the configuration folder (see Configuration files) is writable and that the disk is not full.

17.2. Editing and saving

A pane cannot be edited

The pane is read-only for one of these reasons:

- The lock in its header is closed.
- It was opened with `--wl`, `--wm` or `--wr`.

- It was extracted from a review snapshot or from a web page tree. A file extracted from an archive compare (see Archives) can be edited; saving it also saves the archive, unless the archive is one that stays read-only, such as a signed package.
- It is a JSON or XML file shown formatted. Turn off **Format JSON and XML before comparing** to edit and merge it.

A file cannot be saved because of its encoding

The encoding cannot represent some characters in the edited text. Undo the change that added these characters, and save again. If you choose another encoding, OxideMerge reads the file again, so copy your edits to another place first.

A save or copy fails, but the file is writable

OxideMerge must also be able to create and replace a temporary file in the same folder (see Saving files). Check the folder permissions, the free space, and programs that lock the file. OxideMerge reports the failure rather than writing over the original. Keep the comparison open until you have saved your edits successfully.

17.3. Comparison results

Files look identical, but their bytes are different

Check the options for whitespace, case, line endings, encoding and comments, and the compare rules. A text comparison can ignore these differences on purpose. To check that files are exactly the same, use a binary comparison, or the **Binary contents** folder method.

Files are missing from a folder comparison

Check the filter, **Skip hidden files and folders**, **Include subfolders** and the **Show** settings. **Find by name** also hides rows. **All differences** uses the filter and the display settings of the folder comparison. It does not include hidden results.

A file is shown as an error

OxideMerge could not read the file. The message names the file. Check its permissions. On macOS, also check the folder access described above.

Automatic folder merge does not start, or merges nothing

Turn on **Include subfolders** and **Include contents of unique subfolders**, and let the comparison finish. If a file changed after the comparison, nothing is merged and the file is listed. If a change fails during the merge, all changes made so far are undone. Refresh the comparison and review the merge plan again.

17.4. Web pages

A web comparison is missing content that a browser shows

OxideMerge compares the HTML from the server, without the content that scripts add. It does not open pages in a browser, so the scripts of the page never run. Content inside frames is not included. The status bar counts the resources that could not be downloaded, and the resources that were skipped because they are on this computer or on a private network.

17.5. Performance

Large files are slow or use a lot of memory

A text comparison keeps the decoded text, the comparison data and the edit history in memory. The memory use depends on the content and on what you do. Binary and image views also load the complete files.

- Valid UTF-8 files of 16 MiB or more load in parts. UTF-16 files and legacy encodings are decoded completely. The text stays editable, and the compare rules work as usual.
- Syntax highlighting is turned off above 50,000 lines or 16 MiB of decoded text.
- Editing, undo, word wrap and unusual compare rules can still use more memory.

To use less memory, use a binary comparison, or compare folders by size and date instead of by content (see Comparison methods).

Chapter 18. Known limitations

- Archives: ZIP-based formats, TAR (plain, gzip, bzip2, xz and Zstandard) and 7z are supported, password-protected ZIP and 7z included. RAR and archives split into parts (`.zip.001`, `.7z.001`) are not supported.
- The text panes are meant for reviewing and merging changes, not as a full editor. The inline view does not edit.
- Unpacker plugins use external commands. WinMerge plugin DLLs and scripts are not supported.
- Web page comparison does not support cookies or logins. A page that needs a login shows its login page. Page scripts do not run, so content that scripts add is missing. Content inside frames is not compared.
- Semantic comparison copies one value at a time, and only when the edit can be confirmed by reading the file again. It does not move array elements or merge all changes at once.
- Folder synchronization works on local folders only, and only when you start it. A renamed file appears as a deletion and a copy.
- SFTP and FTP: FTP's active mode does not go through a proxy, and rarely works from behind NAT. SSH host certificates signed with RSA SHA-1 or ECDSA P-521 are refused. `Match exec` and `Match canonical` in `~/.ssh/config` never match. A question on the terminal can be stopped only with `Ctrl+C`.
- The translations were made with the help of machines and have not been reviewed by native speakers.